電腦視覺
瑕疵檢測教程

完整技術手冊

方法原理 | 數學公式 | 處理流程 | 實際案例
從基礎到工業應用的完整指南

涵蓋：影像前處理、相機校正、光源選型、色彩檢測、Blob 分析、
邊緣檢測、頻率域、深度學習、3D 檢測、即時優化、25 章完整指南

參考：CSDN Halcon 瑕疵檢測系列 / PCB 形態學檢測方法

目錄

第 1 章：影像前處理基礎
1.1 灰度轉換與色彩空間
1.2 降噪與濾波
1.3 對比度增強
1.4 幾何變換（仿射）
第 2 章：閾值分割
2.1 全域閾值（Otsu 法）
2.2 自適應閾值
2.3 多層閾值
第 3 章：形態學運算
3.1 腐蝕與膨脹
3.2 開運算與閉運算
3.3 開閉差分缺陷檢測
3.4 頂帽與黑帽變換
第 4 章：邊緣檢測
4.1 Sobel / Prewitt / Roberts
4.2 Canny 邊緣檢測器
4.3 高斯拉普拉斯（LoG）
第 5 章：Blob 分析與連通域
5.1 連通域標記
5.2 區域特徵提取
5.3 基於特徵的分類
第 6 章：頻率域分析
6.1 傅立葉變換（DFT/FFT）
6.2 高通 / 低通 / 帶通濾波
6.3 紋理缺陷檢測
第 7 章：模板匹配
7.1 歸一化互相關（NCC）
7.2 基於形狀的匹配
7.3 差分影像法
第 8 章：光度立體法
8.1 原理與打光配置
8.2 表面法向量估計
8.3 表面缺陷檢測
第 9 章：量測與擬合
9.1 卡尺工具與亞像素邊緣
9.2 直線/圓/橢圓擬合
9.3 尺寸驗證
第 10 章：機器學習瑕疵分類
10.1 特徵工程
10.2 SVM 分類器
10.3 深度學習異常檢測

第 11 章：變異模型（統計背景）

11.1 訓練階段：均值與變異數模型

11.2 檢測階段：統計比較

第 12 章：完整瑕疵檢測案例

12.1-12.10 10 個工業案例

第 13 章：方法選擇決策指南

13.1 缺陷類型 vs 方法矩陣

13.2 打光建議

13.3 完整決策流程圖

第 14 章：比對、評分與瑕疵影像篩選

14.1 良品比對方法總覽

14.2 瑕疵評分機制

14.3 篩選判定與分級

14.4 瑕疵影像標記與輸出

14.5 批次篩選流程（完整範例）

14.6 常見判定策略與陷阱

第 15 章：相機校正與像素-毫米轉換

15.1 為什麼需要相機校正

15.2 內部參數與畸變係數

15.3 棋盤格校正流程

15.4 像素到毫米轉換

15.5 畸變校正與影像矯正

第 16 章：影像對齊與配準

16.1 為什麼需要對齊

16.2 基於特徵點的對齊

16.3 基於輪廓的對齊

16.4 基於相位相關的對齊

16.5 多尺度配準策略

第 17 章：光源原理與選型

17.1 光源的核心角色

17.2 照明幾何詳解

17.3 光源類型與特性

17.4 瑕疵類型最佳打光

17.5 打光實驗方法論

第 18 章：色彩瑕疵檢測

18.1 色彩空間選擇

18.2 色差計算與閾值

18.3 色彩區域分割

18.4 色彩一致性檢測

18.5 多通道融合檢測

第 19 章：進階輪廓分析

19.1 輪廓層次結構

19.2 輪廓近似與形狀描述

19.3 最小外接形狀

19.4 輪廓距離與相似度

19.5 缺陷輪廓分類策略

第 20 章：深度學習瑕疵檢測進階

20.1 YOLO 目標檢測
20.2 U-Net 語義分割
20.3 PatchCore / PADIM
20.4 遷移學習與微調
20.5 資料增強策略
20.6 模型評估與部署

第 21 章：進階分割方法

21.1 分水嶺分割
21.2 GrabCut 互動分割
21.3 超像素分割（SLIC）
21.4 區域生長分割
21.5 語義分割應用

第 22 章：即時處理與效能優化

22.1 效能瓶頸分析
22.2 影像處理加速
22.3 多執行緒管線化
22.4 GPU 加速
22.5 演算法選擇
22.6 即時系統設計

第 23 章：系統穩健性與邊界案例

23.1 環境變化的影響
23.2 自動曝光與白平衡
23.3 異常輸入處理
23.4 假陽性與假陰性分析
23.5 穩健性測試
23.6 日誌與追溯

第 24 章：3D 檢測技術

24.1 3D 檢測的必要性
24.2 結構光三維重建
24.3 雷射三角測量
24.4 立體視覺
24.5 點雲處理
24.6 3D 瑕疵檢測

第 25 章：參數調校方法論

25.1 系統化調校流程
25.2 ROC 曲線與閾值選擇
25.3 交叉驗證
25.4 敏感度分析
25.5 自動參數搜索
25.6 參數版本管理

第 1 章

影像前處理基礎

影像前處理是所有瑕疵檢測流程的基礎。工業相機拍攝的原始影像包含雜訊、不均勻照明和無關資訊。前處理的目標是將原始影像轉換為能最大化瑕疵可見性、最小化誤檢的形式。

1.1 灰度轉換與色彩空間

多數工業瑕疵檢測使用灰度影像，因為瑕疵通常表現為亮度異常。從彩色轉灰度可將資料從 3 通道減為 1 通道，加速後續處理。

公式：RGB 轉灰度（亮度法）

$$Y = 0.299 * R + 0.587 * G + 0.114 * B$$

權重反映人眼感知靈敏度。綠色權重最高，因為人眼對綠光最為敏感。

公式：HSV 色彩空間轉換

$$H = \arctan2(\sqrt{3}*(G-B),\ 2R-G-B)$$

$$S = 1 - 3*\min(R,G,B)/(R+G+B)$$

$$V = \max(R,G,B)$$

HSV 將顏色（H）與亮度（V）分離，當瑕疵具有獨特顏色特徵時非常有用（如金屬上的鏽蝕、食品上的變色）。

```
[python]
import cv2
import numpy as np

# 讀取彩色影像
img = cv2.imread('part.jpg')

# 轉為灰度
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# 轉為 HSV 用於顏色缺陷檢測
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
h, s, v = cv2.split(hsv)
```

提示

何時使用彩色 vs 灰度：如果瑕疵主要是顏色差異（污漬、鏽蝕、變色），使用 HSV 或 Lab。如果瑕疵是形狀、紋理或亮度差異，灰度即可且更快。

1.2 降噪與濾波

相機感測器會引入雜訊，可能被誤判為瑕疵。濾波在降低雜訊的同時保留邊緣（瑕疵邊界）。關鍵取捨：濾波太少則雜訊殘留；濾波太多則模糊真正的瑕疵。

>> 高斯濾波

公式：二維高斯核

```
G(x,y) = (1/(2*pi*sigma^2)) * exp(-(x^2+y^2)/(2*sigma^2))
```

sigma 控制模糊強度。sigma 越大平滑越多。核大小通常為 3x3、5x5 或 7x7。適用於一般高斯雜訊。

>> 中值濾波

公式：中值濾波

output(x,y) = median{ I(x+i, y+j) : (i,j) in 鄰域 }

非線性濾波器。對椒鹽雜訊效果極佳，邊緣保留能力優於高斯。工業檢測中脈衝雜訊常見時首選。

>> 雙邊濾波

公式：雙邊濾波

```
I_f(x) = (1/W) * SUM[ I(xi)*f_r(|I(xi)-I(x)|)*g_s(|xi-x|) ]
```

結合空間鄰近度（g_s）和亮度相似度（f_r）。在平坦區域平滑，同時保留邊緣。速度較慢但邊緣保留效果最佳。

```
 [python]
# 降噪比較
 gaussian  = cv2.GaussianBlur(gray, (5,5), sigmaX=1.0)
 median    = cv2.medianBlur(gray, 5)
 bilateral = cv2.bilateralFilter(gray, d=9,
                                 sigmaColor=75, sigmaSpace=75)

# 工業應用選擇：
# 椒鹽雜訊 -> 中值濾波
# 高斯雜訊 -> 高斯濾波
# 需保留邊緣 -> 雙邊濾波
```

1.3 對比度增強（直方圖均衡化）

低對比度影像難以區分瑕疵與背景。直方圖均衡化重新分配像素亮度以覆蓋完整範圍 [0, 255]，提升細微瑕疵的可見性。

公式：直方圖均衡化

```
s_k = (L-1) * SUM(j=0..k)[ n_j / N ]
```

s_k = 輸入級別 k 的輸出亮度
L = 亮度級別數 (256)
n_j = 亮度為 j 的像素數
N = 總像素數

本質上是直方圖的累積分佈函數（CDF），縮放到 [0, L-1]。

>> CLAHE - 對比度受限自適應直方圖均衡化

全域直方圖均衡化可能過度放大雜訊。CLAHE
將影像分成小塊分別均衡化，並設定裁剪限制以防雜訊放大。這是工業檢測中首選的方法。

```
 [python]
# 全域直方圖均衡化
 equalized = cv2.equalizeHist(gray)

# CLAHE（推薦用於瑕疵檢測）
 clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
 enhanced = clahe.apply(gray)
```

1.4 幾何變換（仿射變換）

產線上的零件很少在完全相同的位置/方向。仿射變換將被檢測件對齊到參考座標系，使得能與模板進行逐像素比較。

公式：仿射變換矩陣

```
   [x']   [a  b  tx] [x]
   [y'] = [c  d  ty] [y]
   [1 ]   [0  0  1 ] [1]
```

包含：旋轉、縮放、平移、剪切

用於將測試影像對齊（配準）到黃金參考影像。通過匹配特徵點或已知幾何估計。

```
 [python]
# 仿射對齊範例
 pts_src = np.float32([[50,50],[200,50],[50,200]])
 pts_dst = np.float32([[60,40],[210,55],[55,210]])
 M = cv2.getAffineTransform(pts_src, pts_dst)
 aligned = cv2.warpAffine(img, M,
                          (img.shape[1], img.shape[0]))
```

第 2 章

閾值分割

閾值分割將灰度影像轉換為二值影像（瑕疵 vs 背景）。這是最基本的分割技術，也是大多數瑕疵檢測流程的第一步。選擇正確的閾值方法至關重要。

2.1 全域閾值 - Otsu 法

Otsu 法通過最大化類間變異數自動找到最佳閾值。假設直方圖呈雙峰分佈（背景和前景/瑕疵兩個峰值）。

公式：Otsu 類間變異數

```
sigma_B^2(t) = w0(t)*w1(t)*[mu0(t)-mu1(t)]^2

w0(t) = SUM(i=0..t)[p_i]      (類別 0 權重)
w1(t) = SUM(i=t+1..L-1)[p_i]  (類別 1 權重)
mu0   = SUM(i=0..t)[i*p_i]/w0  (類別 0 均值)
mu1   = SUM(i=t+1..L-1)[i*p_i]/w1

最佳閾值：t* = argmax{ sigma_B^2(t) }
```

p_i 是亮度 i 的機率。使類間變異數最大的閾值最能分離兩組。

```
[python]
# Otsu 閾值分割
ret, binary = cv2.threshold(gray, 0, 255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU)
print(f'Otsu 閾值: {ret}')

# 反向二值化（瑕疵為暗色時）
ret, binary_inv = cv2.threshold(gray, 0, 255,
    cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)
```

注意

Otsu 法在直方圖非雙峰時會失效（如均勻紋理，或瑕疵佔影像比例極小時）。此時應使用自適應閾值或手動選擇閾值。

2.2 自適應 / 動態閾值

當影像照明不均勻時（工業環境中非常常見），單一全域閾值會失敗。自適應閾值根據每個像素的鄰域計算局部閾值。

公式：自適應閾值

```
T(x,y) = mean(鄰域(x,y)) - C

Binary(x,y) = 255  若 I(x,y) > T(x,y)
              0    否則
```

C 是常數偏移量。鄰域通常為方形窗口。高斯加權均值優於簡單均值。

```
 [python]
# 自適應閾值
 adapt_gauss = cv2.adaptiveThreshold(
     gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
     cv2.THRESH_BINARY, blockSize=51, C=10)

# 關鍵參數：
# blockSize: 局部鄰域大小（須為奇數）
#  小 blockSize -> 檢測細小瑕疵，雜訊多
#  大 blockSize -> 檢測大瑕疵，可能遺漏小的
# C: 偏移常數
#  較大 C -> 靈敏度降低，誤檢減少
```

2.3 多層閾值

當存在多個類別時（如背景、正常表面、輕微瑕疵、嚴重瑕疵），單一閾值不足。可使用多層 Otsu 或 K-means 聚類。

```
 [python]
# 使用 K-means 多層分割
 Z = gray.reshape((-1,1)).astype(np.float32)
 criteria = (cv2.TERM_CRITERIA_EPS +
             cv2.TERM_CRITERIA_MAX_ITER, 10, 1.0)
 K = 3
 _, labels, centers = cv2.kmeans(
     Z, K, None, criteria, 10,
     cv2.KMEANS_RANDOM_CENTERS)
 seg = centers[labels.flatten()]
 seg = seg.reshape(gray.shape).astype(np.uint8)
```

第 3 章

形態學運算

閾值分割後的二值影像常包含雜訊（小的假陽性區域）和瑕疵區域中的空洞。形態學運算使用結構元素清理二值影像，移除雜訊、填補空洞、分離或連接區域。

3.1 腐蝕與膨脹

公式：腐蝕

```
output(x,y) = min{ I(x+i,y+j) : (i,j) in B }
```

縮小前景物體。移除小雜訊區域。可斷開弱連接的物體。

公式：膨脹

```
output(x,y) = max{ I(x+i,y+j) : (i,j) in B }
```

擴大前景物體。填補小空洞。連接鄰近的斷開區域。

```
 [python]
# 結構元素
 k_rect = cv2.getStructuringElement(
     cv2.MORPH_RECT, (5,5))
 k_ellipse = cv2.getStructuringElement(
     cv2.MORPH_ELLIPSE, (5,5))

# 腐蝕：移除小雜訊
 eroded = cv2.erode(binary, k_rect, iterations=1)
# 膨脹：填補小間隙
 dilated = cv2.dilate(binary, k_rect, iterations=1)
```

3.2 開運算與閉運算

公式：開運算 = 先腐蝕後膨脹

```
A o B = (A erode B) dilate B
```

移除小的前景雜訊，同時保持較大物體的大小和形狀。用於清理閾值分割結果。

公式：閉運算 = 先膨脹後腐蝕

```
A . B = (A dilate B) erode B
```

填補前景物體中的小空洞和間隙。當瑕疵區域有內部空洞需要填補時使用。

```
[python]
# 開運算：移除雜訊
opened = cv2.morphologyEx(binary,
    cv2.MORPH_OPEN, k_rect)
# 閉運算：填補空洞
closed = cv2.morphologyEx(binary,
    cv2.MORPH_CLOSE, k_rect)
# 常見流程：先開後閉
cleaned = cv2.morphologyEx(binary,
    cv2.MORPH_OPEN, k_rect)
cleaned = cv2.morphologyEx(cleaned,
    cv2.MORPH_CLOSE, k_rect)
```

3.3 開閉差分缺陷檢測（PCB 方法）

利用原始影像與形態學處理結果的差異來檢測瑕疵。此方法廣泛用於 PCB 檢測：

- 開運算差分 = 原圖 - 開運算(原圖) -> 檢測亮色突出物（毛刺、短路、多餘銅）
- 閉運算差分 = 閉運算(原圖) - 原圖 -> 檢測暗色缺口（斷路、缺損、鼠咬痕）
- 所有缺陷 = 開運算缺陷 + 閉運算缺陷

公式：形態學缺陷提取

Defect_bright = I - Opening(I, B) (毛刺/短路)

Defect_dark = Closing(I, B) - I (斷路/缺口)

Defect_all = Defect_bright + Defect_dark

然後: threshold(Defect_all, T) -> 二值缺陷遮罩

開運算會平滑掉亮色突出物，閉運算會填補暗色缺口。差異恰好隔離這些異常。

```
[python]
# PCB 開閉差分缺陷檢測
el = cv2.getStructuringElement(
    cv2.MORPH_RECT, (5,5))

# 開運算差分 -> 亮色缺陷
opened = cv2.morphologyEx(gray,
    cv2.MORPH_OPEN, el)
defect_bright = cv2.subtract(gray, opened)

# 閉運算差分 -> 暗色缺陷
closed = cv2.morphologyEx(gray,
    cv2.MORPH_CLOSE, el)
defect_dark = cv2.subtract(closed, gray)

# 合併所有缺陷
defect_all = cv2.add(defect_bright, defect_dark)
_, mask = cv2.threshold(defect_all, 58, 255,
    cv2.THRESH_BINARY)
```

提示

最佳化：原始方法可能在非線路區域產生假陽性。解決方案：用較大閉運算核（15x15）提取非線路區域，反轉後膨脹（11x11），然後從開運算結果中扣除，可大幅減少誤檢。

3.4 頂帽與黑帽變換

公式：頂帽變換

```
TopHat(I) = I - Opening(I)
```

提取暗背景上的亮色細節（小亮點瑕疵）。用於檢測均勻暗色表面上的亮色刮痕或斑點。

公式：黑帽變換

```
BlackHat(I) = Closing(I) - I
```

提取亮背景上的暗色細節（小暗點瑕疵）。用於檢測亮色表面上的暗色斑點、凹坑或孔洞。

```
 [python]
# 頂帽：暗背景上的亮色缺陷
 tophat = cv2.morphologyEx(gray,
     cv2.MORPH_TOPHAT,
     cv2.getStructuringElement(
         cv2.MORPH_RECT, (15,15)))

# 黑帽：亮背景上的暗色缺陷
 blackhat = cv2.morphologyEx(gray,
     cv2.MORPH_BLACKHAT,
     cv2.getStructuringElement(
         cv2.MORPH_RECT, (15,15)))
```

第 4 章

邊緣檢測

邊緣是不同亮度區域之間的邊界。在瑕疵檢測中，邊緣揭示裂紋、刮痕、缺損邊界和形狀變形。邊緣檢測計算影像亮度的梯度。

4.1 Sobel / Prewitt / Roberts 運算子

公式：Sobel 運算子（3x3）

```
Gx = [-1  0 +1]   Gy = [-1 -2 -1]
     [-2  0 +2]        [ 0  0  0]
     [-1  0 +1]        [+1 +2 +1]
```

梯度幅值: G = sqrt(Gx^2 + Gy^2)

梯度方向: theta = arctan(Gy / Gx)

Gx 檢測垂直邊緣，Gy 檢測水平邊緣。中心行/列的 2x 加權提供輕微平滑。

```
[python]
# Sobel 邊緣檢測
sobelx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
sobely = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
mag = np.sqrt(sobelx**2 + sobely**2)
mag = np.uint8(np.clip(mag, 0, 255))
```

4.2 Canny 邊緣檢測器

Canny 檢測器是邊緣檢測的黃金標準。通過多階段處理產生細而連續的邊緣：

1. 高斯平滑（降噪）
2. 計算梯度幅值和方向（Sobel）
3. 非極大值抑制（邊緣細化為 1 像素寬）
4. 雙閾值分類（強邊緣/弱邊緣/無邊緣）
5. 滯後邊緣追蹤（將弱邊緣連接到強邊緣）

公式：Canny 雙閾值

強邊緣: G(x,y) >= T_high

弱邊緣: T_low <= G(x,y) < T_high

無邊緣: G(x,y) < T_low

建議比例: T_high / T_low = 2:1 或 3:1

強邊緣永遠保留。弱邊緣僅在連接到強邊緣時保留（滯後追蹤）。

```
 [python]
# Canny 邊緣檢測
 edges = cv2.Canny(gray, 50, 150)

# 自動閾值（基於中位數）
 v = np.median(gray)
 lo = int(max(0, 0.67 * v))
 hi = int(min(255, 1.33 * v))
 edges_auto = cv2.Canny(gray, lo, hi)
```

4.3 高斯拉普拉斯（LoG）

公式：高斯拉普拉斯

```
LoG = Laplacian( Gaussian(I) )
    = d^2G/dx^2 + d^2G/dy^2
```

在零交叉點檢測邊緣。各向同性（檢測所有方向的邊緣）。sigma 控制檢測的邊緣尺度。

第 5 章

Blob 分析與連通域

Blob 分析是工業瑕疵檢測中最常用的方法。閾值分割後，連通的白色（或黑色）區域被識別為 'blob'。計算每個 blob 的幾何和亮度特徵，與驗收標準比較以分類為瑕疵或正常。

5.1 連通域標記

連通域標記為每組連通的前景像素分配唯一標籤。
- 4-連通：水平/垂直方向連接的像素
- 8-連通：所有 8 個方向連接（含對角線）
8-連通更常用，因為它更好地保留對角特徵。

```
 [python]
# 連通域分析
 num, labels, stats, centroids = \
     cv2.connectedComponentsWithStats(
         binary, connectivity=8)

# stats 欄位: [left, top, w, h, area]
# 按面積過濾以移除雜訊
 min_area = 100
for i in range(1, num):  # 跳過 0（背景）
     area = stats[i, cv2.CC_STAT_AREA]
     if area < min_area:
         labels[labels == i] = 0
```

5.2 區域特徵提取

識別 blob 後，提取特徵用於分類。瑕疵檢測中的關鍵特徵：

公式：面積與周長

面積 = 區域中的像素數

周長 = 邊界像素數

```
Area = cv2.contourArea(contour)
Perimeter = cv2.arcLength(contour, True)
```

面積區分大瑕疵與雜訊。周長有助於識別細長瑕疵（刮痕）。

公式：圓度（緊密度）

```
Circularity = 4 * pi * Area / Perimeter^2
```

完美圓形: Circularity = 1.0

細長形狀: Circularity << 1.0

圓度區分圓形瑕疵（凹坑、孔洞）和細長瑕疵（刮痕、裂紋）。

公式：長寬比、填充度、實心度

```
AspectRatio = Width / Height
Extent      = Area / BoundingRect_Area
Solidity    = Area / ConvexHull_Area
```

AspectRatio > 3：可能是刮痕或裂紋。低 Solidity：不規則形狀（破損邊緣）。

```python
[python]
# 輪廓特徵提取
contours, _ = cv2.findContours(
    binary, cv2.RETR_EXTERNAL,
    cv2.CHAIN_APPROX_SIMPLE)

for cnt in contours:
    area = cv2.contourArea(cnt)
    peri = cv2.arcLength(cnt, True)
    if peri == 0: continue
    circ = 4*np.pi*area / (peri**2)
    x,y,w,h = cv2.boundingRect(cnt)
    ar = w/h if h > 0 else 0
    hull = cv2.convexHull(cnt)
    sol = area / cv2.contourArea(hull)

 # 分類規則
 if area < 50:     typ = '雜訊'
 elif circ > 0.8:  typ = '凹坑/孔洞'
 elif ar > 4:      typ = '刮痕'
 else:             typ = '不規則瑕疵'
```

5.3 基於特徵的瑕疵分類

特徵	低值含義	高值含義	對應瑕疵類型
面積	< 50 px（雜訊）	> 5000 px（重大）	雜訊 / 重大瑕疵
圓度	< 0.3（細長）	> 0.8（圓形）	刮痕 / 孔洞
長寬比	~1.0（方形）	> 5.0（細長）	圓形 / 線性
實心度	< 0.5（不規則）	> 0.9（實心）	不規則 / 實心

第 6 章

頻率域分析

頻率域揭示空間域中不可見的週期性模式。織物、網格或電路板走線等紋理會產生強烈的頻率峰值。瑕疵破壞這些模式，在頻譜中表現為異常。

6.1 傅立葉變換（DFT/FFT）

公式：二維離散傅立葉變換

```
F(u,v) = SUM_x SUM_y f(x,y) *
          e^(-j*2*pi*(ux/M + vy/N))
```

幅值: |F(u,v)| = sqrt(Re^2 + Im^2)

相位: phi(u,v) = arctan(Im / Re)

低頻（靠近中心）= 平滑區域、整體照明。高頻（遠離中心）= 邊緣、雜訊、細節。週期性模式 = 頻譜中的明顯峰值。

```
[python]
# 傅立葉變換用於紋理分析
dft = cv2.dft(np.float32(gray),
              flags=cv2.DFT_COMPLEX_OUTPUT)
dft_shift = np.fft.fftshift(dft)
mag = cv2.magnitude(
    dft_shift[:,:,0], dft_shift[:,:,1])
mag_log = np.log1p(mag) # 對數尺度可視化
```

6.2 高通 / 低通 / 帶通濾波

公式：高斯低通濾波器（頻率域）

```
H_LP(u,v) = exp(-D(u,v)^2 / (2*D0^2))
D(u,v) = sqrt((u-M/2)^2 + (v-N/2)^2)
```

D0 = 截止頻率

移除高頻雜訊，保留整體結構。

公式：高斯高通濾波器

```
H_HP(u,v) = 1 - H_LP(u,v)
```

增強邊緣和細節。用於檢測紋理表面上的刮痕。

公式：帶阻/帶通濾波器

帶阻：移除特定頻率帶

帶通：H_BP = 1 - H_BR（僅保留該頻帶）

帶阻用於移除週期性模式（如網格），以揭示隱藏在模式中的瑕疵。

```
 [python]
# 頻率域高通濾波
 rows, cols = gray.shape
 crow, ccol = rows//2, cols//2
D0 = 30 # 截止頻率
 u = np.arange(rows).reshape(-1,1) - crow
 v = np.arange(cols).reshape(1,-1) - ccol
 D = np.sqrt(u**2 + v**2)
 H_hp = 1 - np.exp(-D**2 / (2*D0**2))

# 套用濾波
 dft_f = dft_shift.copy()
 dft_f[:,:,0] *= H_hp
 dft_f[:,:,1] *= H_hp
# 反傅立葉
 f_is = np.fft.ifftshift(dft_f)
 result = cv2.idft(f_is)
 result = cv2.magnitude(
     result[:,:,0], result[:,:,1])
```

6.3 紋理缺陷檢測應用

案例研究：LCD 面板雲狀缺陷（Mura）檢測

LCD 面板具有均勻背光。雲狀亮度不均（Mura 缺陷）是低頻變化。

流程：
1. 顯示均勻白色測試圖案
2. 計算 FFT
3. 套用高通濾波器（D0=5~10）移除直流分量
4. IFFT 轉回空間域
5. 正規化並拉伸對比度
6. 閾值分割 + Blob 分析

替代方案：擬合二維多項式曲面（模擬理想背光），從原始影像中減去，殘差即為 Mura 缺陷。

案例研究：織布紋理缺陷檢測

織布具有規律的編織模式，在頻譜中產生強週期峰值。瑕疵（破洞、斷線、污漬）破壞此模式。

流程：
1. 計算織布影像的 FFT
2. 識別編織頻率的主要峰值
3. 建立陷波濾波器抑制這些峰值
4. 套用 IFFT
5. 結果僅包含非週期性內容（瑕疵）
6. 閾值分割 + Blob 分析

第 7 章

模板匹配

模板匹配將測試影像與已知良品參考（模板）進行比較。兩者之間的差異指示潛在瑕疵。對於外觀一致且可重複的產品（PCB 板、印刷標籤等）特別有效。

7.1 歸一化互相關（NCC）

公式：歸一化互相關

```
NCC(x,y) =
 SUM[(T(i,j)-T_mean)*(I(x+i,y+j)-I_mean)]
 / sqrt[SUM(T-T_mean)^2 * SUM(I-I_mean)^2]

結果範圍: [-1, +1]
 +1 = 完美匹配
  0 = 無相關
 -1 = 反向匹配
```

用於在測試影像中定位模板。對線性亮度變化具有不變性。

```
[python]
# 模板匹配
template = cv2.imread('golden_ref.jpg', 0)
result = cv2.matchTemplate(
    gray, template, cv2.TM_CCOEFF_NORMED)
_, max_val, _, max_loc = cv2.minMaxLoc(result)
```

7.2 基於形狀的匹配

基於形狀的匹配使用邊緣/梯度資訊而非原始像素值，對照明變化和輕微表面變化更具魯棒性。這是 Halcon find_shape_model 使用的方法。

流程：

1. 從參考零件建立形狀模型（提取邊緣）
2. 對每個測試影像搜尋形狀模型
3. 獲取找到實例的姿態（位置、角度、縮放）
4. 使用姿態將測試影像對齊到參考
5. 計算逐像素差異以檢測瑕疵

7.3 差分影像法

公式：絕對差分

```
Diff(x,y) = |I_test(x,y) - I_ref(x,y)|
```

瑕疵遮罩: Diff(x,y) > 閾值

對齊準確時簡單有效。對未對齊敏感——即使 1 像素偏移也會產生假邊緣。

```
[python]
# 差分瑕疵檢測
# 1. 對齊測試影像到參考
# 2. 計算絕對差分
diff = cv2.absdiff(aligned_test, reference)

# 3. 閾值分割
_, mask = cv2.threshold(
    diff, 30, 255, cv2.THRESH_BINARY)

# 4. 形態學清理
k = cv2.getStructuringElement(
    cv2.MORPH_ELLIPSE, (5,5))
mask = cv2.morphologyEx(
    mask, cv2.MORPH_OPEN, k)
mask = cv2.morphologyEx(
    mask, cv2.MORPH_CLOSE, k)

# 5. 分析瑕疵區域
contours, _ = cv2.findContours(
    mask, cv2.RETR_EXTERNAL,
    cv2.CHAIN_APPROX_SIMPLE)
```

第 8 章

光度立體法

光度立體法使用從不同方向照明拍攝的多張影像來重建表面法向量。這揭示了在單方向照明下不可見或模糊的表面瑕疵（凹痕、凸起、刮痕）。

8.1 原理與打光配置

配置：相機固定在物體上方。多個光源（通常 4~8 個）位於物體周圍不同角度。每次開啟一個光源並拍攝一張影像。每個像素的亮度隨該點的表面法向量而變化。

公式：朗伯反射模型

```
I(x,y) = rho(x,y) * max(0, n(x,y) . L)
```

I = 觀測亮度
rho = 表面反照率（反射率）
n = 表面法向量 [nx, ny, nz]
L = 光源方向向量 [lx, ly, lz]
(.) = 點積

每張影像為每個像素提供一個方程式。3+ 張不同光源方向的影像即可求解法向量 n(x,y)。

8.2 表面法向量估計

公式：最小二乘法向量估計

對 k 張影像，光源方向 L1..Lk:

```
[I1]   [L1x L1y L1z] [rho*nx]
[I2] = [L2x L2y L2z] [rho*ny]
[..]   [..  ..  .. ] [rho*nz]

=> I = L * g
```

解: g = (L^T * L)^(-1) * L^T * I
反照率: rho = |g|, 法向量: n = g/|g|

需要至少 3 個非共面的光源方向。更多光源 = 更穩健的估計。

```
 [python]
# 光度立體法實作
# images: k 張灰度影像列表
# lights: kx3 光源方向矩陣
 H, W = images[0].shape
 I = np.vstack([img.flatten().astype(np.float64)
                for img in images])  # (k, N)
 L = np.array(lights)  # (k, 3)

# 求解
 g = np.linalg.inv(L.T @ L) @ L.T @ I

# 反照率和法向量
 albedo = np.linalg.norm(g, axis=0)
 albedo = albedo.reshape(H, W)
 normals = g / (np.linalg.norm(
     g, axis=0, keepdims=True) + 1e-8)
```

8.3 利用法向量檢測表面瑕疵

表面瑕疵導致法向量局部變化：
- 刮痕：法向量跨越刮痕處急劇偏轉
- 凹痕：法向量向凹痕中心傾斜
- 凸起：法向量向外傾斜

檢測方法：計算法向量場的梯度（曲率），閾值分割以找到異常曲率區域。

第 9 章

量測與擬合

基於量測的瑕疵檢測驗證幾何屬性：尺寸、直線度、圓度、角度和距離。如果測量值超出公差範圍，則零件為不良品。

9.1 卡尺工具與亞像素邊緣檢測

卡尺工具在邊緣上投射一條線（或弧線），沿該線精確找到邊緣位置。沿著邊緣放置多個卡尺，可以亞像素精度量測邊緣輪廓。

公式：亞像素邊緣檢測（一維梯度插值）

給定 3 個連續梯度值 g[i-1], g[i], g[i+1]
其中 g[i] 為最大值:

```
offset = (g[i-1]-g[i+1]) /
         (2*(g[i-1]-2*g[i]+g[i+1]))
edge_pos = i + offset
```

梯度峰值附近的拋物線插值。可達到約 0.1 像素精度。

9.2 直線 / 圓 / 橢圓擬合

公式：最小二乘圓擬合

最小化: SUM[(sqrt((xi-cx)^2+(yi-cy)^2)-R)^2]

線性化後可用最小二乘法求解 (cx, cy, R)

將圓擬合到邊緣點。最大/最小半徑偏差指示圓度誤差。用於檢測孔洞、銷釘和墊圈。

```
 [python]
# OpenCV 擬合
 pts = np.array(edge_pts, dtype=np.float32)

# 最小外接圓
 (cx,cy), r = cv2.minEnclosingCircle(pts)

# 擬合橢圓（需 >= 5 點）
 if len(pts) >= 5:
     ellipse = cv2.fitEllipse(pts)

# 擬合直線
 line = cv2.fitLine(
     pts, cv2.DIST_L2, 0, 0.01, 0.01)
```

9.3 尺寸驗證

擬合幾何基元後，量測關鍵尺寸並與規格比較：
- 兩邊緣間距（寬度、間隙）
- 孔洞/銷釘直徑

- 表面間角度
- 直線度、平面度、圓度

任何尺寸超出公差範圍 → 不良品。

第 10 章

機器學習瑕疵分類

當基於規則的方法（閾值 +
特徵）變得過於複雜或脆弱時，機器學習分類器可以從訓練數據中學習區分瑕疵類型。兩種方法：傳統 ML（手工特徵）和深度學習（自動學習特徵）。

10.1 特徵工程

對於傳統 ML，特徵品質決定分類準確度。常用瑕疵分類特徵：

- 幾何特徵：面積、周長、圓度、長寬比、實心度、Hu 矩

•

公式：GLCM（灰度共生矩陣）

P(i,j|d,theta): 距離 d、角度 theta 處
像素對亮度為 (i,j) 的計數

對比度 = SUM[(i-j)^2 * P(i,j)]
相關性 = SUM[(i-mu_i)(j-mu_j)*P/(s_i*s_j)]
能量 = SUM[P(i,j)^2]
均勻性 = SUM[P(i,j) / (1+|i-j|)]

這 4 個 Haralick 特徵捕捉紋理屬性。對比度衡量局部變化；能量衡量均勻性。

10.2 SVM 分類器

公式：SVM 決策函數

```
f(x) = sign(SUM[a_i*y_i*K(x_i,x)] + b)
```

常用核函數:
線性: K(x,y) = x . y
RBF: K(x,y) = exp(-gamma*|x-y|^2)
多項式: K(x,y) = (gamma*x.y + r)^d

SVM 找到最大化類間間距的最佳超平面。RBF 核是瑕疵分類的預設選擇。

```
[python]
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler

# 正規化特徵
scaler = StandardScaler()
X_s = scaler.fit_transform(features)

# 訓練 SVM
svm = SVC(kernel='rbf', C=10, gamma='scale')
svm.fit(X_train, y_train)
acc = svm.score(X_test, y_test)
print(f'準確率: {acc:.2%}')
```

10.3 深度學習：CNN 異常檢測

>> 1. 分類 CNN

訓練 CNN 將影像區塊分類為「OK」或「瑕疵」。需要標註訓練數據。架構：通常是小型網路（3~5 個卷積層 + 全連接層），在裁剪區塊上訓練。

>> 2. 異常檢測（自編碼器）

僅用良品樣本訓練自編碼器。網路學習重建正常影像。當輸入瑕疵影像時，瑕疵區域的重建誤差會很高。

公式：自編碼器異常分數

```
anomaly(x,y) = |I_input - I_reconstructed|^2
瑕疵判定: anomaly > 閾值
```

不需要瑕疵訓練樣本！僅需要一組良品影像用於訓練。

```
[python]
# 異常檢測自編碼器（PyTorch）
class DefectAE(nn.Module):
    def __init__(self):
        super().__init__()
        self.enc = nn.Sequential(
          nn.Conv2d(1,32,3,stride=2,padding=1),
          nn.ReLU(),
          nn.Conv2d(32,64,3,stride=2,padding=1),
          nn.ReLU())
        self.dec = nn.Sequential(
          nn.ConvTranspose2d(64,32,3,
            stride=2,padding=1,output_padding=1),
          nn.ReLU(),
          nn.ConvTranspose2d(32,1,3,
            stride=2,padding=1,output_padding=1),
          nn.Sigmoid())
    def forward(self, x):
        return self.dec(self.enc(x))

# 訓練：僅對良品影像最小化 MSE
# 推論：高 MSE 區域 = 瑕疵
```

第 11 章

變異模型（統計背景）

變異模型是 Halcon 等系統使用的統計方法。利用多張訓練影像計算每個位置像素值的均值和標準差來建模。檢測時，顯著偏離模型的像素被標記為瑕疵。

11.1 訓練階段：均值與變異數模型

公式：變異模型訓練

對每個像素位置 (x,y)，從 N 張訓練影像:

```
mean(x,y) = (1/N) * SUM(k=1..N)[I_k(x,y)]

var(x,y) = (1/(N-1)) *
  SUM(k=1..N)[(I_k(x,y) - mean(x,y))^2]

std(x,y) = sqrt(var(x,y))
```

需要 15~50 張良品影像。影像必須在訓練前對齊（相同位置/方向）。

```
[python]
# 變異模型訓練
import glob
train_imgs = []
for f in sorted(glob.glob('train/*.png')):
    img = cv2.imread(f, 0).astype(np.float64)
    train_imgs.append(img)

stack = np.array(train_imgs)  # (N, H, W)
mean_m = np.mean(stack, axis=0)
std_m = np.std(stack, axis=0, ddof=1)
std_m = np.maximum(std_m, 5.0)  # 避免除零
```

11.2 檢測階段：統計比較

公式：變異模型瑕疵檢測

```
z(x,y) = |I_test(x,y) - mean(x,y)| / std(x,y)
```

瑕疵判定: z(x,y) > k (通常 k = 3.0~5.0)

k=3: 99.7% 正常像素在範圍內

k=4: 99.99% 正常像素在範圍內

k 越高 = 假陽性越少但可能遺漏細微瑕疵。k 越低 = 越敏感但假陽性越多。

```
 [python]
# 變異模型檢測
 test = cv2.imread('test.png', 0)
 test = test.astype(np.float64)

# 計算 z-score
 z = np.abs(test - mean_m) / std_m

# 閾值分割
 k = 3.5
 mask = (z > k).astype(np.uint8) * 255

# 形態學清理
 kern = cv2.getStructuringElement(
     cv2.MORPH_ELLIPSE, (5,5))
 mask = cv2.morphologyEx(
     mask, cv2.MORPH_OPEN, kern)
 mask = cv2.morphologyEx(
     mask, cv2.MORPH_CLOSE, kern)
```

第 12 章

完整瑕疵檢測案例

本章展示 10 個實際瑕疵檢測場景，每個都包含從影像擷取到瑕疵分類的完整流程。

12.1 案例 1：餅乾破損檢測（Blob 分析）

案例研究：餅乾品質檢測

檢測輸送帶上破碎、缺角或裂開的餅乾。

瑕疵特徵：缺損材料改變形狀/面積。

流程：
1. 前處理：灰度轉換，高斯模糊（5x5）
2. 分割：Otsu 閾值分離餅乾與輸送帶
3. 形態學：開運算（去碎屑），閉運算（填小隙）
4. 特徵提取：面積、圓度、實心度
5. 判定規則：
   OK：面積>8000 且 圓度>0.75 且 實心度>0.90
   破損：否則

關鍵：實心度是最具區分力的特徵——破損邊緣產生凹陷，降低面積/凸包比。

12.2 案例 2：瓶蓋裂紋檢測（邊緣檢測）

案例研究：瓶蓋完整性檢查

檢測塑膠瓶蓋上的裂紋和斷裂。

流程：
1. ROI 提取：用霍夫圓檢測定位瓶蓋
2. 前處理：CLAHE 增強對比，中值濾波降噪
3. 邊緣檢測：Canny（低閾值 30, 80）
4. 遮罩已知邊緣區域（瓶蓋邊緣、文字）
5. 計算瓶蓋本體區域的殘餘邊緣像素
6. 邊緣數 > 閾值 → 裂紋

關鍵：使用良品瓶蓋的期望邊緣遮罩，抑制正常邊緣，僅隔離裂紋邊緣。

12.3 案例 3：PCB 缺陷檢測（模板差分 + 形態學）

案例研究：PCB 板品質控制

檢測缺件、焊橋、斷線。

方法 A — 模板差分：
1. 形狀匹配對齊測試 PCB 到黃金參考
2. 仿射變換對齊
3. 差分：diff = |test - ref|
4. 自適應閾值 + 形態學清理
5. 按位置分類瑕疵

方法 B — 形態學開閉差分（無需參考影像）：
1. 開運算差分 → 亮色缺陷（毛刺、短路）
2. 閉運算差分 → 暗色缺陷（斷路、缺口）
3. 合併並閾值分割（T=58）
4. 15x15 閉運算遮罩去除非線路區假陽性

PCB 缺陷類別：缺孔、鼠咬痕、斷路、短路、毛刺、多餘銅。

12.4 案例 4：LCD 雲狀缺陷（頻率域）

案例研究：LCD 面板 Mura 檢測

檢測 LCD 螢幕上的非均勻亮度區域。

流程：
1. 顯示均勻白色測試圖案
2. FFT 計算
3. 高通濾波移除直流分量
4. IFFT 轉回空間域
5. 正規化 + 閾值 + Blob 分析

替代：擬合 2D 多項式曲面模擬理想背光，殘差即為瑕疵。

12.5 案例 5：藥丸檢測（形狀 + 顏色）

案例研究：製藥藥丸品質檢測

檢測缺角、顏色錯誤、形狀異常、污染。

流程：
1. HSV 閾值分割藥丸顏色
2. 形狀驗證：橢圓擬合 + 圓度/長寬比
3. 表面檢測：灰度均值/標準差異常
4. 印字驗證：NCC < 0.7 → 印字缺失/模糊

判定矩陣：
- 顏色錯誤：H 通道超出範圍
- 缺角：圓度 < 0.85 或面積過小
- 污染：局部亮度異常 > 3 std
- 印字異常：NCC < 0.7

12.6 案例 6：金屬表面刮痕（光度立體法）

案例研究：拋光金屬表面檢測

檢測拋光金屬零件上的細微刮痕。

挑戰：刮痕在普通照明下幾乎無對比度。

流程：
1. 拍攝 4 張方向光影像（0/90/180/270 度）
2. 光度立體法計算表面法向量
3. 曲率圖：|d(nx)/dx| + |d(ny)/dy|
4. 自適應閾值分割曲率圖
5. Blob 分析：長度和方向

嚴重程度分級：
- 輕微：長度<2mm, 深度<0.01mm
- 重大：長度>5mm 或 深度>0.05mm
- 致命：A 級表面區域中的任何刮痕

12.7 案例 7：塑膠網孔驗證（Blob + 形態學）

案例研究：塑膠網格檢測

驗證網格中所有孔洞是否存在且形狀正確。

流程：
1. 前處理：高斯模糊 + CLAHE
2. Otsu 閾值（孔洞為暗色）
3. 反轉使孔洞為前景
4. 開運算移除雜訊
5. 連通域標記每個孔洞
6. 每孔特徵：面積、圓度、質心
7. 瑕疵判定：
   - 缺孔：期望位置無 blob
   - 堵孔：面積遠小於預期
   - 變形孔：圓度低於閾值
   - 多餘孔：非預期位置出現 blob

12.8 案例 8：吸盤瑕疵（變異模型）

案例研究：橡膠吸盤品質檢測

檢測氣泡、撕裂、變形。

為何用變異模型：橡膠有自然紋理變化，固定閾值不可靠。

流程：
1. 訓練：拍攝 30 張對齊良品影像
2. 計算 mean_model 和 std_model
3. 測試：
   a. 形狀匹配對齊
   b. 計算 z-score
   c. 閾值 k=3.5
   d. 形態學清理
   e. 按區域分類：
      邊緣區 → 變形/撕裂
      本體區 → 氣泡/污染
      中心區 → 凹陷缺陷

12.9 案例 9：織布紋理缺陷（頻率域）

案例研究：編織布料檢測

檢測破洞、斷線、污漬和編織不規則。

挑戰：規律編織模式主導影像，瑕疵隱藏其中。

流程：
1. FFT 分析
2. 識別編織頻率的主要峰值
3. 建立陷波濾波器抑制這些峰值
4. IFFT：結果僅含非週期性內容
5. 閾值 + Blob 分析

缺陷類型：
- 破洞：濾波後的大暗色 blob
- 污漬：大亮色 blob
- 斷線：細長 blob
- 編織偏移：帶狀高能量

12.10 案例 10：焊縫檢測（卡尺量測）

案例研究：自動焊縫品質驗證

量測焊縫寬度、檢測間隙、驗證焊道輪廓。

流程：
1. ROI：提取焊縫區域
2. Canny 邊緣檢測焊縫兩側
3. 卡尺量測：
   - 50~100 條垂直卡尺線
   - 每條量測左右邊緣間距
4. 輪廓分析：均值、標準差、最小/最大寬度
5. 邊緣直線擬合：直線度偏差

缺陷分類：
- 底切：寬度 < 最小規格
- 過量：寬度 > 最大規格
- 間隙：寬度=0（缺焊）
- 波浪：邊緣直線度 > 閾值
- 氣孔：焊縫中的小暗點

第 13 章

方法選擇決策指南

選擇正確的方法取決於缺陷類型、產品特性和可用硬體。本章提供系統化的方法選擇指南。

13.1 缺陷類型 vs 方法矩陣

缺陷類型	主要方法	替代方法	關鍵特徵
缺損材料	Blob 分析	模板差分	面積/實心度
裂紋/斷裂	邊緣檢測	光度立體	邊緣密度
刮痕	頂帽/光度立體	頻率域	圓度低
污漬/變色	顏色閾值	Blob 分析	色相/飽和度
表面凹坑	Blob+圓度	光度立體	圓度>0.8
尺寸偏差	卡尺/擬合	Blob 分析	量測值
紋理異常	頻率域	變異模型	頻譜變化
週期性圖案	FFT+陷波	模板差分	頻率峰值
表面拓撲	光度立體	結構光	法向量
隨機變異	變異模型	自編碼器	Z-score
複雜/多種	ML 分類器	深度學習	學習特徵

13.2 打光與設置建議

產品類型	打光方式	相機	方法
平面均勻	漫射圓頂	面掃描	閾值+Blob
反射金屬	暗場照明	面掃描	光度/邊緣
透明材料	背光照明	面掃描	Blob 分析
紋理表面	漫射+角度	面掃描	頻率/變異
3D 特徵	結構光	面掃描	量測
移動輸送	頻閃+線光	線掃描	任何+同步
多面檢測	多方向光	面掃描	光度立體
精細細節	同軸/環形	微距鏡頭	邊緣/模板

13.3 完整決策流程圖

按照以下流程選擇最佳方法：

```
[決策流程圖]
步驟 1：瑕疵是形狀/尺寸異常嗎？
 是 -> 已知良品參考？
     是 -> 模板匹配 + 差分
     否 -> Blob 分析 + 特徵分類
 否 -> 步驟 2

步驟 2：瑕疵是表面/紋理異常嗎？
 是 -> 紋理是週期性/規律的？
     是 -> 頻率域（FFT + 陷波）
     否 -> 表面反光？
         是 -> 光度立體法
         否 -> 變異模型
 否 -> 步驟 3

步驟 3：瑕疵與尺寸/幾何相關嗎？
 是 -> 卡尺 + 擬合 + 量測
 否 -> 步驟 4

步驟 4：瑕疵是顏色/污漬異常嗎？
 是 -> HSV 色彩分割 + Blob
 否 -> 步驟 5

步驟 5：瑕疵類型多且難以定義規則？
 是 -> 有標註數據？
     是 -> CNN / SVM
     否 -> 異常檢測（自編碼器）
 否 -> 結合上述多種方法
```

通用流程（適用大多數場景）：

```
[通用流程]
1. 影像擷取
  - 選擇適合缺陷類型的打光
  - 設定曝光、增益、對焦

2. 前處理
  - 降噪（高斯/中值/雙邊）
  - 對比度增強（CLAHE）
  - ROI 提取 + 對齊

3. 分割
  - 閾值 / 邊緣檢測 /
    頻率濾波 / 統計比較

4. 後處理
  - 形態學（先開後閉）
  - 連通域標記 + 小區域移除

5. 特徵提取
  - 幾何/亮度/位置/形狀

6. 分類決策
  - 規則式 / ML式
  - OK / 輕微 / 重大 / 報廢
```

7. 報告
   - 瑕疵位置標記
   - 類型與嚴重程度
   - 統計分析

13.4 快速參考：方法選擇總覽

情境	推薦方法	原因
有良品參考影像	模板差分	直接比較
僅形狀瑕疵	Blob + 特徵	快速且規則簡單
細微表面瑕疵	光度立體法	揭示表面拓撲
週期性紋理	FFT 濾波	移除背景模式
自然變異產品	變異模型	統計自適應
多種瑕疵類型	ML 分類器	自動學習規則
無瑕疵樣本	自編碼器	僅學習正常
尺寸檢查	卡尺 + 擬合	精確量測
顏色缺陷	HSV 分割	隔離顏色
未知缺陷	異常檢測	捕捉任何異常

提示

實務上，最佳結果來自結合 2~3 種方法。例如：Blob 分析檢測大瑕疵 + 變異模型檢測細微瑕疵 + 量測驗證尺寸。永遠從最簡單的方法開始（閾值 + Blob），只在需要時增加複雜度。

第 14 章

比對、評分與瑕疵影像篩選

前面章節介紹了如何檢測瑕疵。但在實際產線中，檢測只是第一步。完成檢測後，還需要：

1. 將測試件與良品進行比對，量化差異程度
2. 根據差異計算瑕疵分數
3. 依據分數和規則篩選出不良品
4. 標記瑕疵位置並保存分類影像
5. 產生檢測報告供後續分析

本章涵蓋這整個「後處理 → 判定 → 輸出」的完整流程。

14.1 良品比對方法總覽

比對的核心思想是：建立「良品應該長什麼樣」的基準，然後量化測試件與基準的偏離程度。根據基準的表示方式，分為三個層次：

>> 層次一：像素級比對（Pixel-level）

直接將測試影像與參考影像逐像素相減。差異大的區域即為瑕疵。
適用於外觀高度一致的產品（PCB、印刷標籤、IC 晶片）。
前提：測試件必須先精確對齊到參考影像的座標系。

公式：像素級差異分數

```
diff(x,y) = |I_test(x,y) - I_ref(x,y)|

defect_map = (diff > T)

SSIM(x,y) = (2*mu_x*mu_y + C1)(2*sigma_xy + C2)
          / (mu_x^2+mu_y^2+C1)(sig_x^2+sig_y^2+C2)
```

SSIM = 1 : 完全相同
SSIM ~ 0 : 完全不同

SSIM（結構相似性指標）綜合考慮亮度、對比度和結構三個面向，比單純的像素差分更能反映人眼感知到的品質差異。

```python
[python]
import cv2
import numpy as np
from skimage.metrics import structural_similarity

def pixel_compare(test, ref, thresh=30):
 """像素級比對，返回差異圖和分數"""
 # 絕對差分
    diff = cv2.absdiff(test, ref)

 # 二值化差異區域
    _, mask = cv2.threshold(
        diff, thresh, 255, cv2.THRESH_BINARY)

 # SSIM 分數（越高越好）
    score, ssim_map = structural_similarity(
        test, ref, full=True)

 # 差異面積百分比
    diff_ratio = np.count_nonzero(mask) / mask.size

    return {
        'diff_img': diff,
        'mask': mask,
        'ssim': score,
        'ssim_map': ssim_map,
        'diff_ratio': diff_ratio
    }
```

>> 層次二：特徵級比對（Feature-level）

不直接比像素，而是提取特徵（面積、圓度、顏色等），與良品特徵的允許範圍進行比較。
適用於產品有自然變異、無法逐像素比對的場景。

公式：特徵偏離度

```
deviation_i = |feature_i - mean_i| / std_i

偏離度 > k (如 k=3) => 該特徵異常

綜合偏離度 = sqrt(SUM(deviation_i^2) / N)
```

每個特徵的偏離度類似 z-score。多特徵的綜合偏離度使用歐式距離。

```
[python]
def feature_compare(features, golden_stats):
 """特徵級比對
 features: dict, 測試件的特徵值
 golden_stats: dict, 良品特徵的
                  {name: (mean, std)} """
    deviations = {}
    for name, val in features.items():
        mean, std = golden_stats[name]
  std = max(std, 1e-6) # 避免除零
        dev = abs(val - mean) / std
        deviations[name] = dev

# 綜合偏離度
    vals = list(deviations.values())
    overall = np.sqrt(
        sum(v**2 for v in vals) / len(vals))

    return deviations, overall
```

>> 層次三：模型級比對（Model-level）

使用統計模型（變異模型）或神經網路（自編碼器）建立良品模型。測試件輸入模型後，模型輸出重建結果或統計分數，偏離越大表示越可能有瑕疵。

- 變異模型：z-score 圖（第 11 章）
- 自編碼器：重建誤差圖（第 10 章）
- 這些方法的優勢是自動學習「正常」的定義。

公式：自編碼器異常圖

```
anomaly_map(x,y) = (I(x,y) - AE(I)(x,y))^2
anomaly_score = mean(anomaly_map)
```

歸一化: score_norm = anomaly_score / threshold

score_norm > 1 表示該影像整體異常程度超過基準閾值。

比對層次	適用場景	精度	對齊要求
像素級	PCB/印刷品/IC	最高	必須精確對齊
特徵級	零件/食品/藥品	中等	粗略對齊即可
模型級	紋理/橡膠/複雜	高	需要對齊

14.2 瑕疵評分機制

檢測到瑕疵後，需要量化其嚴重程度。單一的「有/無」判定不足以支撐品質管控決策。評分機制將瑕疵轉換為可比較的數值分數。

>> 單一缺陷分數

公式：缺陷嚴重度分數

```
score_i = w_area * (area_i / area_total)
        + w_contrast * (mean_diff_i / 255)
        + w_position * position_weight_i
```

w_area = 面積權重 (如 0.4)
w_contrast = 對比度權重 (如 0.3)
w_position = 位置權重 (如 0.3)
position_weight: 關鍵區域=1, 邊緣=0.3

不同維度加權。位置權重反映「瑕疵出現在哪裡」的重要性——A 級表面瑕疵比邊角瑕疵嚴重得多。

>> 多缺陷聚合分數

公式：整件評分

Score_part = max(score_i) -- 最嚴重的
或
Score_part = SUM(score_i) -- 累加
或
Score_part = SUM(score_i * w_type_i) -- 類型加權

歸一化到 [0, 100]:

```
Score_norm = min(Score_part / S_max * 100, 100)
```

max 策略：一個致命瑕疵即判死。sum 策略：允許多個輕微瑕疵。根據品質標準選擇。

```
[python]
def defect_score(contours, gray, diff_img,
                 position_map=None):
"""計算每個缺陷和整件的瑕疵分數
contours:   瑕疵輪廓列表
gray:       原始灰度影像
diff_img:   差異影像
position_map: 位置權重圖 (可選)"""
    total_area = gray.shape[0] * gray.shape[1]
    scores = []

    for cnt in contours:
        area = cv2.contourArea(cnt)
        mask_i = np.zeros_like(gray)
        cv2.drawContours(
            mask_i, [cnt], -1, 255, -1)

  # 面積比
        area_ratio = area / total_area

  # 對比度 ( 差異均值 )
        mean_diff = cv2.mean(
            diff_img, mask=mask_i)[0] / 255

  # 位置權重
        if position_map is not None:
            pos_w = cv2.mean(
```

```
                position_map, mask=mask_i)[0]
        else:
            pos_w = 1.0

        s = (0.4*area_ratio + 0.3*mean_diff
             + 0.3*pos_w)
        scores.append({
            'area': area,
            'area_ratio': area_ratio,
            'contrast': mean_diff,
            'pos_weight': pos_w,
            'score': s
        })

    # 整件分數（取最嚴重）
    part_score = max(
        (d['score'] for d in scores),
        default=0)
    score_100 = min(part_score * 1000, 100)

    return scores, score_100
```

14.3 篩選判定與分級

有了分數後，需要設定判定規則將產品分類。常見策略：

>> 二元判定（OK / NG）

公式：簡單閾值判定

```
result = 'OK'  if score < T_ng  else 'NG'
```

或基於缺陷數量:

```
result = 'OK'  if defect_count == 0  else 'NG'
```

最簡單的判定。適用於零容忍的高精度產品。

>> 多級分類

等級	分數範圍	定義	處置
OK	0 ~ 10	無瑕疵或極輕微	正常出貨
輕微 (Minor)	10 ~ 40	可接受的小瑕疵	降級出貨/標記
重大 (Major)	40 ~ 70	明顯瑕疵	返工/降級
報廢 (Scrap)	70 ~ 100	嚴重瑕疵	報廢處理

```
[python]
def classify_part(score, defects,
                  rules=None):
"""多級品質判定
score:  整件分數 [0,100]
defects: 缺陷列表
rules:  自訂規則 (可選)"""
    if rules is None:
        rules = {
            'scrap_score': 70,
            'major_score': 40,
            'minor_score': 10,
            'max_defects': 5,
            'max_single_area': 500,
        }

# 檢查致命條件
    for d in defects:
        if d['area'] > rules['max_single_area']:
    return 'SCRAP', '單一瑕疵面積過大'

    if len(defects) > rules['max_defects']:
  return 'MAJOR', '瑕疵數量過多'

# 分數判定
    if score >= rules['scrap_score']:
  return 'SCRAP', f'分數={score:.1f}'
    elif score >= rules['major_score']:
  return 'MAJOR', f'分數={score:.1f}'
    elif score >= rules['minor_score']:
  return 'MINOR', f'分數={score:.1f}'
    else:
  return 'OK', f'分數={score:.1f}'
```

>> 動態閾值（SPC 統計製程控制）

固定閾值可能不適合所有批次。使用 SPC 控制圖動態調整閾值：

- 計算近 N 批次的平均分數和標準差
- UCL（上控制限）= mean + 3*std
- 超過 UCL 的批次觸發警報
- 連續多個批次趨近 UCL 表示製程飄移

公式：SPC 控制限

UCL = mu + 3*sigma　(上控制限)
CL = mu　　(中心線)
LCL = mu - 3*sigma　(下控制限)

mu, sigma 來自近 N 個批次的統計

基於常態分佈，99.7% 的正常值落在 UCL/LCL 之間。超出即為異常。

14.4 瑕疵影像標記與輸出

檢測結果需要可視化，供品管人員複核。標記方式包括：

- 外接矩形框（Bounding Box）：最常用，快速定位
- 輪廓描繪：精確顯示瑕疵形狀
- 熱力圖疊加：顯示差異強度分佈
- 瑕疵區域裁剪：放大瑕疵細節
- 綜合報告圖：原圖 + 標記 + 文字資訊

```
[python]
def annotate_defects(img, contours, scores,
                     grade):
"""在影像上標記所有瑕疵
img:      原始彩色影像
contours: 瑕疵輪廓列表
scores:   每個瑕疵的分數 dict 列表
grade:    整件判定等級"""
    result = img.copy()

# 等級顏色
    colors = {
        'OK':    (0, 200, 0),
        'MINOR': (0, 200, 255),
        'MAJOR': (0, 100, 255),
        'SCRAP': (0, 0, 255),
    }
    color = colors.get(grade, (0,0,255))

    for cnt, sc in zip(contours, scores):
  # 畫輪廓
        cv2.drawContours(
            result, [cnt], -1, color, 2)

  # 畫外接矩形
        x,y,w,h = cv2.boundingRect(cnt)
        cv2.rectangle(result,
            (x-3,y-3), (x+w+3,y+h+3),
            color, 1)

  # 標記分數
        label = f"{sc['score']:.3f}"
        cv2.putText(result, label,
            (x, y-8),
            cv2.FONT_HERSHEY_SIMPLEX,
            0.4, color, 1)

# 整件判定標記（左上角）
    cv2.putText(result, grade,
        (10, 30),
        cv2.FONT_HERSHEY_SIMPLEX,
        1.0, color, 2)
```

```
    return result
```

>> 熱力圖疊加

```
[python]
def overlay_heatmap(img, diff_img, alpha=0.5):
 """將差異圖以熱力圖形式疊加到原圖"""
 # 歸一化差異圖到 0-255
    norm = cv2.normalize(
        diff_img, None, 0, 255,
        cv2.NORM_MINMAX).astype(np.uint8)

 # 套用色彩映射
    heatmap = cv2.applyColorMap(
        norm, cv2.COLORMAP_JET)

 # 確保原圖是彩色
    if len(img.shape) == 2:
        img_c = cv2.cvtColor(
            img, cv2.COLOR_GRAY2BGR)
    else:
        img_c = img.copy()

 # 疊加
    overlay = cv2.addWeighted(
        img_c, 1-alpha, heatmap, alpha, 0)
    return overlay
```

>> 瑕疵區域裁剪

```
[python]
def crop_defects(img, contours, margin=20):
 """裁剪每個瑕疵區域並保存"""
    crops = []
    h, w = img.shape[:2]
    for i, cnt in enumerate(contours):
        x,y,bw,bh = cv2.boundingRect(cnt)
  # 加邊距
        x1 = max(0, x - margin)
        y1 = max(0, y - margin)
        x2 = min(w, x + bw + margin)
        y2 = min(h, y + bh + margin)
        crop = img[y1:y2, x1:x2]
        crops.append(crop)
    return crops
```

>> 綜合檢測報告影像

```
[python]
def make_report_image(original, annotated,
                      diff_img, heatmap,
                      grade, score, info):
"""拼接 4 宮格報告影像"""
    h, w = original.shape[:2]
# 統一轉為彩色
    imgs = []
    for im in [original, annotated,
               diff_img, heatmap]:
        if len(im.shape) == 2:
            im = cv2.cvtColor(
                im, cv2.COLOR_GRAY2BGR)
        im = cv2.resize(im, (w//2, h//2))
        imgs.append(im)

# 2x2 拼接
    top = np.hstack([imgs[0], imgs[1]])
    bot = np.hstack([imgs[2], imgs[3]])
    report = np.vstack([top, bot])

# 加標題
    cv2.putText(report,
        f'{grade} | Score:{score:.1f}',
        (10, 25),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.7, (255,255,255), 2)

    return report
```

14.5 批次篩選流程（完整範例）

以下是完整的端到端批次篩選流程，將所有步驟整合為一個可直接運行的函數。流程：遍歷資料夾中的影像 → 前處理 → 檢測 → 評分 → 分級 → 標記 → 分類保存 → 輸出報告。

```
[python]
import os, csv, glob, shutil

def batch_inspect(input_dir, ref_img_path,
                  output_dir, thresh=30):
"""完整的批次瑕疵篩選流程
input_dir:   待檢影像資料夾
ref_img_path: 良品參考影像路徑
output_dir:  輸出根目錄
thresh:      差異閾值"""

# 讀取參考影像
    ref = cv2.imread(ref_img_path, 0)

# 建立輸出目錄結構
    for grade in ['OK','MINOR','MAJOR','SCRAP']:
        os.makedirs(
            os.path.join(output_dir, grade),
            exist_ok=True)
```

```
    os.makedirs(
        os.path.join(output_dir, 'reports'),
        exist_ok=True)

# CSV 報告
    csv_path = os.path.join(
        output_dir, 'inspection_report.csv')
    csv_f = open(csv_path, 'w', newline='')
    writer = csv.writer(csv_f)
    writer.writerow([
        'filename', 'grade', 'score',
        'defect_count', 'max_area', 'reason'
    ])

# 統計
    stats = {'OK':0, 'MINOR':0,
             'MAJOR':0, 'SCRAP':0}

    files = sorted(glob.glob(
        os.path.join(input_dir, '*.png'))
        + glob.glob(
            os.path.join(input_dir, '*.jpg')))

    for fpath in files:
        fname = os.path.basename(fpath)
        img = cv2.imread(fpath)
        gray = cv2.cvtColor(
            img, cv2.COLOR_BGR2GRAY)

  # --- 1. 對齊（此處簡化）---
  aligned = gray # 實際需做配準

  # --- 2. 差異比對 ---
        diff = cv2.absdiff(aligned, ref)
        _, mask = cv2.threshold(
            diff, thresh, 255,
            cv2.THRESH_BINARY)

  # --- 3. 形態學清理 ---
        k = cv2.getStructuringElement(
            cv2.MORPH_ELLIPSE, (5,5))
        mask = cv2.morphologyEx(
            mask, cv2.MORPH_OPEN, k)
        mask = cv2.morphologyEx(
            mask, cv2.MORPH_CLOSE, k)

  # --- 4. 找輪廓 + 評分 ---
        cnts, _ = cv2.findContours(
            mask, cv2.RETR_EXTERNAL,
            cv2.CHAIN_APPROX_SIMPLE)
        cnts = [c for c in cnts
                if cv2.contourArea(c) > 30]

        scores, score_100 = defect_score(
            cnts, gray, diff)
```

```
    # --- 5. 分級 ---
        grade, reason = classify_part(
            score_100, scores)

    # --- 6. 標記 ---
        annotated = annotate_defects(
            img, cnts, scores, grade)

    # --- 7. 保存到對應目錄 ---
        dst = os.path.join(
            output_dir, grade, fname)
        cv2.imwrite(dst, annotated)

    # --- 8. 報告影像 ---
        heat = overlay_heatmap(img, diff)
        report = make_report_image(
            img, annotated, diff, heat,
            grade, score_100, '')
        rpt_path = os.path.join(
            output_dir, 'reports',
            fname.replace('.','_rpt.'))
        cv2.imwrite(rpt_path, report)

    # --- 9. CSV 記錄 ---
        max_a = max(
            (s['area'] for s in scores),
            default=0)
        writer.writerow([
            fname, grade,
            f'{score_100:.1f}',
            len(cnts), max_a, reason
        ])
        stats[grade] += 1

    csv_f.close()
    return stats
```

案例研究：PCB 批次篩選實例

場景：100 張 PCB 影像需要自動篩選分類。

1. 準備良品參考影像（golden_ref.png）
2. 將待檢影像放入 input/ 資料夾
3. 執行 batch_inspect('input/', 'golden_ref.png', 'output/')
4. 輸出結構：
   output/OK/ -> 良品影像（帶綠色標記）
   output/MINOR/ -> 輕微瑕疵（帶黃色標記）
   output/MAJOR/ -> 重大瑕疵（帶橙色標記）
   output/SCRAP/ -> 報廢品（帶紅色標記）
   output/reports/-> 4 宮格報告影像
   output/inspection_report.csv -> 彙總表

CSV 範例：
filename, grade, score, defect_count, max_area, reason
pcb_001.png, OK, 2.3, 0, 0, 分數=2.3
pcb_042.png, MAJOR, 55.1, 3, 820, 分數=55.1
pcb_099.png, SCRAP, 88.7, 1, 2400, 單一瑕疵面積過大

14.6 常見判定策略與陷阱

場景	推薦策略	閾值設定建議
零容忍高精密	二元 OK/NG	defect_count=0
消費電子外觀	多級分級	分數 10/40/70
食品/藥品	二元 + 法規	面積+位置
汽車零件	嚴格多級	依 IATF 標準
半導體晶圓	分區域判定	各區不同閾值
紡織品	允許輕微	分數 20/60/85

提示

閾值設定方法：
1. 收集 100+ 張已知良品和已知瑕疵品的影像
2. 對所有影像計算分數
3. 繪製分數分佈直方圖
4. 找到良品和瑕疵品的分數分界點
5. 設定閾值在分界點附近，留適當安全餘量
6. 計算 Precision / Recall，調整至平衡
7. 定期用新數據驗證並更新閾值

注意

常見錯誤：

1. 閾值設太嚴：大量良品被誤判為瑕疵（假陽性高），增加返工成本
2. 閾值設太鬆：真正的瑕疵品流出（假陰性高），影響客戶品質
3. 忽略位置資訊：同樣大小的瑕疵在不同位置嚴重程度可能完全不同
4. 使用固定閾值不更新：材料批次變化或設備老化會導致閾值失效
5. 僅依賴單一特徵：用多特徵組合判定才穩健

>> 評估指標

公式：Precision 與 Recall

```
Precision = TP / (TP + FP)
```

檢出的瑕疵中真正有瑕疵的比例

```
Recall = TP / (TP + FN)
```

所有瑕疵中被成功檢出的比例

```
F1 = 2 * Precision * Recall
   / (Precision + Recall)
```

TP=真陽性, FP=假陽性, FN=假陰性。工業檢測通常優先確保高 Recall（不漏檢），允許適度的低 Precision（可接受少量誤檢）。

```
[python]
def evaluate(predictions, ground_truth):
 """評估檢測系統表現"""
    tp = sum(1 for p, g in
             zip(predictions, ground_truth)
             if p == 'NG' and g == 'NG')
    fp = sum(1 for p, g in
             zip(predictions, ground_truth)
             if p == 'NG' and g == 'OK')
    fn = sum(1 for p, g in
             zip(predictions, ground_truth)
             if p == 'OK' and g == 'NG')

    precision = tp/(tp+fp) if tp+fp else 0
    recall = tp/(tp+fn) if tp+fn else 0
    f1 = (2*precision*recall /
          (precision+recall)
          if precision+recall else 0)

    return {
        'precision': precision,
        'recall': recall,
        'f1': f1,
        'escape_rate': fn/(tp+fn)
                       if tp+fn else 0
    }
```

第 15 章

相機校正與像素-毫米轉換

在工業瑕疵檢測中，僅僅知道瑕疵的像素大小是不夠的。客戶的品質規格通常以毫米為單位（如：刮痕長度不得超過 0.5mm）。要將像素量測值轉換為真實世界的物理尺寸，必須進行相機校正。此外，所有光學鏡頭都會引入畸變，使直線變彎、圓形變橢圓，導致量測誤差。本章涵蓋完整的相機校正流程、畸變矯正以及像素到毫米的精確轉換。

15.1 為什麼需要相機校正

相機並非完美的成像設備。從三維世界到二維影像的投影過程中，存在兩類主要問題：
1. 透視投影畸變：針孔相機模型將三維點投影到二維平面，遠處物體看起來更小
2. 鏡頭光學畸變：真實鏡頭的製造缺陷和光學特性導致影像變形

針孔相機模型是最基本的相機數學模型。它假設所有光線通過一個無限小的孔（光心），直線投影到感測器平面上。真實相機使用透鏡聚焦光線，但在數學上可以近似為針孔模型加上畸變參數。

>> 畸變類型

鏡頭畸變分為兩大類：
- 徑向畸變（Radial Distortion）：沿著離光心的徑向方向，直線變彎曲。桶形畸變使邊緣向外膨脹，枕形畸變使邊緣向內收縮。廣角鏡頭的桶形畸變尤為嚴重。
- 切向畸變（Tangential Distortion）：由於透鏡與感測器平面不完全平行導致。影像某些區域看起來比實際更近或更遠。通常比徑向畸變小得多，但在高精度量測中不可忽略。

15.2 內部參數與畸變係數

相機內部參數（Intrinsic Parameters）描述相機自身的光學特性，獨立於外部場景。核心參數包括：
- fx, fy：焦距，以像素為單位。fx = f / pixel_width, fy = f / pixel_height。理想情況下 fx = fy，但感測器像素可能非正方形。
- cx, cy：主點座標，光軸與感測器平面的交點。理想情況下在影像中心，但製造誤差會導致偏移。
- k1, k2, k3：徑向畸變係數。k1 影響最大，k3 通常可忽略。
- p1, p2：切向畸變係數。

公式：相機內部參數矩陣

```
K = [fx   0  cx]
    [ 0  fy  cy]
    [ 0   0   1]

3D 點投影到像素座標:
    u = fx * (X/Z) + cx
    v = fy * (Y/Z) + cy
```

K 矩陣將歸一化相機座標轉換為像素座標。fx/fy 的單位是像素。cx/cy 通常接近影像中心 (W/2, H/2)。

公式：徑向畸變模型

```
r^2 = x'^2 + y'^2

x_d = x'*(1 + k1*r^2 + k2*r^4 + k3*r^6)
y_d = y'*(1 + k1*r^2 + k2*r^4 + k3*r^6)

(x', y') = 無畸變歸一化座標
(x_d, y_d) = 有畸變歸一化座標
r = 到光心的徑向距離
```

k1 > 0 產生枕形畸變，k1 < 0 產生桶形畸變。大多數工業鏡頭 k1 為負值（桶形畸變）。

公式：切向畸變模型

```
x_d = x' + [2*p1*x'*y' + p2*(r^2 + 2*x'^2)]
y_d = y' + [p1*(r^2 + 2*y'^2) + 2*p2*x'*y']
```

切向畸變來自透鏡與感測器不平行。在高精度量測（< 0.01mm）中必須考慮。

```
[python]
import cv2
import numpy as np
import glob

# 畸變係數向量
# dist_coeffs = [k1, k2, p1, p2, k3]
# 例如：
# dist = np.array([-0.2413, 0.0998,
#                  0.0001, -0.0003, 0.0])

# 完整投影流程（3D -> 2D 像素）
def project_point(P_3d, K, dist, rvec, tvec):
 """將 3D 點投影到像素座標"""
    pts_2d, _ = cv2.projectPoints(
        P_3d.reshape(-1,1,3).astype(np.float64),
        rvec, tvec, K, dist)
    return pts_2d.reshape(-1, 2)
```

15.3 棋盤格校正流程

棋盤格（Checkerboard）校正是最常用的相機校正方法。原理：棋盤格的角點位置在物體座標系中是已知的（等間距網格），通過檢測多張不同角度的棋盤格影像中的角點，建立 3D-2D 對應關係，使用最小二乘法求解內部參數和畸變係數。

>> 校正步驟

1. 準備棋盤格校正板（建議 9x6 或 11x8 內角點，方格尺寸已知）
2. 從不同角度、距離拍攝 15~30 張棋盤格影像
3. 對每張影像檢測角點（findChessboardCorners）
4. 精細化角點位置到亞像素精度（cornerSubPix）
5. 建立 3D 物體點與 2D 影像點的對應
6. 使用 calibrateCamera 求解所有參數
7. 評估重投影誤差，驗證校正品質

```
 [python]
# === 完整棋盤格相機校正流程 ===
 import cv2
 import numpy as np
 import glob

# 棋盤格參數
BOARD_SIZE = (9, 6)  # 內角點數 (列, 行)
SQUARE_SIZE = 25.0   # 方格邊長 (mm)

# 建立物體座標點 (z=0 的平面)
 objp = np.zeros(
     (BOARD_SIZE[0]*BOARD_SIZE[1], 3),
     dtype=np.float32)
 objp[:, :2] = np.mgrid[
     0:BOARD_SIZE[0],
     0:BOARD_SIZE[1]].T.reshape(-1, 2)
objp *= SQUARE_SIZE # 換算為 mm

# 收集所有影像的角點
obj_points = []  # 3D 物體點
img_points = []  # 2D 影像點

 images = glob.glob('calib_images/*.jpg')
 for fname in images:
     img = cv2.imread(fname)
     gray = cv2.cvtColor(
         img, cv2.COLOR_BGR2GRAY)

  # 尋找棋盤格角點
     ret, corners = cv2.findChessboardCorners(
         gray, BOARD_SIZE,
         cv2.CALIB_CB_ADAPTIVE_THRESH +
         cv2.CALIB_CB_NORMALIZE_IMAGE)

     if ret:
         obj_points.append(objp)

    # 亞像素精細化
         criteria = (
             cv2.TERM_CRITERIA_EPS +
             cv2.TERM_CRITERIA_MAX_ITER,
             30, 0.001)
         corners2 = cv2.cornerSubPix(
```

```
            gray, corners, (11,11),
            (-1,-1), criteria)
        img_points.append(corners2)

# 執行校正
 ret, K, dist, rvecs, tvecs = \
     cv2.calibrateCamera(
         obj_points, img_points,
         gray.shape[::-1], None, None)

print(f'RMS 重投影誤差: {ret:.4f} pixels')
print(f'內部參數矩陣 K:\n{K}')
print(f'畸變係數: {dist.ravel()}')

# 計算每張影像的重投影誤差
 errors = []
 for i in range(len(obj_points)):
     proj, _ = cv2.projectPoints(
         obj_points[i], rvecs[i],
         tvecs[i], K, dist)
     err = cv2.norm(
         img_points[i], proj,
         cv2.NORM_L2) / len(proj)
     errors.append(err)
print(f'平均誤差: {np.mean(errors):.4f} px')
print(f'最大誤差: {np.max(errors):.4f} px')

# 保存校正結果
 np.savez('calibration.npz',
          K=K, dist=dist,
          rvecs=rvecs, tvecs=tvecs)
```

15.4 像素到毫米轉換

校正完成後，需要建立像素與毫米之間的對應關係。轉換方式取決於應用場景：

- 已知工作距離和感測器尺寸：直接計算像素物理尺寸
- 使用校正結果：從焦距和工作距離推算
- 實測法：拍攝已知尺寸的物體，計算比例因子

在工業檢測中，通常物體到相機的距離固定（固定安裝），因此轉換比例是常數。

公式：基於感測器的像素尺寸

```
pixel_size = sensor_size / resolution
```

sensor_size: 感測器物理尺寸 (mm)
resolution: 解析度 (像素)

例: 1/2" 感測器 (6.4mm x 4.8mm),
1280x960 解析度:

```
pixel_w = 6.4/1280 = 0.005 mm/pixel
pixel_h = 4.8/960  = 0.005 mm/pixel
```

這是感測器上每個像素的物理大小。實際場景中物體上的對應尺寸還需考慮放大倍率。

公式：像素到毫米轉換（含放大倍率）

```
mm = pixels * pixel_size / magnification

magnification = focal_length / working_distance
```

或更實用的方式：

```
mm_per_pixel = FOV_mm / image_width_pixels
```

FOV_mm: 視場範圍 (mm)
image_width_pixels: 影像寬度 (像素)

mm_per_pixel 是最常用的轉換係數。在固定安裝的工業相機中，此值為常數。推薦使用已知尺寸的校正件實測此值。

```
[python]
# === 像素到毫米轉換 ===

# 方法一：基於感測器規格
sensor_w_mm = 6.4    # 感測器寬度 (mm)
img_w_px = 1280      # 影像寬度 (像素)
focal_mm = 12.0      # 焦距 (mm)
work_dist_mm = 300.0 # 工作距離 (mm)

pixel_size = sensor_w_mm / img_w_px
magnification = focal_mm / work_dist_mm
mm_per_pixel = pixel_size / magnification
print(f'mm/pixel = {mm_per_pixel:.4f}')

# 方法二：實測法（推薦）
# 拍攝已知尺寸的標準件
known_length_mm = 50.0   # 標準件長度
measured_length_px = 628 # 影像中像素數
mm_per_pixel_real = (
    known_length_mm / measured_length_px)
print(f'實測 mm/pixel = '
      f'{mm_per_pixel_real:.4f}')

# 方法三：從校正結果計算
# K[0,0] = fx (像素為單位的焦距)
# mm_per_pixel = working_dist / fx
```

```
 fx = K[0, 0]
 mm_per_px_calib = work_dist_mm / fx
print(f'校正 mm/pixel = '
       f'{mm_per_px_calib:.4f}')

# 應用：量測瑕疵的真實尺寸
 def measure_defect_mm(contour, mm_per_px):
 """量測瑕疵的真實尺寸 (mm)"""
     area_px = cv2.contourArea(contour)
     peri_px = cv2.arcLength(contour, True)
     rect = cv2.minAreaRect(contour)
     w_px, h_px = rect[1]

     return {
         'area_mm2': area_px * mm_per_px**2,
         'perimeter_mm': peri_px * mm_per_px,
         'width_mm': w_px * mm_per_px,
         'height_mm': h_px * mm_per_px,
     }
```

15.5 畸變校正與影像矯正

校正得到畸變係數後，可以將畸變影像矯正為無畸變影像。這對精確量測至關重要。OpenCV 提供兩種方式：直接 undistort 和使用映射表（remap）。映射表方式在批次處理時效率更高，因為映射表只需計算一次。

```
 [python]
# === 畸變矯正 ===

# 載入校正結果
 calib = np.load('calibration.npz')
 K = calib['K']
 dist = calib['dist']

 img = cv2.imread('test_image.jpg')
 h, w = img.shape[:2]

# 方法一：直接矯正
 undist = cv2.undistort(img, K, dist)

# 方法二：使用最佳新相機矩陣
# alpha=0: 裁剪掉所有黑邊
# alpha=1: 保留所有像素（有黑邊）
 new_K, roi = cv2.getOptimalNewCameraMatrix(
     K, dist, (w, h), alpha=0)
 undist2 = cv2.undistort(
     img, K, dist, None, new_K)

# 裁剪有效區域
 x, y, rw, rh = roi
 undist2 = undist2[y:y+rh, x:x+rw]

# 方法三：映射表方式（批次處理推薦）
 map1, map2 = cv2.initUndistortRectifyMap(
     K, dist, None, new_K,
```

```
    (w, h), cv2.CV_32FC1)

# 對每張影像快速矯正
for fname in glob.glob('images/*.jpg'):
    img_i = cv2.imread(fname)
    fixed = cv2.remap(
        img_i, map1, map2,
        cv2.INTER_LINEAR)
    cv2.imwrite(
        fname.replace('.jpg', '_fixed.jpg'),
        fixed)
```

案例研究：精密零件量測中的校正實例

場景：精密沖壓件的孔徑公差檢測，規格為 5.00 +/- 0.05 mm。

問題：未校正時量測結果為 5.12 mm，一直判定為不良品，但用千分尺實測為 5.01 mm。

根因分析：
1. 鏡頭桶形畸變導致影像邊緣區域的孔洞被放大
2. 像素到毫米的轉換係數使用了理論值而非實測值

解決流程：
1. 使用 9x6 棋盤格拍攝 25 張校正影像
2. 校正得到 RMS 重投影誤差 = 0.12 pixels
3. 對所有檢測影像進行畸變矯正
4. 使用 5mm 標準量規實測 mm/pixel = 0.01563
5. 校正後量測結果：5.02 mm（與千分尺一致）

效果：量測精度從 +/- 0.15mm 提升至 +/- 0.02mm，誤判率從 18% 降至 0.3%。

提示

校正板選擇和拍攝注意事項：
1. 校正板應為高精度印刷或蝕刻，平面度 < 0.1mm
2. 拍攝時棋盤格應覆蓋影像的各個區域（中心、四角、邊緣）
3. 至少拍攝 15 張不同角度的影像，建議 20~30 張
4. 棋盤格傾斜角度不應超過 45 度，否則角點檢測容易失敗
5. 確保所有角點清晰可見，避免反光和陰影
6. 校正環境的照明條件應與實際檢測環境一致

注意

常見校正失敗原因：
1. 校正板不平整：紙質列印的棋盤格容易彎曲，應貼附在平整硬板上
2. 拍攝角度太少或太集中：僅在正面拍攝無法準確估計畸變係數
3. 角點檢測失敗卻未過濾：應檢查 findChessboardCorners 的返回值
4. 重投影誤差過大（> 1.0 pixel）：表示校正結果不可靠，需重新拍攝
5. 工作距離改變後未重新校正：mm/pixel 隨距離變化
6. 溫度變化導致焦距漂移：精密量測應在恆溫環境中進行

第 16 章

影像對齊與配準

影像對齊（Image Registration / Alignment）是將測試影像轉換到與參考影像相同的座標系中。在瑕疵檢測中，對齊是模板比對、差分檢測和變異模型的前提條件。如果對齊不準確，正常的紋理邊緣會產生大量假陽性，使得瑕疵檢測完全失效。本章介紹從粗對齊到精對齊的完整方法體系。

16.1 為什麼需要對齊

在工業產線上，被檢測產品進入視場時的位置和角度每次都不同。即使使用定位治具，仍然存在 0.5~2mm 的位置偏差和 0.5~3 度的旋轉偏差。對於逐像素比較的檢測方法（如差分影像法、變異模型），即使 1 個像素的錯位也會產生邊緣處的假差異。

對齊方法分為兩大類：
- 手動/固定對齊：使用機械治具和觸發器精確定位。成本高但延遲低。
- 軟體自動對齊：通過影像處理計算變換矩陣。靈活且成本低，但需要計算時間。

軟體對齊的精度通常可達 0.1~0.3 像素（使用亞像素方法），足以滿足大多數瑕疵檢測需求。

16.2 基於特徵點的對齊

特徵點對齊是最通用的方法。流程：
1. 在參考影像和測試影像中分別檢測特徵點（角點/斑點）
2. 計算每個特徵點的描述子
3. 匹配兩組描述子，找到對應點對
4. 使用 RANSAC 剔除錯誤匹配（外點）
5. 從正確的匹配點對估計變換矩陣（仿射或單應性）
6. 使用變換矩陣將測試影像對齊到參考影像

公式：單應性矩陣（Homography）

```
            [h11 h12 h13] [x]
   [x']     [h21 h22 h23] [y]
   [y'] ~= [h31 h32 h33] [1]
   [1 ]

     x' = (h11*x + h12*y + h13) /
          (h31*x + h32*y + h33)
     y' = (h21*x + h22*y + h23) /
          (h31*x + h32*y + h33)
```

8 個自由度，需至少 4 對匹配點

單應性矩陣描述兩個平面之間的投影變換。對於平面物體（如 PCB、標籤），Homography 是精確的變換模型。

```
[python]
import cv2
import numpy as np

def align_by_features(test, ref,
                      max_features=500,
                      good_match_pct=0.15):
"""基於 ORB 特徵的影像對齊
test: 測試影像 (灰度)
ref:  參考影像 (灰度)
返回: 對齊後的影像, 變換矩陣"""

# 1. 建立 ORB 偵測器
    orb = cv2.ORB_create(
        nfeatures=max_features)

# 2. 偵測特徵點和描述子
    kp1, des1 = orb.detectAndCompute(
        test, None)
    kp2, des2 = orb.detectAndCompute(
        ref, None)

# 3. 暴力匹配 (Hamming 距離)
    matcher = cv2.BFMatcher(
        cv2.NORM_HAMMING,
        crossCheck=True)
    matches = matcher.match(des1, des2)

# 4. 按距離排序，取最佳匹配
    matches = sorted(
        matches, key=lambda x: x.distance)
    n_good = int(
        len(matches) * good_match_pct)
    n_good = max(n_good, 10)
    good = matches[:n_good]

# 5. 提取匹配點座標
    pts_test = np.float32(
```

```
        [kp1[m.queryIdx].pt for m in good]
    ).reshape(-1, 1, 2)
    pts_ref = np.float32(
        [kp2[m.trainIdx].pt for m in good]
    ).reshape(-1, 1, 2)

# 6. RANSAC 估計 Homography
    H, mask = cv2.findHomography(
        pts_test, pts_ref,
        cv2.RANSAC, 5.0)

# 7. 應用變換
    h, w = ref.shape[:2]
    aligned = cv2.warpPerspective(
        test, H, (w, h))

    inliers = mask.ravel().sum()
print(f'內點數: {inliers}/{len(good)}')

    return aligned, H
```

16.3 基於輪廓/形狀的對齊

當產品是單一明確形狀（零件輪廓）且背景簡單時，基於輪廓的對齊更加穩健快速。流程：

1. 分割出目標物體的輪廓
2. 找到最大輪廓的最小外接矩形（minAreaRect）
3. 計算中心點和旋轉角度
4. 使用平移和旋轉將輪廓對齊到標準位置

這種方法不依賴紋理特徵，對於無紋理的均勻表面零件非常有效。

```
[python]
def align_by_contour(test, ref):
"""基於輪廓的影像對齊
適用於背景簡單的單一零件"""

    def get_pose(gray):
  """提取物體中心和角度"""
        _, binary = cv2.threshold(
            gray, 0, 255,
            cv2.THRESH_BINARY + cv2.THRESH_OTSU)
        cnts, _ = cv2.findContours(
            binary, cv2.RETR_EXTERNAL,
            cv2.CHAIN_APPROX_SIMPLE)
  # 取最大輪廓
        cnt = max(cnts, key=cv2.contourArea)
        rect = cv2.minAreaRect(cnt)
        center = rect[0]  # (cx, cy)
  angle = rect[2]  # 旋轉角度
        return center, angle, cnt

# 取得參考和測試的姿態
    c_ref, a_ref, _ = get_pose(ref)
```

```
    c_test, a_test, _ = get_pose(test)

    h, w = ref.shape[:2]
    img_center = (w / 2.0, h / 2.0)

# 計算旋轉差
    delta_angle = a_ref - a_test

# 旋轉矩陣（圍繞測試件中心）
    M_rot = cv2.getRotationMatrix2D(
        c_test, delta_angle, 1.0)

# 加上平移（旋轉後的中心對齊到參考中心）
    cos_a = np.cos(np.radians(delta_angle))
    sin_a = np.sin(np.radians(delta_angle))
    new_cx = (cos_a*c_test[0]
              - sin_a*c_test[1] + M_rot[0,2])
    new_cy = (sin_a*c_test[0]
              + cos_a*c_test[1] + M_rot[1,2])
    M_rot[0, 2] += c_ref[0] - new_cx
    M_rot[1, 2] += c_ref[1] - new_cy

    aligned = cv2.warpAffine(
        test, M_rot, (w, h))
    return aligned, M_rot
```

16.4 基於相位相關的對齊

相位相關（Phase Correlation）利用傅立葉變換計算兩張影像之間的平移偏移。它在頻率域中運作，對亮度變化具有魯棒性，且速度快（O(N log N)）。適用於僅有平移偏移的情況（無旋轉/縮放），如傳送帶上移動的零件。

公式：相位相關公式

互功率譜：R(u,v) = F1(u,v) * conj(F2(u,v))
/ |F1 * conj(F2)|

反傅立葉：r(x,y) = IFFT(R(u,v))

偏移量 = argmax{r(x,y)} 的位置 (dx, dy)

相位相關的峰值位置直接對應兩張影像的平移偏移。峰值的銳度指示對齊的可信度。

```
[python]
def align_by_phase(test, ref):
 """基於相位相關的平移對齊
 返回: 對齊後的影像, (dx, dy)"""

 # 轉為 float
     t = test.astype(np.float32)
     r = ref.astype(np.float32)

 # 漢寧窗減少邊界效應
     h, w = t.shape
     win_r = np.hanning(h).reshape(-1,1)
     win_c = np.hanning(w).reshape(1,-1)
     window = (win_r * win_c).astype(
         np.float32)
     t_w = t * window
     r_w = r * window

 # 相位相關
     (dx, dy), response = cv2.phaseCorrelate(
         t_w, r_w)
 print(f'偏移: dx={dx:.2f}, dy={dy:.2f}')
 print(f'相關峰值: {response:.4f}')

 # 應用平移
     M = np.float32([
         [1, 0, -dx],
         [0, 1, -dy]])
     aligned = cv2.warpAffine(
         test, M, (w, h))

     return aligned, (dx, dy)
```

16.5 多尺度/多步驟配準策略

在實際應用中，單一對齊方法可能不夠穩健或精確。多步驟策略結合不同方法的優勢：

粗對齊（Coarse Alignment）：
- 目標：消除大偏移和旋轉（> 5 pixels, > 2 度）
- 方法：輪廓對齊或低解析度特徵匹配
- 速度快，精度要求低

精對齊（Fine Alignment）：
- 目標：達到亞像素精度（< 0.5 pixels）
- 方法：相位相關或高解析度模板匹配
- 在粗對齊結果上進行微調

```
[python]
def multi_step_align(test, ref):
 """多步驟配準：粗對齊 + 精對齊"""

 # 步驟 1：粗對齊（輪廓法）
     coarse, M1 = align_by_contour(
         test, ref)

 # 步驟 2：精對齊（相位相關）
     fine, (dx, dy) = align_by_phase(
         coarse, ref)

 # 步驟 3：驗證對齊品質
     diff = cv2.absdiff(fine, ref)
     mean_diff = np.mean(diff)
 print(f'對齊後平均差異: {mean_diff:.2f}')

 if mean_diff > 30: # 品質不佳
    print('警告: 對齊品質不佳')
    # 嘗試特徵點方法
         fine, _ = align_by_features(
             test, ref)

     return fine
```

對齊方法	速度	精度	適用場景
特徵點 (ORB)	中等	1~3 px	有紋理的平面物體
特徵點 (SIFT)	慢	0.5~2 px	多尺度/旋轉變化
輪廓/形狀	快	2~5 px	背景簡單的零件
相位相關	快	0.1~0.5 px	僅平移偏移
模板匹配	慢	1~2 px	小 ROI 精確定位
多步驟融合	中等	< 0.5 px	高精度需求

案例研究：PCB 對齊到黃金參考

場景：PCB 板瑕疵檢測需要將測試影像與黃金參考精確對齊後差分。

挑戰：
- PCB 在傳送帶上位移可達 +/- 5mm
- 旋轉偏差可達 +/- 2 度
- 需要亞像素精度以避免邊緣處假陽性

解決方案（三步法）：
1. 粗定位：ORB 特徵匹配 + Homography，消除大偏移和旋轉
   - 匹配 500 個特徵點，RANSAC 剔除外點
   - 精度約 2 像素
2. 精對齊：相位相關，消除殘餘平移
   - 精度約 0.2 像素
3. 局部微調：在關鍵 ROI 區域使用 NCC 模板匹配
   - 精度約 0.1 像素

結果：
- 對齊成功率 > 99.5%（500 張測試影像僅 2 張失敗）
- 假陽性率從未對齊的 45% 降至 < 2%
- 整體對齊耗時約 50ms（1280x960 影像）

注意

對齊失敗的常見原因和解決方案：

1. 特徵點太少：物體表面無紋理。解決方案：改用輪廓對齊或增加表面標記。
2. 重複紋理導致錯誤匹配：週期性圖案（如網格）使匹配混亂。解決方案：限制搜尋範圍或使用 SIFT。
3. 遮擋或缺損改變外觀：嚴重瑕疵使特徵消失。解決方案：使用 RANSAC 並提高外點容忍度。
4. 光照變化過大：不同時間拍攝的亮度差異大。解決方案：先做直方圖均衡化再對齊。
5. 變換模型錯誤：使用仿射變換對齊有透視變形的物體。解決方案：改用 Homography。

第 17 章

光源原理與選型

在工業視覺瑕疵檢測中，有一句經典名言：「打光佔 70%，演算法佔 30%」。正確的照明能讓瑕疵變得顯而易見，使簡單的閾值分割就能檢測；錯誤的照明則讓再先進的演算法也無能為力。本章深入探討各種照明方式的物理原理、光源類型選擇以及針對不同瑕疵的最佳打光策略。

17.1 光源在瑕疵檢測中的核心角色

光源的作用是在瑕疵區域和正常區域之間產生最大的亮度對比度。沒有對比度，就沒有可檢測的信號。好的照明方案能夠：
- 最大化瑕疵信號（亮度或顏色差異）
- 最小化背景雜訊（表面紋理、反光）
- 提供均勻一致的照明（避免假陽性）
- 在產線速度下穩定工作

為什麼光源比演算法更重要？考慮以下情境：
- 金屬表面的細微刮痕在正面照明下幾乎看不到，但低角度暗場照明可使其清晰可見
- 透明薄膜中的氣泡在反射光下不可見，但背光下立即顯現
- 不均勻的照明會在均勻表面產生虛假的亮度梯度，被誤判為瑕疵

經驗法則：如果無法用肉眼在檢測影像中看到瑕疵，演算法也幾乎無法可靠地檢測它。

17.2 照明幾何（打光方式詳解）

>> 正面照明

光源從物體表面法線方向照射。適用於檢測平面上的印刷、標記和顏色差異。

同軸光（Coaxial Light）：
光線通過半透明鏡從鏡頭光軸方向照射，消除陰影。鏡面反射回到相機。適用於高反射率表面（拋光金屬、玻璃、晶圓）的缺陷檢測。
原理：平坦表面將光直接反射回相機（亮），缺陷處法線偏轉導致光偏離（暗）。

環形光（Ring Light）：
圍繞鏡頭排列的 LED 環。角度可調（0~75 度）。提供均勻無陰影的照明。是最通用的工業照明方式。
高角度環形光：類似正面照明，凸顯顏色和紋理差異。
低角度環形光：產生側射效果，凸顯表面凸起和刮痕。

穹頂光（Dome Light）：
半球形的漫射照明。光線從所有方向均勻照射。完全消除反射和陰影。適用於高反光曲面物體（如金屬殼體、圓柱零件）的表面檢測。
缺點：消除了所有表面拓撲資訊，不適合檢測凹凸類瑕疵。

>> 側面照明（暗場照明）

光源以極低角度（5~30
度）從側面照射表面。正常的平坦區域將光反射到遠離相機的方向（暗），而凸起、刮痕、邊緣等表面不規則處會將光散射向相機（亮）。

原理：暗場照明利用的是散射光而非鏡面反射。在暗背景上，瑕疵表現為明亮的特徵，產生極高的對比度。

最適瑕疵類型：刮痕、毛邊、凸起、裂紋、表面粗糙度變化。
限制：對表面輕微顏色變化不敏感。

技巧：使用 4 個方向的低角度光源（0/90/180/270 度）逐一照射，可實現光度立體法並檢測所有方向的刮痕。

>> 背光照明

光源放在物體背面，相機在物體前面。物體遮擋光線形成輪廓/剪影。

適用場景：
- 外形尺寸量測（高對比度輪廓）
- 透明/半透明材料中的缺陷（氣泡、異物、裂紋）
- 孔洞、缺口的檢測（通光 vs 遮光）
- 瓶子內的液位、異物檢測

背光類型：直射背光（平行光，高對比度邊緣）vs 漫射背光（均勻照明，適合透明件）。

>> 結構光

投射特定的光圖案（線、網格、條紋）到物體表面。表面形狀的變化會導致光圖案變形，通過分析變形可以重建三維表面形貌。

常見類型：
- 單線結構光：用於線掃描場景，逐行重建高度
- 多線結構光：加速掃描速度
- 相移條紋投影：最高精度的 3D 測量

適用於需要三維表面形貌的檢測：焊縫高度、零件平面度、翹曲變形。

17.3 光源類型與特性

光源類型	壽命	波長	適用場景
白光 LED	>50000h	寬頻	通用檢測
單色 LED	>50000h	特定波長	增強特定對比
UV LED	>20000h	365~405nm	螢光/膠水檢測
IR LED	>50000h	850~940nm	穿透/熱成像
鹵素燈	~2000h	寬頻	高亮度照明
螢光燈	~8000h	寬頻	均勻面照明

光源類型	壽命	波長	適用場景

LED 光源是現代工業視覺的主流選擇。原因：
- 壽命長（50000+ 小時）
- 可瞬間開關（頻閃模式）
- 波長可選（紅/綠/藍/白/紅外/紫外）
- 亮度穩定且可調節
- 不產生熱量（相比鹵素燈）

頻閃控制：在高速產線上，持續照明會導致運動模糊。頻閃模式在短時間內（1~100 微秒）提供高亮度脈衝，有效「凍結」運動。

>> 波長選擇

不同波長與材料的交互不同。關鍵原則：
- 使用與瑕疵顏色互補的波長增強對比：紅色瑕疵用藍光、藍色瑕疵用紅光
- 紅色光穿透性較好，適合半透明材料的內部檢測
- 藍色光波長短，聚焦更精確，適合高解析度檢測
- UV 光可激發螢光材料（膠水、油脂、標記），使其在暗背景中發光
- IR 光可穿透某些不透明材料（矽晶圓、塑膠），檢測內部缺陷

17.4 不同瑕疵類型的最佳打光方案

瑕疵類型	推薦光源	原理	備註
刮痕	低角度暗場	散射光凸顯凸起	多方向照明更佳
凹坑/凹痕	穹頂光/同軸	均勻照明顯陰影	深凹坑用側光
毛邊/毛刺	背光	輪廓清晰可見	最簡單有效
污漬/變色	漫射正光	顏色差異最大化	選合適波長
裂紋	低角度側光	散射光增強細縫	螢光滲透增強
氣泡	背光/暗場	透射光凸顯氣泡	漫射背光更佳
缺件/多件	正面環光	整體外觀對比	穩定均勻即可
印刷缺陷	同軸光	消除紋理反射	UV 光檢螢光
焊接缺陷	多角度光	3D 形貌信息	結構光最佳
表面粗糙	暗場對比	粗糙區散射強	量化粗糙度

17.5 打光實驗方法論

選擇光源不應憑經驗猜測，而應系統化測試。建議流程：
1. 準備含有已知瑕疵的樣品（包括良品和各類瑕疵品）
2. 收集可用的光源和打光配件
3. 固定相機參數（曝光、增益、對焦）
4. 逐一測試每種打光方式，拍攝同一組樣品

5. 對每組影像計算瑕疵的信噪比（SNR）
6. 選擇 SNR 最高的打光方案
7. 微調光源角度、亮度、距離

```
[python]
import cv2
import numpy as np
import os

def evaluate_lighting(images_dir, roi=None):
 """評估不同打光方案的信噪比
 images_dir: 包含多組打光影像的目錄
 每個子目錄為一種打光方案
 roi: 已知瑕疵區域 (x,y,w,h) 或 None"""

    results = {}
    for light_dir in sorted(
            os.listdir(images_dir)):
        path = os.path.join(
            images_dir, light_dir)
        if not os.path.isdir(path):
            continue

        snr_list = []
        for f in sorted(os.listdir(path)):
            img = cv2.imread(
                os.path.join(path, f), 0)
            if img is None:
                continue

            if roi:
                x,y,w,h = roi
                defect = img[y:y+h, x:x+w]
            else:
      # 自動：中心 vs 邊緣
                ch, cw = img.shape
                defect = img[
                    ch//4:3*ch//4,
                    cw//4:3*cw//4]

    # 計算 SNR
            signal = float(
                np.mean(defect))
            noise = float(
                np.std(img)) + 1e-6
            snr = abs(signal
                - np.mean(img)) / noise
            snr_list.append(snr)

        avg_snr = np.mean(snr_list)
        results[light_dir] = avg_snr
        print(f'{light_dir}: '
              f'SNR={avg_snr:.2f}')
```

```
    # 推薦最佳方案
        best = max(results, key=results.get)
    print(f'\n推薦方案: {best}'
              f'(SNR={results[best]:.2f})')
        return results
```

案例研究：金屬表面刮痕的光源選擇對比

場景：拋光不銹鋼面板上的細微刮痕檢測（長度 1~10mm，深度 < 0.01mm）。

測試了 5 種打光方案：
1. 正面環形光（45 度）：SNR = 1.2（刮痕幾乎不可見）
2. 同軸光：SNR = 2.8（部分刮痕可見，但對比度低）
3. 穹頂光：SNR = 0.8（完全消除了表面拓撲信息）
4. 單方向低角度暗場（10 度）：SNR = 8.5（垂直方向刮痕清晰，平行方向不可見）
5. 四方向低角度暗場：SNR = 12.3（所有方向的刮痕都清晰可見）

結論：四方向低角度暗場是最佳方案。每個方向的光源逐一點亮，拍攝 4 張影像後取最大值合成。

後處理：
- 合成影像 = max(img_0, img_90, img_180, img_270)
- 簡單的 Otsu 閾值即可分割出刮痕
- 檢出率 > 98%，假陽性率 < 1%

提示

打光的黃金法則：
1. 先看樣品，用手電筒從各角度照射，觀察什麼時候瑕疵最明顯
2. 暗場照明是表面瑕疵（刮痕、凸起、裂紋）的首選
3. 背光是輪廓/透明件檢測的首選
4. 穹頂光是高反光曲面的首選
5. 同軸光是鏡面反射物體的首選
6. 如果一種光源不夠，組合多種光源（多通道照明）
7. 始終使用可控光源（LED），避免依賴環境光

注意

環境光干擾和解決方案：
1. 問題：窗戶自然光隨時間變化，導致檢測結果不穩定
   解決：用遮光罩或暗箱完全隔離環境光
2. 問題：鄰近工作站的照明干擾
   解決：使用窄頻 LED + 對應波長的帶通濾光片
3. 問題：LED 光源亮度隨溫度衰減
   解決：使用恆流驅動器，或定期自動校正亮度
4. 問題：高速產線需要短曝光時間，亮度不足
   解決：使用頻閃控制器提供高峰值亮度脈衝（過驅動模式）

第 18 章

色彩瑕疵檢測

許多瑕疵通過顏色差異表現：印刷偏色、塑膠變色、食品腐敗變色、紡織品染色不均、金屬氧化鏽蝕等。灰度影像無法捕捉這些資訊。本章介紹如何利用色彩空間分析、色差計算和多通道處理來檢測色彩相關的瑕疵。

18.1 色彩空間選擇

不同的色彩空間將顏色資訊以不同方式組織。選擇合適的色彩空間對色彩瑕疵檢測至關重要：
- RGB：相機原生格式，但三通道高度耦合，不適合直接分析
- HSV：將色調（H）、飽和度（S）和亮度（V）分離，最直覺的色彩分割方式
- Lab（CIE L*a*b*）：感知均勻的色彩空間，色差計算最準確
- YCrCb：將亮度（Y）和色度（Cr/Cb）分離，對光照變化更魯棒

公式：RGB 到 Lab 轉換

步驟 1: RGB -> XYZ (線性轉換)

```
[X]   [0.4124 0.3576 0.1805] [R]
[Y] = [0.2126 0.7152 0.0722] [G]
[Z]   [0.0193 0.1192 0.9505] [B]
```

步驟 2: XYZ -> Lab (非線性)

```
L* = 116*f(Y/Yn) - 16
a* = 500*[f(X/Xn) - f(Y/Yn)]
b* = 200*[f(Y/Yn) - f(Z/Zn)]
```

f(t) = t^(1/3)　　若 t > 0.008856

f(t) = 7.787*t + 16/116 否則

Lab 的優勢：L* 代表亮度，a* 代表紅-綠軸，b* 代表黃-藍軸。兩個顏色的歐式距離（Delta E）與人眼感知的色差成正比。

公式：色差 Delta E

CIE76 (最簡單):

```
dE76 = sqrt((L1-L2)^2 + (a1-a2)^2
            + (b1-b2)^2)
```

感知閾值:

- dE < 1.0：人眼幾乎無法區分
- dE ~ 2.0：近距離仔細觀察可區分
- dE ~ 3.5：明顯可見的色差
- dE > 5.0：不同顏色

CIE2000 (更精確，考慮感知非均勻性):

```
dE00 = sqrt((dL'/(kL*SL))^2
            + (dC'/(kC*SC))^2
            + (dH'/(kH*SH))^2
            + RT*(dC'/(kC*SC))
                *(dH'/(kH*SH)))
```

CIE2000 更精確但計算更複雜。對於工業色差檢測，CIE76 通常足夠。嚴格的印刷品質控制建議使用 CIE2000。

色彩空間	優點	缺點	適用場景
RGB	相機原生格式	通道耦合	原始資料存儲
HSV	直覺的色彩分割	H 值不連續	快速色彩篩選
Lab	感知均勻色差	計算較慢	精確色差量測
YCrCb	亮度色度分離	不常用	光照補償

18.2 色差計算與閾值

色差（Delta E）是量化兩個顏色之間視覺差異的標準方法。在瑕疵檢測中，計算每個像素與參考顏色的 Delta E，超過閾值的像素標記為色彩異常。

```
[python]
import cv2
import numpy as np

def compute_delta_e(img, ref_color_lab,
                    threshold=5.0):
 """計算影像每個像素與參考色的 Delta E
 img:       BGR 影像
 ref_color_lab: 參考色 Lab 值 [L, a, b]
 threshold:    色差閾值"""

     # BGR -> Lab
     lab = cv2.cvtColor(
         img, cv2.COLOR_BGR2Lab)
     lab = lab.astype(np.float32)

 # OpenCV Lab 範圍:
```

```
# L: 0~255 (需映射到 0~100)
# a: 0~255 (需映射到 -128~127)
# b: 0~255 (需映射到 -128~127)
    lab[:,:,0] = lab[:,:,0] * 100.0 / 255.0
    lab[:,:,1] = lab[:,:,1] - 128.0
    lab[:,:,2] = lab[:,:,2] - 128.0

# 計算 Delta E (CIE76)
    ref = np.array(ref_color_lab,
                   dtype=np.float32)
    dL = lab[:,:,0] - ref[0]
    da = lab[:,:,1] - ref[1]
    db = lab[:,:,2] - ref[2]
    delta_e = np.sqrt(
        dL**2 + da**2 + db**2)

# 閾值分割
    mask = (delta_e > threshold).astype(
        np.uint8) * 255

    return delta_e, mask
```

18.3 色彩區域分割

HSV 色彩空間最適合進行色彩區域分割。H 通道直接表示色調（色輪上的角度），S 通道表示飽和度（顏色純度），V 通道表示亮度。通過設定 H/S/V 的上下限，可以精確分割出特定顏色的區域。

```
[python]
def color_segmentation(img,
                       target='red'):
"""基於 HSV 的色彩區域分割
img:  BGR 影像
target: 目標顏色名稱"""

    hsv = cv2.cvtColor(
        img, cv2.COLOR_BGR2HSV)

# 常用顏色的 HSV 範圍
    # H: 0~180 (OpenCV), S: 0~255, V: 0~255
    ranges = {
        'red':    [(0,100,100),
                   (10,255,255),
                   (160,100,100),
                   (180,255,255)],
        'green':  [(35,80,80),
                   (85,255,255)],
        'blue':   [(95,100,100),
                   (130,255,255)],
        'yellow': [(20,100,100),
                   (35,255,255)],
    }

if target == 'red': # 紅色跨 0 度
        lo1, hi1, lo2, hi2 = [
```

```
            np.array(x)
            for x in ranges['red']]
        mask1 = cv2.inRange(
            hsv, lo1, hi1)
        mask2 = cv2.inRange(
            hsv, lo2, hi2)
        mask = mask1 | mask2
    else:
        lo = np.array(ranges[target][0])
        hi = np.array(ranges[target][1])
        mask = cv2.inRange(
            hsv, lo, hi)

# 形態學清理
    k = cv2.getStructuringElement(
        cv2.MORPH_ELLIPSE, (5,5))
    mask = cv2.morphologyEx(
        mask, cv2.MORPH_OPEN, k)
    mask = cv2.morphologyEx(
        mask, cv2.MORPH_CLOSE, k)

# 提取彩色區域
    result = cv2.bitwise_and(
        img, img, mask=mask)

    return mask, result
```

18.4 色彩一致性檢測

在批次生產中，不同批次之間的顏色偏差是常見問題。色彩一致性檢測比較目前產品的顏色統計量與歷史基準，判斷是否有系統性的色偏。

```
[python]
def color_consistency_check(
        img, roi, reference_stats,
        tolerance=3.0):
"""色彩一致性檢測
img:      BGR 影像
roi:      ROI 區域 (x,y,w,h)
reference_stats: 參考統計量
      {'L_mean','a_mean','b_mean',
       'L_std','a_std','b_std'}
tolerance:  容忍倍數 (std)"""

    x,y,w,h = roi
    region = img[y:y+h, x:x+w]

# 轉 Lab
    lab = cv2.cvtColor(
        region, cv2.COLOR_BGR2Lab)
    lab = lab.astype(np.float32)
    lab[:,:,0] = lab[:,:,0]*100.0/255.0
    lab[:,:,1] -= 128.0
    lab[:,:,2] -= 128.0
```

```
# 當前批次統計
    L_m = np.mean(lab[:,:,0])
    a_m = np.mean(lab[:,:,1])
    b_m = np.mean(lab[:,:,2])

# 與參考比較
    ref = reference_stats
    dL = abs(L_m - ref['L_mean'])
    da = abs(a_m - ref['a_mean'])
    db = abs(b_m - ref['b_mean'])

# Delta E（均值色差）
    de = np.sqrt(dL**2 + da**2 + db**2)

# 各通道偏離度
    dev_L = dL / max(ref['L_std'], 0.1)
    dev_a = da / max(ref['a_std'], 0.1)
    dev_b = db / max(ref['b_std'], 0.1)

    ok = (dev_L < tolerance and
          dev_a < tolerance and
          dev_b < tolerance)

    return {
        'delta_e': de,
        'dev_L': dev_L,
        'dev_a': dev_a,
        'dev_b': dev_b,
        'pass': ok,
        'current_Lab': (L_m, a_m, b_m)
    }
```

18.5 多通道融合瑕疵檢測

某些瑕疵在單一色彩通道中不明顯，但在多通道組合中可被揭露。多通道融合策略：

1. 分別在各通道中檢測異常
2. 使用邏輯運算（OR/AND）合併結果
3. 或計算通道間的統計偏離度

這種方法特別適合瑕疵可能表現為顏色變化、亮度變化或兩者兼有的情況。

```
[python]
def multi_channel_detect(img, ref,
                         thresh_lab=5.0,
                         thresh_gray=30):
"""多通道融合瑕疵檢測
結合灰度差異和色差兩個維度
img: 測試影像 (BGR)
ref: 參考影像 (BGR)"""

# 通道 1: 灰度差異
    g_test = cv2.cvtColor(
        img, cv2.COLOR_BGR2GRAY)
    g_ref = cv2.cvtColor(
        ref, cv2.COLOR_BGR2GRAY)
    diff_gray = cv2.absdiff(
        g_test, g_ref)
    _, mask_gray = cv2.threshold(
        diff_gray, thresh_gray, 255,
        cv2.THRESH_BINARY)

# 通道 2: Lab 色差
    lab_t = cv2.cvtColor(
        img, cv2.COLOR_BGR2Lab)
    lab_r = cv2.cvtColor(
        ref, cv2.COLOR_BGR2Lab)
    diff_lab = np.sqrt(np.sum(
        (lab_t.astype(np.float32)
         - lab_r.astype(np.float32))**2,
        axis=2))
    mask_color = (
        diff_lab > thresh_lab
    ).astype(np.uint8) * 255

# 通道 3: HSV 色調差異
    hsv_t = cv2.cvtColor(
        img, cv2.COLOR_BGR2HSV)
    hsv_r = cv2.cvtColor(
        ref, cv2.COLOR_BGR2HSV)
    diff_h = np.abs(
        hsv_t[:,:,0].astype(np.int16)
        - hsv_r[:,:,0].astype(np.int16))
    diff_h = np.minimum(
        diff_h, 180 - diff_h)
    mask_hue = (
        diff_h > 15
    ).astype(np.uint8) * 255

# 融合：任一通道檢測到即標記
    combined = (mask_gray | mask_color
                | mask_hue)

# 形態學清理
    k = cv2.getStructuringElement(
        cv2.MORPH_ELLIPSE, (5,5))
    combined = cv2.morphologyEx(
```

```
        combined, cv2.MORPH_OPEN, k)
    combined = cv2.morphologyEx(
        combined, cv2.MORPH_CLOSE, k)

    return {
        'mask_gray': mask_gray,
        'mask_color': mask_color,
        'mask_hue': mask_hue,
        'combined': combined
    }
```

案例研究：食品包裝印刷色差檢測

場景：零食包裝袋的印刷品質檢測。品牌 LOGO 顏色必須一致（Pantone 色號規格）。

品質標準：
- LOGO 區域平均 Delta E < 3.0（CIE76）
- 單一像素最大 Delta E < 8.0
- 色差面積佔比 < 5%

檢測流程：
1. ROI 提取：使用模板匹配定位 LOGO 區域
2. 轉換至 Lab 色彩空間
3. 計算每個像素的 Delta E（與標準色比較）
4. 統計分析：
   - 平均 Delta E = 2.1（OK，< 3.0）
   - 最大 Delta E = 6.2（OK，< 8.0）
   - 超標面積 = 2.3%（OK，< 5%）
5. 結果：PASS

實施效果：
- 每分鐘檢測 120 個包裝
- 色差檢出率 > 99%
- 客訴率降低 85%

案例研究：紡織品染色均勻性

場景：布料染色後的均勻性檢測。要求整匹布的顏色偏差 < 1.5 Delta E。

挑戰：
- 布料表面有紋理，像素級比較會產生大量雜訊
- 染色不均通常是漸變的（非銳利邊界）
- 需要區域級（而非像素級）的色差評估

方法：
1. 將布料影像分割為 N x M 個小區塊（如 50x50 像素）
2. 計算每個區塊的平均 Lab 值
3. 計算所有區塊之間的最大 Delta E
4. 繪製色差熱力圖定位不均勻區域
5. 判定：max(Delta E 所有區塊對) < 1.5

結果：成功檢測出邊緣到中心的色差梯度問題，幫助調整染缸浴比。

提示

色彩校正（白平衡）：

1. 在每次檢測開始前，拍攝標準白色參考板
2. 計算白平衡校正係數：gain_R, gain_G, gain_B
3. 對所有檢測影像套用校正
4. 這確保了不同時間點的色彩一致性
5. 使用 X-Rite ColorChecker 等標準色卡可進行更精確的多點色彩校正
6. 定期（每班次或每小時）重新校正白平衡

注意

光源色溫對色彩檢測的影響：

1. 不同色溫的光源會使同一物體呈現不同顏色。5500K 日光色最接近自然色
2. LED 光源的色溫可能隨時間和溫度漂移。使用恆溫驅動和定期校正
3. 混合光源（如 LED + 日光）會導致色彩不一致。必須隔離環境光
4. 不同批次的 LED 可能有色溫差異。購買時要求相同 bin 號
5. 相機感測器的色彩回應也會隨溫度變化。精密色彩檢測需要溫控環境

第 19 章

進階輪廓分析

前面章節介紹了基本的輪廓提取和特徵計算。本章深入探討進階輪廓分析技術，包括輪廓層次結構解析、形狀描述與匹配、最小外接形狀擬合、輪廓距離計算以及基於多特徵的缺陷分類引擎。這些技術使得瑕疵檢測系統能夠精確描述和分類各種複雜形狀的缺陷。

19.1 輪廓層次結構

OpenCV 的 findContours 可以返回輪廓的層次結構（hierarchy），描述輪廓之間的父子關係和嵌套關係。不同的檢索模式會產生不同的層次結構：

- RETR_EXTERNAL：只返回最外層輪廓（無層次）
- RETR_LIST：返回所有輪廓，無父子關係（扁平列表）
- RETR_CCOMP：返回兩層結構（外輪廓和內孔洞）
- RETR_TREE：返回完整的層次樹結構

hierarchy 的每個條目格式：[next, prev, child, parent]
- next：同層次的下一個輪廓索引（-1 表示無）
- prev：同層次的上一個輪廓索引
- child：第一個子輪廓索引
- parent：父輪廓索引

```
[python]
import cv2
import numpy as np

def analyze_hierarchy(binary):
"""解析輪廓層次結構
識別外輪廓、孔洞和嵌套結構"""

    contours, hierarchy = cv2.findContours(
        binary, cv2.RETR_TREE,
        cv2.CHAIN_APPROX_SIMPLE)

    if hierarchy is None:
        return [], [], []

    hier = hierarchy[0]  # shape: (N, 4)

outer = []  # 最外層輪廓
holes = []  # 孔洞（有父輪廓的）
nested = []  # 嵌套物件（孔洞中的物件）

    for i, h in enumerate(hier):
        nxt, prev, child, parent = h
```

```
        if parent == -1:
    # 頂層輪廓
            area = cv2.contourArea(
                contours[i])
            n_holes = 0
            c = child
            while c != -1:
                n_holes += 1
                c = hier[c][0]  # next
            outer.append({
                'idx': i,
                'contour': contours[i],
                'area': area,
                'holes': n_holes
            })
        elif hier[parent][3] == -1:
    # 父輪廓是頂層 -> 這是孔洞
            holes.append({
                'idx': i,
                'contour': contours[i],
                'area': cv2.contourArea(
                    contours[i]),
                'parent': parent
            })
        else:
    # 更深層嵌套
            nested.append({
                'idx': i,
                'contour': contours[i],
                'depth': get_depth(
                    hier, i)
            })

    return outer, holes, nested

def get_depth(hier, idx):
 """計算輪廓在層次樹中的深度"""
    depth = 0
    current = idx
    while hier[current][3] != -1:
        depth += 1
        current = hier[current][3]
    return depth
```

19.2 輪廓近似與形狀描述

輪廓可以用多種方式描述和簡化：

- 多邊形近似（approxPolyDP）：用更少的頂點近似輪廓
- 凸包（convexHull）：最小凸多邊形
- 凸包缺陷（convexityDefects）：輪廓與凸包的偏差
- Hu 矩：旋轉、縮放和平移不變的形狀描述子
- matchShapes：基於 Hu 矩的形狀比較

公式：Hu 矩不變量

基於中心矩 mu_pq 的歸一化中心矩:

```
eta_pq = mu_pq / mu_00^((p+q)/2 + 1)
```

7 個 Hu 矩不變量 (h1~h7):

```
h1 = eta_20 + eta_02
h2 = (eta_20-eta_02)^2
     + 4*eta_11^2
h3 = (eta_30-3*eta_12)^2
     + (3*eta_21-eta_03)^2
...
h7 = (3*eta_21-eta_03)
     *(eta_30+eta_12)*[...]
     - (eta_30-3*eta_12)
     *(eta_21+eta_03)*[...]
```

Hu 矩對平移、旋轉和縮放不變。h1~h6 對鏡像也不變，h7 的符號在鏡像時翻轉。在形狀匹配中非常有用。

```
[python]
def shape_analysis(contour, ref_contour):
"""全面的形狀分析與匹配
contour:    待分析輪廓
ref_contour: 參考形狀輪廓"""

# 1. 多邊形近似
    epsilon = 0.02 * cv2.arcLength(
        contour, True)
    approx = cv2.approxPolyDP(
        contour, epsilon, True)
    n_vertices = len(approx)

# 2. 凸包和凸包缺陷
    hull = cv2.convexHull(contour)
    hull_idx = cv2.convexHull(
        contour, returnPoints=False)
    if len(hull_idx) > 3 and \
       len(contour) > 3:
        defects = cv2.convexityDefects(
            contour, hull_idx)
    else:
        defects = None

# 分析凸包缺陷
    max_defect_depth = 0
    n_defects = 0
    if defects is not None:
        for d in defects:
            s, e, f, depth = d[0]
    # depth 單位是像素*256
            real_depth = depth / 256.0
            if real_depth > 5:
                n_defects += 1
```

```
            max_defect_depth = max(
                max_defect_depth,
                real_depth)

# 3. Hu 矩
    moments = cv2.moments(contour)
    hu = cv2.HuMoments(moments)
# 對數尺度（便於比較）
    hu_log = -np.sign(hu) * np.log10(
        np.abs(hu) + 1e-30)

# 4. 形狀匹配（與參考比較）
# 方法 1: I1 (最常用)
    match_1 = cv2.matchShapes(
        contour, ref_contour,
        cv2.CONTOURS_MATCH_I1, 0)
# 方法 2: I2
    match_2 = cv2.matchShapes(
        contour, ref_contour,
        cv2.CONTOURS_MATCH_I2, 0)
# 方法 3: I3
    match_3 = cv2.matchShapes(
        contour, ref_contour,
        cv2.CONTOURS_MATCH_I3, 0)

    return {
        'n_vertices': n_vertices,
        'n_convex_defects': n_defects,
        'max_defect_depth':
            max_defect_depth,
        'hu_moments': hu_log.ravel(),
        'match_I1': match_1,
        'match_I2': match_2,
        'match_I3': match_3,
    }
```

19.3 最小外接形狀

將輪廓擬合到標準幾何形狀（矩形、圓、橢圓），可以量化形狀特徵並與規格比較。OpenCV 提供多種擬合方法：

```
[python]
def fit_shapes(contour):
"""擬合各種最小外接形狀"""
    results = {}

# 1. 最小外接旋轉矩形
    rect = cv2.minAreaRect(contour)
    center, (w, h), angle = rect
    results['rect'] = {
        'center': center,
        'width': max(w, h),
        'height': min(w, h),
        'angle': angle,
        'aspect_ratio': max(w,h) /
```

```
            (min(w,h) + 1e-6)
    }

# 2. 最小外接圓
    (cx,cy), radius = \
        cv2.minEnclosingCircle(contour)
    area = cv2.contourArea(contour)
    circle_area = np.pi * radius**2
    results['circle'] = {
        'center': (cx, cy),
        'radius': radius,
        'fill_ratio':
            area / (circle_area + 1e-6)
    }

# 3. 擬合橢圓（需 >= 5 點）
    if len(contour) >= 5:
        ellipse = cv2.fitEllipse(contour)
        e_center, (e_ma, e_mi), e_angle = \
            ellipse
        results['ellipse'] = {
            'center': e_center,
            'major_axis': max(e_ma, e_mi),
            'minor_axis': min(e_ma, e_mi),
            'angle': e_angle,
            'eccentricity': np.sqrt(
                1 - (min(e_ma,e_mi)/
                     max(e_ma,e_mi))**2)
        }

# 4. 擬合直線
    if len(contour) >= 2:
        line = cv2.fitLine(
            contour, cv2.DIST_L2,
            0, 0.01, 0.01)
        vx, vy, x0, y0 = line.ravel()
        results['line'] = {
            'direction': (vx, vy),
            'point': (x0, y0),
            'angle_deg': np.degrees(
                np.arctan2(vy, vx))
        }

    return results
```

19.4 輪廓距離與相似度

比較兩個輪廓的相似程度，在模板比對和缺陷分類中非常重要。常用的輪廓距離度量包括 Hausdorff 距離和基於 Hu 矩的匹配距離。

公式：Hausdorff 距離

```
h(A,B) = max{ min{d(a,b) : b in B}
              : a in A }

H(A,B) = max{ h(A,B), h(B,A) }
```

其中 d(a,b) 為兩點間的歐式距離

Hausdorff 距離衡量兩個點集之間的最大不匹配程度。值越小表示兩個輪廓越相似。對異常值敏感，實務中常使用百分位數版本（如第 95 百分位）。

```
[python]
from scipy.spatial.distance import \
    directed_hausdorff

def contour_similarity(cnt1, cnt2):
 """計算兩個輪廓的多種相似度指標"""

 # 1. Hausdorff 距離
    pts1 = cnt1.reshape(-1, 2).astype(
        np.float64)
    pts2 = cnt2.reshape(-1, 2).astype(
        np.float64)
    h_fwd = directed_hausdorff(
        pts1, pts2)[0]
    h_bwd = directed_hausdorff(
        pts2, pts1)[0]
    hausdorff = max(h_fwd, h_bwd)

 # 2. 平均最近點距離
    from scipy.spatial import cKDTree
    tree2 = cKDTree(pts2)
    dists1, _ = tree2.query(pts1)
    tree1 = cKDTree(pts1)
    dists2, _ = tree1.query(pts2)
    avg_dist = (
        np.mean(dists1) +
        np.mean(dists2)) / 2

 # 3. matchShapes (Hu 矩)
    match_hu = cv2.matchShapes(
        cnt1, cnt2,
        cv2.CONTOURS_MATCH_I1, 0)

 # 4. 面積比
    a1 = cv2.contourArea(cnt1)
    a2 = cv2.contourArea(cnt2)
    area_ratio = min(a1,a2) / (
        max(a1,a2) + 1e-6)

 # 5. 圓度比較
    p1 = cv2.arcLength(cnt1, True)
    p2 = cv2.arcLength(cnt2, True)
    c1 = 4*np.pi*a1/(p1**2+1e-6)
```

```
    c2 = 4*np.pi*a2/(p2**2+1e-6)
    circ_diff = abs(c1 - c2)

    return {
        'hausdorff': hausdorff,
        'avg_dist': avg_dist,
        'hu_match': match_hu,
        'area_ratio': area_ratio,
        'circ_diff': circ_diff
    }
```

19.5 缺陷輪廓分類策略

在工業檢測中，不同類型的瑕疵有不同的輪廓特徵。通過計算多個輪廓特徵並建立規則引擎，可以自動將檢測到的缺陷分類為不同的瑕疵類型。

瑕疵類型	面積	圓度	長寬比	實心度
刮痕	中等	< 0.2	> 5.0	> 0.6
凹坑	小~中	> 0.7	~ 1.0	> 0.85
缺角	大	< 0.5	1~3	< 0.7
氣泡	小	> 0.8	~ 1.0	> 0.9
裂紋	小~中	< 0.15	> 8.0	> 0.5
污漬	中~大	0.3~0.7	1~3	> 0.8
毛邊	小	< 0.3	> 3.0	< 0.6

```
[python]
def classify_defect_contour(contour,
                            gray_img):
 """基於多特徵的缺陷輪廓分類器
 contour: 單個瑕疵輪廓
 gray_img: 灰度影像（計算亮度特徵）"""

 # --- 幾何特徵 ---
    area = cv2.contourArea(contour)
    peri = cv2.arcLength(contour, True)
    if peri < 1e-6:
        return 'noise', 0.0, {}

    circ = 4*np.pi*area / (peri**2)

 # 最小外接旋轉矩形
    rect = cv2.minAreaRect(contour)
    w, h = rect[1]
    if w < 1e-6 or h < 1e-6:
        return 'noise', 0.0, {}
    aspect = max(w,h) / min(w,h)

 # 凸包
    hull = cv2.convexHull(contour)
    hull_area = cv2.contourArea(hull)
```

```
    solidity = area / (
        hull_area + 1e-6)

# 凸包缺陷數
    hull_idx = cv2.convexHull(
        contour, returnPoints=False)
    n_defects = 0
    if len(hull_idx) > 3 and \
       len(contour) > 3:
        defs = cv2.convexityDefects(
            contour, hull_idx)
        if defs is not None:
            for d in defs:
                if d[0][3]/256.0 > 3:
                    n_defects += 1

# 外接矩形填充率
    bx,by,bw,bh = cv2.boundingRect(
        contour)
    extent = area / (bw * bh + 1e-6)

# --- 亮度特徵 ---
    mask = np.zeros_like(gray_img)
    cv2.drawContours(
        mask, [contour], -1, 255, -1)
    mean_val = cv2.mean(
        gray_img, mask=mask)[0]
# 周圍亮度
    k = cv2.getStructuringElement(
        cv2.MORPH_ELLIPSE, (15,15))
    surround = cv2.dilate(mask, k)
    surround = surround - mask
    bg_val = cv2.mean(
        gray_img, mask=surround)[0]
    contrast = abs(
        mean_val - bg_val) / 255.0

# --- 特徵向量 ---
    features = {
        'area': area,
        'circularity': circ,
        'aspect_ratio': aspect,
        'solidity': solidity,
        'extent': extent,
        'n_defects': n_defects,
        'contrast': contrast,
        'is_dark': mean_val < bg_val
    }

# --- 規則引擎分類 ---
    if area < 30:
        label = 'noise'
        conf = 0.95
    elif circ < 0.15 and aspect > 8:
        label = 'crack'
```

```
            conf = 0.85
        elif circ < 0.25 and aspect > 4:
            label = 'scratch'
            conf = 0.80
        elif circ > 0.8 and aspect < 1.5:
            if area < 200:
                label = 'bubble'
                conf = 0.85
            else:
                label = 'pit'
                conf = 0.80
        elif solidity < 0.65:
            if n_defects > 2:
                label = 'chipping'
                conf = 0.75
            else:
                label = 'burr'
                conf = 0.70
        elif circ > 0.3 and aspect < 3:
            label = 'stain'
            conf = 0.70
        else:
            label = 'unknown'
            conf = 0.50

        return label, conf, features

# 使用範例
 def inspect_all_defects(binary, gray):
  """檢測並分類所有缺陷"""
        contours, _ = cv2.findContours(
            binary, cv2.RETR_EXTERNAL,
            cv2.CHAIN_APPROX_SIMPLE)

        results = []
        for cnt in contours:
            label, conf, feat = \
                classify_defect_contour(
                    cnt, gray)
            if label != 'noise':
                results.append({
                    'contour': cnt,
                    'label': label,
                    'confidence': conf,
                    'features': feat
                })

  # 按類型統計
        from collections import Counter
        types = Counter(
            r['label'] for r in results)
  print(f'缺陷統計: {dict(types)}')

        return results
```

案例研究：沖壓件邊緣缺陷分類

場景：金屬沖壓件的邊緣品質檢測。常見缺陷包括毛邊、缺角、裂紋和卷邊。

檢測流程：
1. 背光照明拍攝零件輪廓影像
2. Otsu 閾值分割出零件輪廓
3. 與標準輪廓（CAD 產生）進行 Hausdorff 距離比較
4. 距離 > 3 像素的區域標記為缺陷候選
5. 提取每個缺陷區域的輪廓
6. 多特徵分類：
   - 毛邊：向外凸出、長寬比 > 3、面積小
   - 缺角：向內缺損、面積大、圓度低
   - 裂紋：從邊緣延伸、極細長、長寬比 > 8
   - 卷邊：邊緣附近的雙重輪廓、灰度漸變

判定標準：
- 毛邊長度 < 0.3mm：OK
- 毛邊長度 0.3~0.5mm：Minor
- 毛邊長度 > 0.5mm：Major
- 缺角面積 > 0.5mm2：Scrap
- 任何裂紋：Scrap

結果：分類準確率 94%，檢出率 98%。主要混淆：短毛邊 vs 小缺角。

提示

輪廓分析的最佳實踐：
1. 始終先用 RETR_EXTERNAL 排除內部雜訊，再用 RETR_TREE 分析結構
2. approxPolyDP 的 epsilon 值影響近似精度：0.01*周長 保留細節，0.05*周長 簡化形狀
3. matchShapes 返回值越小越相似（0 = 完全匹配）。閾值通常設 0.1~0.3
4. Hu 矩對非常細長的形狀效果不佳，此時應改用長寬比+方向特徵
5. 建立標準缺陷樣本庫，用 matchShapes 與樣本庫比對可提升分類準確度
6. 對於複雜形狀，組合多個特徵（幾何+亮度+紋理）比依賴單一特徵更穩健
7. 使用 mm 單位的特徵（而非像素）使規則在不同解析度下通用

第 20 章

深度學習瑕疵檢測進階

隨著深度學習技術的快速發展，基於 CNN 的目標檢測、語義分割和異常檢測方法已成為工業瑕疵檢測的重要工具。相比傳統方法，深度學習能自動學習特徵，在複雜場景中表現更優。本章涵蓋 YOLO 目標檢測、U-Net 語義分割、PatchCore/PADIM 無監督異常檢測、遷移學習策略、資料增強技巧以及模型評估與部署。

20.1 目標檢測：YOLO 系列應用於瑕疵檢測

YOLO（You Only Look Once）是單階段目標檢測模型，將影像分成網格，每個網格同時預測邊界框和類別機率。YOLO 系列以其高速和高精度著稱，非常適合產線即時瑕疵檢測。YOLOv5 到 YOLOv8 在工業場景中被廣泛使用。

公式：IoU（交並比）

IoU = Area(A ∩ B) / Area(A ∪ B)

A = 預測邊界框
B = 真實邊界框

IoU = 1 : 完美重合
IoU = 0 : 完全不重疊
IoU > 0.5 : 通常視為正確檢測

IoU 是目標檢測中最核心的評估指標，用於衡量預測框與真實框的重疊程度。

公式：NMS（非極大值抑制）

輸入: B = {b1, b2, ..., bn} (候選框集合)
S = {s1, s2, ..., sn} (對應信心分數)

步驟:
1. 選出最高分數的框 b_max
2. 移除所有與 b_max 的 IoU > T_nms 的框
3. 重複直到無剩餘框

T_nms 通常取 0.45 ~ 0.65

NMS 用於消除重複檢測。同一瑕疵可能產生多個重疊框，NMS 僅保留信心最高的一個。

```
 [python]
 from ultralytics import YOLO
 import cv2

# 載入預訓練或自訓練模型
model = YOLO('best.pt') # 自訓練的瑕疵檢測模型

# 推論
 img = cv2.imread('test_part.jpg')
 results = model(img, conf=0.25, iou=0.45)

# 解析結果
 for r in results:
     boxes = r.boxes
     for box in boxes:
         x1, y1, x2, y2 = box.xyxy[0].int().tolist()
         conf = float(box.conf[0])
         cls = int(box.cls[0])
         label = model.names[cls]
         print(f'{label}: {conf:.2f} '
               f'@ ({x1},{y1})-({x2},{y2})')

    # 繪製
         cv2.rectangle(img, (x1,y1), (x2,y2),
                       (0,0,255), 2)
         cv2.putText(img, f'{label} {conf:.2f}',
                     (x1, y1-5),
                     cv2.FONT_HERSHEY_SIMPLEX,
                     0.5, (0,0,255), 1)
```

版本	特點	速度(FPS)	適用場景
YOLOv5	PyTorch 原生, 社群活躍	~140	通用瑕疵檢測
YOLOv7	高精度, E-ELAN 架構	~120	高精度需求
YOLOv8	統一框架, 支援分割	~160	檢測+分割場景
YOLOv9	PGI+GELAN, 資訊保留	~130	小目標瑕疵
YOLO11	輕量化, C3k2 模組	~180	邊緣部署

20.2 語義分割：U-Net 用於瑕疵像素級分割

語義分割為每個像素分配類別標籤，實現像素級的瑕疵定位。U-Net 是最經典的分割架構，採用編碼器-解碼器結構和跳躍連接，在醫學影像和工業檢測中表現優異。跳躍連接將編碼器的低層特徵傳遞給解碼器，幫助恢復空間細節，這對精確定位瑕疵邊界至關重要。

公式：Dice Loss

Dice = 2 * |A ∩ B| / (|A| + |B|)

```
Dice Loss = 1 - Dice

  = 1 - 2*SUM(p_i * g_i) / (SUM(p_i) + SUM(g_i))
```

p_i = 預測機率

g_i = 真實標籤 (0 或 1)

Dice Loss 直接優化前景/背景的重疊度。在瑕疵佔比極小時（類別不平衡），比 BCE 更穩定，因為它不會被大量背景像素主導。

公式：BCE + Dice 組合損失

```
Loss = alpha * BCE + (1-alpha) * Dice_Loss

BCE = -1/N * SUM[g*log(p) + (1-g)*log(1-p)]
```

alpha 通常取 0.5

組合損失結合了 BCE 的像素級精確優化和 Dice 的全局重疊度優化，效果通常優於單獨使用任何一種。

```
[python]
import torch
import torch.nn as nn

class UNetBlock(nn.Module):
    def __init__(self, in_c, out_c):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(in_c, out_c, 3, padding=1),
            nn.BatchNorm2d(out_c),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_c, out_c, 3, padding=1),
            nn.BatchNorm2d(out_c),
            nn.ReLU(inplace=True))
    def forward(self, x):
        return self.conv(x)

class SimpleUNet(nn.Module):
    def __init__(self, in_ch=1, out_ch=1):
        super().__init__()
        self.enc1 = UNetBlock(in_ch, 64)
        self.enc2 = UNetBlock(64, 128)
        self.pool = nn.MaxPool2d(2)
        self.bottleneck = UNetBlock(128, 256)
        self.up2 = nn.ConvTranspose2d(
            256, 128, 2, stride=2)
        self.dec2 = UNetBlock(256, 128)
        self.up1 = nn.ConvTranspose2d(
            128, 64, 2, stride=2)
        self.dec1 = UNetBlock(128, 64)
        self.out_conv = nn.Conv2d(64, out_ch, 1)
```

```
    def forward(self, x):
        e1 = self.enc1(x)
        e2 = self.enc2(self.pool(e1))
        b = self.bottleneck(self.pool(e2))
        d2 = self.dec2(torch.cat(
            [self.up2(b), e2], dim=1))
        d1 = self.dec1(torch.cat(
            [self.up1(d2), e1], dim=1))
        return torch.sigmoid(self.out_conv(d1))
```

20.3 無監督異常檢測進階：PatchCore, PADIM

無監督異常檢測方法僅需良品影像即可訓練，無需收集和標註瑕疵樣本。這在工業場景中極具價值，因為瑕疵樣本稀少且類型多變。

PatchCore 的核心思想是建立「正常特徵記憶庫」：
1. 使用預訓練 CNN（如 ResNet）提取良品影像的中間層特徵
2. 將所有良品的 patch 特徵收集為記憶庫
3. 使用 coreset 採樣壓縮記憶庫（保留代表性特徵）
4. 測試時，計算每個 patch 特徵到記憶庫的最近鄰距離
5. 距離越大，越可能是異常

PADIM（Patch Distribution Modeling）則用多元高斯分佈對每個位置的正常特徵建模。測試時計算 Mahalanobis 距離作為異常分數。

```
 [python]
# PatchCore 推論流程 (anomalib)
 from anomalib.models import Patchcore
 from anomalib.data import MVTec
 from anomalib.engine import Engine

# 配置
 model = Patchcore(
     backbone='wide_resnet50_2',
     layers=['layer2', 'layer3'],
     coreset_sampling_ratio=0.1,
     num_neighbors=9
 )

# 資料模組
 datamodule = MVTec(
     root='./datasets/MVTec',
     category='bottle',
     image_size=(256, 256),
     train_batch_size=32
 )

# 訓練（僅使用良品）與推論
 engine = Engine()
 engine.fit(model, datamodule=datamodule)
 predictions = engine.predict(
```

```
    model, datamodule=datamodule)

# 每張影像會得到:
# - anomaly_map: 像素級異常分數圖
# - pred_score: 影像級異常分數
# - pred_label: 0=正常, 1=異常
```

20.4 遷移學習與微調策略

工業瑕疵檢測的資料量通常有限（幾十到幾百張）。遷移學習利用在大規模資料集（ImageNet）上預訓練的模型作為起點，僅需少量目標域資料即可達到良好效果。

策略選擇：
- 資料極少（< 100 張）：僅訓練最後分類層，凍結所有卷積層
- 資料中等（100~1000 張）：凍結前半部分，微調後半部分
- 資料充足（> 1000 張）：全網路微調，使用較小學習率

學習率排程建議：使用差異學習率（底層小、頂層大）和 CosineAnnealing 排程器。

```
 [python]
 import torch
 import torchvision.models as models
 import torch.nn as nn

# 載入預訓練 ResNet50
 model = models.resnet50(pretrained=True)

# 策略: 凍結前 3/4 的層
 for name, param in model.named_parameters():
     if 'layer4' not in name and 'fc' not in name:
         param.requires_grad = False

# 替換分類頭
num_defect_types = 5  # OK + 4 種瑕疵
 model.fc = nn.Sequential(
     nn.Dropout(0.3),
     nn.Linear(2048, 256),
     nn.ReLU(),
     nn.Dropout(0.2),
     nn.Linear(256, num_defect_types)
 )

# 差異學習率
 optimizer = torch.optim.AdamW([
     {'params': model.layer4.parameters(),
      'lr': 1e-4},
     {'params': model.fc.parameters(),
      'lr': 1e-3},
 ], weight_decay=1e-4)

# CosineAnnealing 排程
 scheduler = torch.optim.lr_scheduler\
```

```
    .CosineAnnealingLR(optimizer, T_max=50)
```

20.5 資料增強策略

資料增強是解決工業檢測資料不足的關鍵技術。通過對訓練影像施加隨機變換，人為擴充資料集的多樣性，提升模型的泛化能力。

常用增強方式：
- 幾何變換：旋轉、翻轉、縮放、平移、仿射
- 色彩變換：亮度/對比度抖動、色相偏移、灰度化
- 遮擋類：CutOut（隨機遮擋）、GridMask
- 混合類：MixUp（線性混合兩張圖）、CutMix（區域替換）

```
 [python]
 import albumentations as A
 from albumentations.pytorch import ToTensorV2

# 瑕疵檢測專用資料增強管道
 train_transform = A.Compose([
  # 幾何變換
     A.HorizontalFlip(p=0.5),
     A.VerticalFlip(p=0.5),
     A.RandomRotate90(p=0.5),
     A.ShiftScaleRotate(
         shift_limit=0.05,
         scale_limit=0.1,
         rotate_limit=15, p=0.5),

  # 色彩變換
     A.RandomBrightnessContrast(
         brightness_limit=0.15,
         contrast_limit=0.15, p=0.5),
     A.HueSaturationValue(
         hue_shift_limit=10,
         sat_shift_limit=20,
         val_shift_limit=15, p=0.3),

  # 雜訊與模糊
     A.GaussNoise(var_limit=(5,25), p=0.3),
     A.GaussianBlur(
         blur_limit=(3,5), p=0.2),

  # CutOut 遮擋
     A.CoarseDropout(
         max_holes=4, max_height=32,
         max_width=32, p=0.3),

  # 正規化
     A.Normalize(
         mean=[0.485, 0.456, 0.406],
         std=[0.229, 0.224, 0.225]),
     ToTensorV2()
```

```
])
```

20.6 模型評估與部署

深度學習模型的評估不能僅看整體準確率。工業場景中需要關注：

- 混淆矩陣：各類別的具體表現
- PR 曲線：不同閾值下的精確率-召回率取捨
- AP/mAP：目標檢測的標準指標
- 推論速度：是否滿足產線節拍時間

```
[python]
import numpy as np
from sklearn.metrics import (
    confusion_matrix, precision_recall_curve,
    average_precision_score
)

def compute_metrics(y_true, y_scores, y_pred):
 """計算完整的評估指標"""
 # 混淆矩陣
    cm = confusion_matrix(y_true, y_pred)
    tn, fp, fn, tp = cm.ravel()

 # 基本指標
    precision = tp / (tp + fp + 1e-8)
    recall = tp / (tp + fn + 1e-8)
    f1 = 2*precision*recall / (
        precision + recall + 1e-8)
    specificity = tn / (tn + fp + 1e-8)

 # PR 曲線與 AP
    prec_curve, rec_curve, thresholds = \
        precision_recall_curve(y_true, y_scores)
    ap = average_precision_score(
        y_true, y_scores)

 # 漏檢率（最關鍵）
    escape_rate = fn / (tp + fn + 1e-8)

    return {
        'confusion_matrix': cm,
        'precision': precision,
        'recall': recall,
        'f1': f1,
        'specificity': specificity,
        'AP': ap,
        'escape_rate': escape_rate,
        'pr_curve': (prec_curve, rec_curve)
    }
```

案例研究：MVTec AD 資料集異常檢測

MVTec AD 是工業異常檢測的標準基準資料集，包含 15 個類別（5 種紋理 + 10 種物件）。

實驗設定：
- 訓練集：僅良品影像（每類約 200-300 張）
- 測試集：良品 + 多種瑕疵影像
- 評估指標：Image-level AUROC 和 Pixel-level AUROC

各方法在 MVTec AD 上的表現：
- PatchCore: Image AUROC 99.1%, Pixel AUROC 98.1%
- PADIM: Image AUROC 95.3%, Pixel AUROC 97.5%
- 自編碼器: Image AUROC 86.0%, Pixel AUROC 91.0%
- STFPM: Image AUROC 95.5%, Pixel AUROC 97.0%

結論：PatchCore 在大多數類別上表現最佳，但記憶庫佔用空間較大。PADIM 在速度與精度之間取得良好平衡。

提示

小樣本策略：
1. 優先使用無監督方法（PatchCore/PADIM），僅需良品樣本
2. 使用預訓練模型的特徵提取器，避免從頭訓練
3. 積極使用資料增強，至少擴充 5-10 倍
4. 考慮合成瑕疵：在良品上人工添加模擬瑕疵（貼片、變色、畸變）
5. 使用 few-shot learning 或 meta-learning 技術
6. 先用簡單模型驗證可行性，確認後再使用複雜模型

注意

深度學習的陷阱：
1. 過擬合：訓練集表現好但測試集差，特別是在小資料集上
2. 資料偏差：訓練集與實際產線影像分佈不同（照明、角度、批次差異）
3. 黑箱問題：模型預測難以解釋，品質人員難以信任
4. 部署差距：開發環境的精度無法在產線環境重現
5. 維護成本：模型需要定期重新訓練以適應產品或環境變化
6. 不要迷信深度學習——簡單場景用傳統方法更穩定、更快、更易維護

第 21 章

進階分割方法

影像分割是將影像分成多個有意義區域的過程。在瑕疵檢測中，精確的分割是後續分析的基礎。除了第 2 章介紹的閾值分割外，本章涵蓋更進階的分割技術：分水嶺、GrabCut、超像素、區域生長，以及深度學習語義分割在工業中的應用。這些方法在處理複雜場景（重疊物體、漸變邊界、不均勻背景）時，表現優於簡單閾值方法。

21.1 分水嶺分割

分水嶺演算法將灰度影像視為地形圖（像素值=海拔高度）。從標記的「種子點」開始注水，水位上升直到不同盆地的水相遇，相遇處即為分割邊界。

關鍵步驟：
1. 計算距離變換（前景像素到最近背景像素的距離）
2. 在距離圖上尋找局部極大值作為標記（種子）
3. 執行 watershed 分割

常見問題：過分割（標記太多）。解決方法：合併小區域或使用更嚴格的標記生成。

公式：距離變換

```
D(x,y) = min{ dist((x,y), (i,j)) :
    I(i,j) = 背景 }
```

歐式距離: dist = sqrt(dx^2 + dy^2)
棋盤距離: dist = max(|dx|, |dy|)
城市距離: dist = |dx| + |dy|

距離變換後，物體中心的距離值最大。對距離圖設閾值可分離接觸的物體。

公式：標記生成

```
markers = label(D > alpha * max(D))
```

alpha = 0.5 ~ 0.7（控制標記的嚴格程度）
alpha 越大 -> 標記越少 -> 欠分割
alpha 越小 -> 標記越多 -> 過分割

標記的品質直接決定分水嶺分割的效果。可結合形態學腐蝕生成更穩健的標記。

```
[python]
import cv2
import numpy as np

def watershed_segment(gray, binary):
 """完整的 watershed 分割流程
 gray:  灰度影像
 binary: 二值前景遮罩"""
 # 1. 形態學開運算去噪
    kernel = cv2.getStructuringElement(
        cv2.MORPH_ELLIPSE, (3,3))
    opening = cv2.morphologyEx(
        binary, cv2.MORPH_OPEN, kernel,
        iterations=2)

 # 2. 確定背景區域（膨脹）
    sure_bg = cv2.dilate(
        opening, kernel, iterations=3)

 # 3. 距離變換 -> 確定前景
    dist = cv2.distanceTransform(
        opening, cv2.DIST_L2, 5)
    _, sure_fg = cv2.threshold(
        dist, 0.5 * dist.max(), 255, 0)
    sure_fg = sure_fg.astype(np.uint8)

 # 4. 未知區域
    unknown = cv2.subtract(sure_bg, sure_fg)

 # 5. 標記連通域
    n_labels, markers = \
        cv2.connectedComponents(sure_fg)
    markers = markers + 1
    markers[unknown == 255] = 0

 # 6. 執行 watershed
    img_c = cv2.cvtColor(gray, cv2.COLOR_GRAY2BGR)
    markers = cv2.watershed(img_c, markers)

 # markers == -1 為邊界
    return markers, n_labels
```

21.2 GrabCut 互動分割

GrabCut 是一種半自動分割方法，使用高斯混合模型（GMM）分別對前景和背景建模，並通過圖切割（Graph Cut）優化進行分割。使用者僅需提供一個包含前景的矩形框或初始遮罩。

原理：

1. 使用者指定前景矩形框
2. 框外像素初始化為背景，框內為可能的前景
3. 用 GMM（通常 K=5）分別建模前景/背景的顏色分佈
4. 構建圖模型，邊權重由顏色相似度和 GMM 分配決定
5. 最小割/最大流求解最優分割

6. 可迭代優化以改善結果

```
[python]
import cv2
import numpy as np

def grabcut_segment(img, rect=None,
                    mask_hint=None,
                    n_iter=5):
"""GrabCut 分割
img:     BGR 彩色影像
rect:    前景矩形 (x,y,w,h)
mask_hint: 使用者提供的初始遮罩"""
    mask = np.zeros(
        img.shape[:2], dtype=np.uint8)
    bgd = np.zeros((1,65), np.float64)
    fgd = np.zeros((1,65), np.float64)

    if rect is not None:
        mode = cv2.GC_INIT_WITH_RECT
        cv2.grabCut(img, mask, rect,
                    bgd, fgd, n_iter, mode)
    elif mask_hint is not None:
        mask = mask_hint.copy()
        mode = cv2.GC_INIT_WITH_MASK
        cv2.grabCut(img, mask, None,
                    bgd, fgd, n_iter, mode)

# 提取前景
    fg_mask = np.where(
        (mask == cv2.GC_FGD) |
        (mask == cv2.GC_PR_FGD), 1, 0
    ).astype(np.uint8)

    result = img * fg_mask[:,:,np.newaxis]
    return fg_mask, result

# 使用範例
img = cv2.imread('part.jpg')
rect = (50, 50, 400, 300) # 前景矩形
mask, result = grabcut_segment(img, rect=rect)
```

21.3 超像素分割：SLIC

SLIC（Simple Linear Iterative Clustering）將影像分割為多個緊湊且均勻的超像素區塊。每個超像素包含一群相似的相鄰像素。超像素作為後續分析的基本單位，取代單個像素，可大幅減少計算量並保留邊界資訊。

公式：SLIC 距離度量

```
d_c = sqrt((l_i-l_j)^2 + (a_i-a_j)^2 +
           (b_i-b_j)^2)

d_s = sqrt((x_i-x_j)^2 + (y_i-y_j)^2)

D = sqrt(d_c^2 + (d_s/S)^2 * m^2)
```

d_c = CIELAB 色彩距離
d_s = 空間距離
S = 超像素間距 sqrt(N/k)
m = 緊湊度因子 (通常 10~40)

m 越大，超像素越規則方形；m 越小，超像素越貼合邊緣。

```
[python]
import cv2
import numpy as np

def slic_segment(img, n_segments=200,
                 compactness=10):
"""SLIC 超像素分割 + 後處理"""
# 建立 SLIC 物件
    slic = cv2.ximgproc.createSuperpixelSLIC(
        img, algorithm=cv2.ximgproc.SLIC,
        region_size=int(
            np.sqrt(img.shape[0]*img.shape[1]
                    / n_segments)),
        ruler=compactness)

# 迭代
    slic.iterate(10)
    slic.enforceLabelConnectivity(25)

# 取得標籤與邊界
    labels = slic.getLabels()
    mask = slic.getLabelContourMask()

# 計算每個超像素的特徵
    n = slic.getNumberOfSuperpixels()
    features = []
    for i in range(n):
        region = img[labels == i]
        mean_color = region.mean(axis=0)
        std_color = region.std(axis=0)
        features.append({
            'label': i,
            'mean': mean_color,
            'std': std_color,
            'size': len(region)
        })

    return labels, mask, features
```

21.4 基於區域的分割：區域生長

區域生長從種子點開始，將相鄰像素與種子比較，如果相似度在閾值內則併入區域，不斷擴展直到沒有可合併的像素為止。適合分割具有均勻特性的區域。

```
[python]
import cv2
import numpy as np
from collections import deque

def region_growing(gray, seed, threshold=15):
 """區域生長分割
 gray:    灰度影像
 seed:    種子點 (x, y)
 threshold: 相似度閾值"""
    h, w = gray.shape
    visited = np.zeros((h, w), dtype=bool)
    mask = np.zeros((h, w), dtype=np.uint8)

    queue = deque([seed])
    seed_val = float(gray[seed[1], seed[0]])

    while queue:
        x, y = queue.popleft()
        if (x < 0 or x >= w or y < 0 or y >= h
                or visited[y, x]):
            continue

        visited[y, x] = True
        diff = abs(float(gray[y,x]) - seed_val)

        if diff <= threshold:
            mask[y, x] = 255
    # 8 鄰域擴展
            for dx in [-1, 0, 1]:
                for dy in [-1, 0, 1]:
                    if dx == 0 and dy == 0:
                        continue
                    queue.append(
                        (x+dx, y+dy))

    return mask
```

21.5 語義分割在工業中的應用

深度學習語義分割（如
DeepLabV3+）為每個像素分配類別標籤，實現瑕疵的精確像素級定位和分類。在多種瑕疵共存的場景中尤其有用。

DeepLabV3+ 的核心技術：
- 空洞卷積（Atrous Convolution）：擴大感受野而不降低解析度
- ASPP（Atrous Spatial Pyramid Pooling）：多尺度特徵融合
- 編碼器-解碼器結構：兼顧語義與空間細節

工業標註策略：
- 背景（類別 0）、瑕疵類型 A（類別 1）、瑕疵類型 B（類別 2）...
- 使用 CVAT 或 LabelMe 等工具進行多邊形標註
- 標註時注意瑕疵邊界的精確性

分割方法	速度	精度	人工介入
閾值分割	極快 (<1ms)	低~中	設定閾值
分水嶺	快 (~5ms)	中	標記生成
GrabCut	慢 (~100ms)	中~高	初始矩形/遮罩
SLIC 超像素	中 (~20ms)	中	參數調整
區域生長	中 (~30ms)	中	種子點選擇
U-Net	慢 (~50ms GPU)	高	標註訓練資料
DeepLabV3+	慢 (~80ms GPU)	極高	標註訓練資料

案例研究：重疊零件分割

場景：傳送帶上多個金屬墊圈重疊堆放，需要分離計數。

挑戰：簡單閾值無法分離接觸/重疊的零件。

解決方案：分水嶺分割
1. 前處理：CLAHE 增強 + 中值濾波
2. Otsu 閾值分割出前景
3. 距離變換 -> 找到每個零件的中心
4. 距離圖設閾值（0.6 * max）生成種子標記
5. 分水嶺分割分離重疊區域
6. 對每個分離的區域進行特徵分析

結果：成功分離 95% 的重疊零件。失敗案例多為嚴重重疊（面積覆蓋 > 50%）的零件。
改善：對失敗案例可結合深度學習實例分割（Mask R-CNN）。

提示

分割方法選擇指南：
1. 背景均勻、瑕疵對比明顯 -> 閾值分割（最快最簡單）
2. 背景不均勻但前景明確 -> 自適應閾值
3. 物體接觸/重疊需分離 -> 分水嶺分割
4. 需要互動式精確分割 -> GrabCut
5. 需要區域級特徵分析 -> SLIC 超像素
6. 均勻區域的精確提取 -> 區域生長
7. 多類瑕疵的像素級分類 -> 深度學習語義分割
原則：從最簡單的方法開始，僅在效果不足時升級。

第 22 章

即時處理與效能優化

工業產線要求檢測系統在嚴格的節拍時間（takt time）內完成每件產品的檢測。典型要求為每秒 5-30 件，意味著每件的處理時間僅 33-200 毫秒。本章探討如何分析效能瓶頸、應用加速技術、設計管線架構和利用 GPU，以構建滿足即時要求的檢測系統。

22.1 效能瓶頸分析

優化的第一步是找出瓶頸。典型瑕疵檢測流程中，各步驟的耗時差異很大。

常見瓶頸（按耗時排序）：

1. 深度學習推論（50-500 ms）
2. 影像擷取與傳輸（10-50 ms）
3. 複雜形態學運算（5-20 ms）
4. 頻率域濾波（5-15 ms）
5. 模板匹配（3-10 ms）
6. 閾值分割（< 1 ms）

```
[python]
import time
import cProfile
import cv2
import numpy as np

def profile_pipeline(img):
 """分析各步驟耗時"""
    timings = {}

    t0 = time.perf_counter()
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    timings['grayscale'] = (
        time.perf_counter() - t0) * 1000

    t0 = time.perf_counter()
    blur = cv2.GaussianBlur(gray, (5,5), 1.0)
    timings['blur'] = (
        time.perf_counter() - t0) * 1000

    t0 = time.perf_counter()
    _, binary = cv2.threshold(
        blur, 0, 255,
        cv2.THRESH_BINARY + cv2.THRESH_OTSU)
    timings['threshold'] = (
        time.perf_counter() - t0) * 1000

    t0 = time.perf_counter()
    k = cv2.getStructuringElement(
        cv2.MORPH_ELLIPSE, (5,5))
```

```
    clean = cv2.morphologyEx(
        binary, cv2.MORPH_OPEN, k)
    timings['morphology'] = (
        time.perf_counter() - t0) * 1000

    t0 = time.perf_counter()
    cnts, _ = cv2.findContours(
        clean, cv2.RETR_EXTERNAL,
        cv2.CHAIN_APPROX_SIMPLE)
    timings['contours'] = (
        time.perf_counter() - t0) * 1000

# 列印結果
    total = sum(timings.values())
    for step, ms in timings.items():
        pct = ms / total * 100
        print(f'{step:15s}: {ms:6.2f} ms '
              f'({pct:5.1f}%)')
    print(f'{"TOTAL":15s}: {total:6.2f} ms')
    return timings
```

22.2 影像處理加速技術

>> ROI 縮小搜索範圍

如果瑕疵只可能出現在影像的特定區域，僅處理該 ROI 可大幅減少計算量。例如：僅檢測零件表面區域，跳過背景。

>> 影像降取樣（多解析度金字塔）

先在低解析度影像上快速粗篩，找到可疑區域後再在原始解析度上精細檢測。降取樣 2x 後，處理速度提升約 4 倍。

>> 查找表（LUT）加速

預計算灰度映射表，避免重複的逐像素計算。特別適合自定義的亮度變換。

```
[python]
import cv2
import numpy as np

# --- ROI 處理 ---
def process_roi(img, roi):
    x, y, w, h = roi
    patch = img[y:y+h, x:x+w]
# 僅處理 ROI 區域
    result = cv2.GaussianBlur(patch, (5,5), 1)
    return result

# --- 多解析度金字塔 ---
def pyramid_detect(img, scale=0.5):
    small = cv2.resize(
        img, None, fx=scale, fy=scale)
# 粗篩：快速檢測
    _, mask_s = cv2.threshold(
        small, 0, 255,
        cv2.THRESH_BINARY + cv2.THRESH_OTSU)
```

```
    cnts, _ = cv2.findContours(
        mask_s, cv2.RETR_EXTERNAL,
        cv2.CHAIN_APPROX_SIMPLE)
 # 有可疑區域時才精細檢測
    if len(cnts) > 0:
   return full_inspect(img) # 原解析度
 return None # 快速通過

# --- LUT 加速 ---
 def apply_lut_gamma(img, gamma=1.5):
    lut = np.array([
        np.clip(pow(i/255.0, gamma)*255, 0, 255)
        for i in range(256)
    ], dtype=np.uint8)
    return cv2.LUT(img, lut)
```

22.3 多執行緒與管線化

瑕疵檢測流程可分解為三個獨立階段：影像擷取、影像處理、結果輸出。使用多執行緒管線（pipeline），三個階段並行運作，系統吞吐量由最慢的階段決定，而非三者之和。

```
 [python]
 import threading
 import queue
 import time
 import cv2

 class InspectionPipeline:
  """三級管線：擷取 -> 處理 -> 輸出"""

    def __init__(self, max_queue=5):
        self.q_capture = queue.Queue(max_queue)
        self.q_result = queue.Queue(max_queue)
        self.running = True

    def capture_worker(self, camera):
   """擷取執行緒"""
        while self.running:
            ret, frame = camera.read()
            if ret:
                ts = time.perf_counter()
                self.q_capture.put(
                    (frame, ts))

    def process_worker(self, inspect_fn):
   """處理執行緒"""
        while self.running:
            try:
                frame, ts = \
                    self.q_capture.get(
                        timeout=1)
            except queue.Empty:
                continue
            result = inspect_fn(frame)
```

```
            self.q_result.put(
                (frame, result, ts))

    def output_worker(self, output_fn):
"""輸出執行緒"""
        while self.running:
            try:
                frame, result, ts = \
                    self.q_result.get(
                        timeout=1)
            except queue.Empty:
                continue
            latency = (
                time.perf_counter()-ts)*1000
            output_fn(frame, result, latency)

    def start(self, camera, inspect_fn,
              output_fn):
        threads = [
            threading.Thread(
                target=self.capture_worker,
                args=(camera,)),
            threading.Thread(
                target=self.process_worker,
                args=(inspect_fn,)),
            threading.Thread(
                target=self.output_worker,
                args=(output_fn,)),
        ]
        for t in threads:
            t.daemon = True
            t.start()
        return threads
```

22.4 GPU 加速

GPU 並行計算能力可大幅加速影像處理和深度學習推論。

兩種方式：

1. OpenCV CUDA 模組：加速傳統影像處理（濾波、形態學、模板匹配）
2. TensorRT / ONNX Runtime：加速深度學習推論（通常 3-10 倍加速）

```
[python]
import cv2
import numpy as np

# --- OpenCV CUDA 加速 ---
def gpu_process(img):
 # 上傳到 GPU
    gpu_img = cv2.cuda_GpuMat()
    gpu_img.upload(img)

 # GPU 上執行高斯模糊
    gpu_blur = cv2.cuda.createGaussianFilter(
        cv2.CV_8UC1, cv2.CV_8UC1, (5,5), 1.0)
    gpu_result = gpu_blur.apply(gpu_img)

 # 下載回 CPU
    result = gpu_result.download()
    return result

# --- TensorRT 推論 ---
# 先將 PyTorch 模型轉為 ONNX -> TensorRT
 # torch.onnx.export(model, dummy, 'model.onnx')
 # trtexec --onnx=model.onnx --saveEngine=m.trt

# 使用 ONNX Runtime (更簡單)
import onnxruntime as ort

def onnx_infer(onnx_path, img):
    sess = ort.InferenceSession(
        onnx_path,
        providers=['CUDAExecutionProvider',
                   'CPUExecutionProvider'])
    blob = cv2.dnn.blobFromImage(
        img, 1/255.0, (640,640),
        swapRB=True)
    outputs = sess.run(None, {
        sess.get_inputs()[0].name: blob})
    return outputs
```

22.5 最佳化演算法選擇

不同演算法的時間複雜度差異巨大。根據即時性要求選擇合適的演算法至關重要。

演算法	時間複雜度	1MP 耗時	適用場景
閾值分割	O(N)	< 1 ms	快速粗篩
高斯模糊	O(N*k^2)	~2 ms	預處理
形態學	O(N*k^2)	~3 ms	後處理
Canny	O(N)	~3 ms	邊緣檢測
連通域	O(N)	~2 ms	Blob 分析
模板匹配	O(N*M)	~10 ms	定位/比對

演算法	時間複雜度	1MP 耗時	適用場景
GrabCut	O(N*k*iter)	~100 ms	精確分割

22.6 即時系統設計模式

根據觸發方式，即時檢測系統分為兩種模式：

1. 觸發模式：感測器偵測到零件到位時觸發拍照和檢測。適合離散零件。
2. 連續模式：相機持續擷取，軟體連續處理。適合連續卷材或輸送帶。

公式：系統吞吐量

```
throughput = 1 / max(t_capture, t_process,
                     t_output)

t_capture = 擷取+傳輸時間
t_process = 處理時間
t_output  = 輸出+通訊時間

要求: throughput >= production_rate

例: 產線速度 20 件/秒
      => max(t) <= 50 ms
```

管線化後吞吐量由最慢階段決定。如果處理是瓶頸，考慮多處理執行緒或 GPU 加速。

案例研究：每秒 30 件的產線即時檢測系統

場景：電子連接器高速檢測，要求每秒 30 件（33 ms/件）。

硬體配置：
- 線掃描相機（4096 像素），配頻閃光源
- 工業 PC（i7 + RTX 3060），GigE 介面
- IO 觸發 + PLC 通訊

軟體架構：
- 三級管線（擷取 / 處理 / IO 輸出）
- 處理流程：ROI 裁剪(1ms) -> 閾值(0.5ms) -> 形態學(1ms) -> Blob(1ms) -> 判定(0.5ms)
- 總處理時間：~4 ms << 33 ms（有充足餘量）

效能結果：
- 實際吞吐量：35 件/秒（餘量 17%）
- 平均延遲：12 ms（擷取 5ms + 處理 4ms + IO 3ms）
- 穩定運行 24 小時無丟幀

注意

即時系統的常見問題：

1. 記憶體洩漏：長時間運行後記憶體逐漸增長，最終導致系統變慢或崩潰
2. GC 暫停：Python 垃圾回收可能造成不可預期的延遲尖峰
3. 佇列溢出：處理速度跟不上擷取速度時，佇列無限增長
4. 溫度降頻：GPU/CPU 長時間滿載會過熱降頻，降低效能
5. 網路延遲：GigE 相機可能因網路擁塞丟幀
6. 未做壓力測試：開發環境的效能不代表產線的持續效能

第 23 章

系統穩健性與邊界案例

一個在實驗室中表現完美的檢測系統，在實際產線上可能因為環境變化、異常輸入或邊界情況而失效。系統穩健性（Robustness）是工業檢測系統的核心要求。本章探討影響穩健性的因素、如何處理異常情況、分析假陽性/假陰性的根因，以及建立系統化的穩健性測試和日誌追溯體系。

23.1 環境變化的影響

工業環境中的多種變化會影響檢測結果：

1. 溫度漂移：溫度變化導致相機增益漂移、LED 光源亮度衰減。白天與夜間的環境光差異也可能影響結果。
2. 光源老化：LED 光源隨使用時間逐漸衰減（通常 5000 小時後亮度降至 80%）。光源的光譜特性也可能隨老化改變。
3. 振動：設備振動導致影像模糊或位移。高速產線上特別嚴重。
4. 灰塵與污染：鏡頭或光源表面的灰塵積累導致影像品質下降。
5. 材料批次差異：不同批次的原材料在顏色、紋理上存在自然變異。

對策：
- 定期校正：每班次或每天使用標準參考片驗證系統
- 自動曝光補償：監控影像亮度並自動調整曝光
- 環境監控：記錄溫度、濕度，與檢測結果相關分析
- 預防性維護：定期清潔鏡頭和光源，按壽命更換 LED

23.2 自動曝光與白平衡補償

光源亮度隨時間和溫度變化，導致影像整體亮度偏移。自動曝光補償演算法可在每次拍攝後調整相機參數，維持影像亮度穩定。

```
[python]
import cv2
import numpy as np

class AutoExposure:
 """自動曝光調整控制器"""

    def __init__(self, target_mean=128,
                 tolerance=10,
                 roi=None):
        self.target = target_mean
        self.tol = tolerance
        self.roi = roi
        self.exposure = 10000  # us
        self.gain = 1.0
        self.history = []

    def compute_brightness(self, img):
  """計算 ROI 內的平均亮度"""
```

```
        if len(img.shape) == 3:
            gray = cv2.cvtColor(
                img, cv2.COLOR_BGR2GRAY)
        else:
            gray = img

        if self.roi:
            x, y, w, h = self.roi
            region = gray[y:y+h, x:x+w]
        else:
            region = gray

        return float(np.mean(region))

    def adjust(self, img):
"""根據當前影像調整曝光參數
回傳: (需調整, 新曝光值, 新增益)"""
        brightness = self.compute_brightness(img)
        self.history.append(brightness)

        diff = self.target - brightness

        if abs(diff) <= self.tol:
            return False, self.exposure, self.gain

# P 控制器
        ratio = self.target / max(
            brightness, 1)
        ratio = np.clip(ratio, 0.8, 1.25)

# 優先調整曝光，增益不變
        new_exp = int(self.exposure * ratio)
        new_exp = np.clip(new_exp, 100, 100000)
        self.exposure = int(new_exp)

        return True, self.exposure, self.gain

    def check_drift(self, window=100):
"""檢測亮度是否有持續漂移趨勢"""
        if len(self.history) < window:
            return False
        recent = self.history[-window:]
        trend = np.polyfit(
            range(window), recent, 1)[0]
# 斜率大於閾值表示有漂移
        return abs(trend) > 0.1
```

23.3 異常輸入處理

檢測系統必須能處理各種異常輸入，而非直接崩潰。常見異常：

- 空影像或全黑/全白影像（相機故障、擷取超時）
- 過曝或欠曝影像（光源異常、曝光設定錯誤）
- 模糊影像（對焦失敗、振動）
- 無零件或異物（零件缺失、錯位）

公式：模糊偵測：Laplacian 方差

```
blur_score = Var(Laplacian(I))

Laplacian = d^2I/dx^2 + d^2I/dy^2
```

blur_score 高 -> 影像清晰（邊緣多）
blur_score 低 -> 影像模糊

閾值: 通常 < 100 判定為模糊
(需依據具體場景校準)

Laplacian 是二階導數運算子，對邊緣敏感。清晰影像有大量邊緣，方差大；模糊影像邊緣弱，方差小。

```
[python]
import cv2
import numpy as np

def check_image_quality(img):
"""影像品質檢查，回傳 (通過, 品質報告)"""
    report = {}

# 1. 空影像檢查
    if img is None or img.size == 0:
  return False, {'error': '空影像'}

    gray = (cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
            if len(img.shape) == 3 else img)

# 2. 亮度檢查
    mean_val = float(np.mean(gray))
    report['brightness'] = mean_val
    if mean_val < 20:
        return False, {
    **report, 'error': '欠曝 (過暗)'}
    if mean_val > 240:
        return False, {
    **report, 'error': '過曝 (過亮)'}

# 3. 動態範圍檢查
    min_v, max_v = int(np.min(gray)), \
                   int(np.max(gray))
    dyn_range = max_v - min_v
    report['dynamic_range'] = dyn_range
    if dyn_range < 30:
        return False, {
            **report,
    'error': '動態範圍不足'}

# 4. 模糊偵測 (Laplacian 方差)
    lap = cv2.Laplacian(gray, cv2.CV_64F)
    blur_score = float(lap.var())
    report['blur_score'] = blur_score
    if blur_score < 100:
```

```
        return False, {
    **report, 'error': '影像模糊'}

# 5. 零件存在性檢查
    _, binary = cv2.threshold(
        gray, 0, 255,
        cv2.THRESH_BINARY + cv2.THRESH_OTSU)
    fg_ratio = np.count_nonzero(binary) \
               / binary.size
    report['foreground_ratio'] = fg_ratio
    if fg_ratio < 0.05 or fg_ratio > 0.95:
        return False, {
            **report,
    'error': '零件可能缺失'}

    report['status'] = 'OK'
    return True, report
```

23.4 假陽性與假陰性分析

假陽性（False Positive）：良品被判為瑕疵，導致過度報廢和返工成本增加。
假陰性（False Negative）：瑕疵品未被檢出而流出，導致客訴和品質問題。

在工業檢測中，假陰性（漏檢）通常比假陽性（誤檢）更嚴重。系統設計時需要找到兩者之間的最佳平衡點。

類型	常見原因	對策
假陽性	閾值太嚴格	放寬閾值，加入特徵過濾
假陽性	光源反光/陰影	改善打光，遮罩已知反光區
假陽性	材料自然紋理	使用變異模型學習紋理
假陽性	對齊偏差	改善對齊精度或加大容差
假陰性	閾值太寬鬆	收緊閾值，多特徵組合
假陰性	瑕疵對比度低	改善照明或使用光度立體
假陰性	瑕疵太小	提高解析度或使用微距鏡頭
假陰性	訓練資料不足	增加訓練樣本和資料增強

23.5 穩健性測試方法

Gauge
R&R（量測系統分析）是評估檢測系統穩健性的標準方法。它量化系統的重複性（同一條件重複檢測的一致性）和再現性（不同條件下的一致性）。

```
[python]
import numpy as np

def gauge_rr(measurements):
 """Gauge R&R 量測系統分析
    measurements: shape (n_operators,
                         n_parts,
                         n_repeats)"""
    n_op, n_parts, n_rep = measurements.shape

 # 重複性 (EV - Equipment Variation)
 # 同一操作員/同一零件的變異
    ranges = []
    for op in range(n_op):
        for part in range(n_parts):
            vals = measurements[op, part, :]
            ranges.append(
                np.max(vals) - np.min(vals))
    r_bar = np.mean(ranges)
 # d2 常數 (n_rep=2: 1.128, 3: 1.693)
    d2 = {2: 1.128, 3: 1.693, 4: 2.059}
    ev = r_bar / d2.get(n_rep, 1.693)

 # 再現性 (AV - Appraiser Variation)
 # 不同操作員之間的變異
    op_means = measurements.mean(
        axis=(1, 2))
    x_diff = np.max(op_means) - \
             np.min(op_means)
    d2_op = d2.get(n_op, 1.693)
    av_sq = max(0,
        (x_diff/d2_op)**2 - ev**2/(n_parts*n_rep))
    av = np.sqrt(av_sq)

 # 總 R&R
    grr = np.sqrt(ev**2 + av**2)

 # 零件變異 (PV)
    part_means = measurements.mean(
        axis=(0, 2))
    pv = np.std(part_means, ddof=1)

 # 總變異
    tv = np.sqrt(grr**2 + pv**2)

    # %GRR
    pct_grr = (grr / tv * 100
               if tv > 0 else 0)

    return {
        'EV': ev, 'AV': av,
        'GRR': grr, 'PV': pv, 'TV': tv,
        '%GRR': pct_grr,
        'status': (
            'PASS' if pct_grr < 10
```

```
            else 'MARGINAL' if pct_grr < 30
            else 'FAIL')
    }
```

23.6 日誌記錄與追溯

完整的日誌系統是產線問題排查的基礎。每次檢測都應記錄：

- 時間戳、序列號/批次號
- 檢測結果、瑕疵類型、分數
- 環境參數（曝光、光源電流、溫度）
- 影像品質指標
- 異常事件

日誌應支援快速查詢和統計分析。

```
[python]
import json
import logging
from datetime import datetime
from pathlib import Path

class InspectionLogger:
 """結構化檢測日誌系統"""

    def __init__(self, log_dir, batch_id):
        self.log_dir = Path(log_dir)
        self.log_dir.mkdir(
            parents=True, exist_ok=True)
        self.batch_id = batch_id

  # 文字日誌
        self.logger = logging.getLogger(
            'inspection')
        self.logger.setLevel(logging.INFO)
        fh = logging.FileHandler(
            self.log_dir / f'{batch_id}.log',
            encoding='utf-8')
        fh.setFormatter(logging.Formatter(
            '%(asctime)s | %(levelname)s '
            '| %(message)s'))
        self.logger.addHandler(fh)

  # JSON 結果記錄
        self.results = []

    def log_result(self, serial, grade,
                   score, defects,
                   quality_info=None):
  """記錄單件檢測結果"""
        record = {
            'timestamp': datetime.now()
                .isoformat(),
```

```
            'batch': self.batch_id,
            'serial': serial,
            'grade': grade,
            'score': round(score, 2),
            'defect_count': len(defects),
            'defects': defects,
        }
        if quality_info:
            record['quality'] = quality_info

        self.results.append(record)
        self.logger.info(
            f'{serial} | {grade} '
            f'| score={score:.1f} '
            f'| defects={len(defects)}')

    def log_alert(self, message):
    """記錄警報"""
        self.logger.warning(
            f'ALERT: {message}')

    def save_summary(self):
    """保存批次彙總"""
        summary_path = (
            self.log_dir /
            f'{self.batch_id}_summary.json')
        with open(summary_path, 'w',
                  encoding='utf-8') as f:
            json.dump({
                'batch': self.batch_id,
                'total': len(self.results),
                'results': self.results
            }, f, ensure_ascii=False,
               indent=2)
```

案例研究：產線環境變化導致的檢測失效排查

問題描述：某金屬零件檢測系統在連續運行 6 小時後，假陽性率從 2% 急升到 15%。

排查過程：
1. 查看日誌：假陽性集中在 14:00 後出現
2. 查看亮度趨勢：平均亮度從 135 緩降至 118
3. 分析原因：LED 光源隨溫度升高（車間無空調）亮度衰減
4. 驗證：在固定閾值下，亮度降低導致更多像素被判定為異常

解決方案：
1. 短期：加入自動曝光補償（AutoExposure 類）
2. 短期：改用自適應閾值取代固定閾值
3. 長期：安裝恆溫光源控制器
4. 長期：每 2 小時用參考片自動校正

效果：假陽性率穩定在 1.5% 以下，全天波動 < 0.5%。

提示

系統穩健性檢查清單：

1. 連續運行 24 小時測試，監控假陽性/假陰性率是否漂移
2. 使用參考片（已知良品和已知瑕疵品）定期驗證
3. 記錄環境參數（溫度、濕度）並與檢測結果交叉分析
4. 對閾值做 +/- 10% 的敏感度測試
5. 模擬異常輸入（過曝、模糊、空影像）確認系統不會崩潰
6. 記錄每個被誤判的案例，定期分析根因並優化
7. 監控影像品質指標，設定預警閾值

注意

不要過度依賴單一條件：

1. 不要僅靠一個閾值做判定——組合多個特徵更穩健
2. 不要假設光源永遠穩定——必須有亮度監控
3. 不要忽略罕見案例——邊界情況在大量生產中必然發生
4. 不要假設新批次材料與舊批次相同——留足適應餘量
5. 不要在沒有日誌的情況下部署——問題排查將極其困難

第 24 章

3D 檢測技術

傳統 2D 影像檢測無法獲取高度和深度資訊。對於翹曲、凹陷、毛邊高度等三維特徵的瑕疵，2D 方法力不從心。3D 檢測技術通過結構光、雷射三角測量或立體視覺獲取物體的三維形貌，實現高度測量和三維表面分析。本章介紹主流的 3D 檢測原理、實作方法和應用場景。

24.1 為什麼需要 3D 檢測

2D 影像是三維場景的投影，丟失了深度維度。以下瑕疵在 2D 影像中難以或無法檢測：
- 翹曲/彎曲：平面零件的微小翹曲在 2D 正視圖中不可見
- 凹陷/凸起：淺凹陷在均勻照明下對比度極低
- 毛邊高度：毛邊存在但高度是否超標需要 3D 量測
- 焊球高度：BGA 焊球的共面性需要精確的 3D 資料
- 密封面平面度：密封溝槽的平面度影響密封性能
- 螺紋深度：螺紋是否完整加工需要深度測量

3D 檢測的代價是更高的硬體成本和較慢的速度。僅在 2D 方法不足時使用。

24.2 結構光三維重建

結構光方法將已知的光柵圖案（條紋、棋盤格或灰碼）投射到物體表面。圖案因物體表面的高度變化而產生形變。相機從另一個角度觀察被形變的圖案，通過三角測量原理計算每個點的三維座標。

常見編碼方式：
- 二進制碼（最簡單，需要 N 張圖案編碼 2^N 條紋）
- 灰碼（Gray Code）：相鄰碼只差 1 位，減少邊界誤差
- 相移法（Phase Shifting）：高精度但需要更多圖案
- 混合方法：灰碼 + 相移，兼顧範圍和精度

公式：三角測量公式

```
Z = f * B / (x_L - x_R)
```

Z = 物體深度 (距基線的距離)
f = 焦距
B = 基線距離 (投影儀與相機間距)
x_L = 投射點在投影儀座標的 x 位置
x_R = 觀測點在相機座標的 x 位置

精度取決於: B (越大越好), f, 和像素解析度
基線越長精度越高，但陰影區域也會增加。實際系統需要在精度和覆蓋率之間取捨。

```
[python]
import cv2
import numpy as np

def decode_gray_code(images, threshold=30):
 """灰碼結構光解碼
 images: list, 每張為 (正碼, 反碼) 元組
 threshold: 二值化閾值"""
    n_bits = len(images)
    h, w = images[0][0].shape[:2]
    gray_code = np.zeros((h, w),
                         dtype=np.uint32)

    for bit_idx, (pos, neg) in \
            enumerate(images):
  # 正碼減反碼，消除環境光
        diff = pos.astype(np.float32) - \
               neg.astype(np.float32)
        bit_map = (diff > threshold)\
            .astype(np.uint32)
        gray_code |= (
            bit_map << (n_bits - 1 - bit_idx))

    # Gray Code -> Binary
    binary_code = gray_code.copy()
    shift = 1
    while shift < n_bits:
        binary_code ^= (binary_code >> shift)
        shift <<= 1

    return binary_code

def triangulate(code_map, calib_params):
 """從編碼圖計算 3D 座標
 code_map:    解碼後的條紋編號
 calib_params: 校準參數"""
    f = calib_params['focal_length']
    B = calib_params['baseline']
    cx = calib_params['cx']
    cy = calib_params['cy']
    pitch = calib_params['stripe_pitch']

    h, w = code_map.shape
    points_3d = np.zeros((h, w, 3),
                         dtype=np.float64)

    for y in range(h):
        for x in range(w):
            x_proj = code_map[y, x] * pitch
            disparity = x_proj - x
            if abs(disparity) < 1:
                continue
            Z = f * B / disparity
            X = (x - cx) * Z / f
            Y = (y - cy) * Z / f
```

```
            points_3d[y, x] = [X, Y, Z]

    return points_3d
```

24.3 雷射三角測量

雷射三角測量使用一條或多條雷射線投射到物體表面。相機從另一角度觀察雷射線，物體表面的高度變化導致雷射線在影像中產生位移。通過標定的幾何關係，位移量可轉換為高度值。

優點：速度快（單線掃描），精度高（可達微米級）。
常用於線掃描場景：零件在輸送帶上移動，雷射線逐行掃描。

公式：高度計算公式（雷射三角測量）

```
h = delta_x * sin(alpha) * sin(beta)
    / sin(alpha + beta)
```

h = 表面高度變化
delta_x = 雷射線在影像中的位移 (像素)
alpha = 雷射入射角
beta = 相機觀測角

簡化（垂直觀測, beta=90 度）:

```
h = delta_x * sin(alpha) * k_calib
```

k_calib 為校準係數，通過已知高度的標準件校準。精度與像素解析度和角度有關。

```
[python]
import cv2
import numpy as np

def extract_laser_line(img,
                       threshold=200):
"""提取雷射線中心位置（亞像素）
img: 灰度影像，雷射線為亮線
回傳: 每列的雷射線 y 座標"""
    h, w = img.shape
    centers = np.full(w, np.nan)

    for col in range(w):
        profile = img[:, col]\
            .astype(np.float64)

  # 找最大值
        peak_idx = np.argmax(profile)
        if profile[peak_idx] < threshold:
            continue

  # 亞像素插值（高斯擬合）
        if (0 < peak_idx < h - 1
                and profile[peak_idx] > 0):
            y_m1 = np.log(
                max(profile[peak_idx-1], 1))
```

```
            y_0 = np.log(
                max(profile[peak_idx], 1))
            y_p1 = np.log(
                max(profile[peak_idx+1], 1))

            denom = 2*(2*y_0 - y_m1 - y_p1)
            if abs(denom) > 1e-6:
                offset = (
                    y_m1 - y_p1) / denom
                centers[col] = (
                    peak_idx + offset)
            else:
                centers[col] = peak_idx

    return centers

def compute_height_profile(centers,
                           ref_centers,
                           k_calib):
 """計算高度輪廓
 centers:    當前掃描的雷射線位置
 ref_centers: 參考平面的雷射線位置
 k_calib:    校準係數 (mm/pixel)"""
    displacement = centers - ref_centers
    height = displacement * k_calib
    return height
```

24.4 立體視覺（雙目相機）

立體視覺使用兩個相機從不同角度同時拍攝。通過匹配兩張影像中的對應點，計算視差（同一物體點在左右影像中的 x 坐標差），進而計算深度。

公式：視差與深度公式

```
Z = f * B / d
```

Z = 深度 (物體到相機的距離)
f = 焦距 (像素)
B = 基線距離 (兩相機間距, mm)
d = 視差 (左影像x - 右影像x, 像素)

d 越大 -> Z 越小 (物體越近)
d 越小 -> Z 越大 (物體越遠)

深度精度: dZ = Z^2 / (f*B) * delta_d
delta_d = 視差精度 (通常 0.1~1 像素)

深度精度與距離的平方成正比。近距離時精度高，遠距離時精度急劇下降。工業應用中通常在近距離（<1m）使用。

```
[python]
import cv2
import numpy as np

def stereo_depth(img_left, img_right,
                  focal_length, baseline):
"""雙目立體視覺深度計算
img_left/right: 已校正的灰度影像
focal_length: 焦距 (像素)
baseline: 基線距離 (mm)"""
# 建立 StereoSGBM 匹配器
    min_disp = 0
num_disp = 128 # 必須是 16 的倍數
    block_size = 5

    stereo = cv2.StereoSGBM_create(
        minDisparity=min_disp,
        numDisparities=num_disp,
        blockSize=block_size,
        P1=8 * block_size**2,
        P2=32 * block_size**2,
        disp12MaxDiff=1,
        uniquenessRatio=10,
        speckleWindowSize=100,
        speckleRange=32,
        mode=cv2.STEREO_SGBM_MODE_SGBM_3WAY
    )

# 計算視差圖
    disparity = stereo.compute(
        img_left, img_right)
    disp_float = disparity.astype(
        np.float32) / 16.0

# 視差 -> 深度
    depth = np.zeros_like(disp_float)
    valid = disp_float > 0
    depth[valid] = (
        focal_length * baseline
        / disp_float[valid])

    return depth, disp_float
```

24.5 點雲處理與瑕疵檢測

3D 掃描的結果通常是點雲（大量 3D 坐標點的集合）。對點雲進行平面擬合、曲面擬合，計算每個點到擬合面的偏差，偏差超過閾值的區域即為瑕疵（凹陷、凸起、翹曲）。

```
[python]
import numpy as np

# 以下為 Open3D 點雲處理範例
# import open3d as o3d

def fit_plane(points):
"""最小二乘平面擬合
    points: Nx3 array
回傳: (a, b, c, d) 使得 ax+by+cz+d=0"""
    centroid = points.mean(axis=0)
    centered = points - centroid

# SVD 分解
    _, _, Vh = np.linalg.svd(centered)
normal = Vh[-1]  # 最小奇異值的向量
    a, b, c = normal
    d = -np.dot(normal, centroid)
    return a, b, c, d

def compute_deviation(points, plane):
"""計算每個點到平面的偏差"""
    a, b, c, d = plane
    norm = np.sqrt(a**2 + b**2 + c**2)
    distances = (
        a*points[:,0] + b*points[:,1]
        + c*points[:,2] + d) / norm
    return distances

def detect_3d_defects(points,
                      threshold=0.05):
"""3D 瑕疵檢測
points:   Nx3 點雲
threshold: 偏差閾值 (mm)"""
# 擬合參考平面
    plane = fit_plane(points)
    deviations = compute_deviation(
        points, plane)

# 分類偏差
    defects = {
        'bumps': points[
            deviations > threshold],
        'dents': points[
            deviations < -threshold],
        'flatness': float(
            np.max(deviations)
            - np.min(deviations)),
        'max_bump': float(
            np.max(deviations)),
        'max_dent': float(
            np.min(deviations)),
    }

    return defects, deviations
```

24.6 3D 瑕疵類型與檢測方法

瑕疵類型	量測指標	檢測方法	精度要求
翹曲/彎曲	平面度偏差	結構光/雷射掃描	~10 um
凹陷/凹坑	深度值	結構光/雷射	~5 um
毛邊高度	邊緣高度	雷射三角測量	~10 um
焊球高度	共面性	結構光+多角度	~5 um
螺紋深度	溝槽深度	雷射掃描	~20 um
表面粗糙度	Ra/Rz	白光干涉儀	~0.01 um
體積缺損	缺失體積	多角度掃描	~50 um

案例研究：BGA 焊球高度檢測

場景：BGA（Ball Grid Array）封裝的焊球共面性檢測。焊球高度偏差會導致虛焊。

規格要求：
- 焊球高度：300 +/- 30 um
- 共面性（Coplanarity）：< 50 um
- 直徑：350 +/- 25 um

檢測方案：
1. 硬體：高精度結構光系統（Z 解析度 2 um）
2. 投射 8 位灰碼 + 4 步相移圖案
3. 重建焊球頂面的 3D 點雲
4. 對每個焊球擬合球面，提取頂點高度
5. 擬合基準平面（PCB 表面）
6. 計算每個焊球相對基準面的高度
7. 共面性 = max(h) - min(h)

判定：
- 任一焊球高度超出 300 +/- 30 um -> NG
- 共面性 > 50 um -> NG
- 焊球缺失 -> NG
- 焊球直徑異常 -> NG

結果：漏檢率 < 0.01%，檢測速度 0.5 秒/件。

提示

3D 精度影響因素：
1. 校準品質：使用高精度標準件，定期重新校準
2. 環境振動：使用防振平台，避免量測時產生振動
3. 光學解析度：像素尺寸和景深限制了空間解析度
4. 材料表面特性：反光面需要使用漫射噴霧或特殊照明
5. 溫度：精密量測需控制在恆溫環境
6. 掃描密度：點與點之間距離越小，重建越精確但速度越慢

注意

3D 檢測的成本與限制：

1. 硬體成本高：結構光系統通常比 2D 相機貴 5-20 倍
2. 速度較慢：結構光需投射多張圖案，雷射需逐線掃描
3. 反光面問題：金屬鏡面、透明材料的 3D 重建非常困難
4. 校準複雜：3D 系統的校準比 2D 複雜得多，需要專業知識
5. 資料量大：點雲資料量遠大於 2D 影像，存儲和處理成本高
6. 不是所有瑕疵都需要 3D——先確認 2D 方法確實不足再考慮 3D

第 25 章

參數調校方法論

瑕疵檢測系統的效能高度依賴參數設定：閾值、核大小、面積範圍、信心分數等。錯誤的參數會導致漏檢或誤檢。本章介紹系統化的參數調校方法論，從 ROC 曲線分析到貝葉斯優化，幫助工程師科學地找到最佳參數組合，而非依賴經驗猜測。

25.1 系統化參數調校流程

有效的參數調校遵循以下流程：

1. 識別關鍵參數：列出所有可調參數，識別對檢測結果影響最大的 3-5 個
2. 準備驗證資料：收集一組有標註的良品和瑕疵品影像（至少各 50 張）
3. 設計實驗：系統化地掃描參數範圍
4. 評估效果：計算 Precision, Recall, F1 等指標
5. 選擇最佳組合：ROC 曲線、Youden's J 或業務導向的選擇
6. 驗證穩定性：交叉驗證確認結果不是偶然
7. 凍結參數：確定後鎖定，版本管理

關鍵原則：永遠在獨立的測試集上評估，避免在調參集上過擬合。

25.2 ROC 曲線與最佳閾值選擇

ROC（Receiver Operating Characteristic）曲線展示了不同閾值下的 TPR（真陽性率）和 FPR（假陽性率）之間的取捨關係。AUC（曲線下面積）衡量分類器的整體性能。

公式：TPR, FPR, AUC

```
  TPR (Recall) = TP / (TP + FN)
 = 實際瑕疵中被正確檢出的比例

  FPR = FP / (FP + TN)
 = 實際良品中被誤判的比例

AUC = 曲線下面積
 AUC = 1.0 : 完美分類器
 AUC = 0.5 : 隨機猜測

  Youden's J = TPR - FPR
 最佳閾值 = argmax(J)
```

Youden's J 同時最大化檢出率和最小化誤報率。但在工業檢測中，可能需要以高 Recall 為優先（寧可誤報不可漏報）。

```
[python]
import numpy as np
from sklearn.metrics import roc_curve, auc
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

def plot_roc_find_threshold(y_true, y_scores,
                            save_path=None):
"""繪製 ROC 曲線並找最佳閾值
y_true:  真實標籤 (0=良品, 1=瑕疵)
y_scores: 分數 ( 越高越可能是瑕疵 ) """
    fpr, tpr, thresholds = roc_curve(
        y_true, y_scores)
    roc_auc = auc(fpr, tpr)

# Youden's J 最佳閾值
    j_scores = tpr - fpr
    best_idx = np.argmax(j_scores)
    best_thresh = thresholds[best_idx]
    best_tpr = tpr[best_idx]
    best_fpr = fpr[best_idx]

# 高 Recall 閾值 (TPR >= 0.99)
    high_recall_idx = np.where(
        tpr >= 0.99)[0]
    if len(high_recall_idx) > 0:
        hr_idx = high_recall_idx[0]
        hr_thresh = thresholds[hr_idx]
    else:
        hr_thresh = thresholds[0]

# 繪圖
    plt.figure(figsize=(8, 6))
    plt.plot(fpr, tpr,
             label=f'AUC={roc_auc:.4f}')
    plt.plot(best_fpr, best_tpr, 'ro',
             label=f'Best (J={best_thresh:.3f})')
    plt.plot([0,1], [0,1], 'k--')
    plt.xlabel('FPR')
    plt.ylabel('TPR')
    plt.title('ROC Curve')
    plt.legend()
    if save_path:
        plt.savefig(save_path, dpi=150)
    plt.close()

    return {
        'auc': roc_auc,
        'best_threshold': float(best_thresh),
        'best_tpr': float(best_tpr),
        'best_fpr': float(best_fpr),
        'high_recall_threshold': float(
            hr_thresh)
    }
```

25.3 交叉驗證

在小資料集上調參容易過擬合（參數恰好適合這組資料但泛化不佳）。交叉驗證將資料分成 K 份，輪流使用其中 K-1 份調參、1 份驗證，取 K 次結果的平均值作為最終評估。

瑕疵檢測中的注意事項：
- 使用分層抽樣（Stratified）確保每份中瑕疵比例一致
- 同一零件的多張影像不能分到不同份中（避免資料洩漏）

```
[python]
from sklearn.model_selection import (
    StratifiedKFold, cross_val_score
)
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
import numpy as np

def cross_validate_classifier(X, y, n_folds=5):
 """K-fold 交叉驗證
 X: 特徵矩陣 (n_samples, n_features)
 y: 標籤 (0=OK, 1=NG)"""
 # 建立管線
    pipeline = Pipeline([
        ('scaler', StandardScaler()),
        ('svm', SVC(kernel='rbf', C=10,
                    gamma='scale',
                    probability=True))
    ])

 # 分層 K-fold
    cv = StratifiedKFold(
        n_splits=n_folds, shuffle=True,
        random_state=42)

 # 多指標評估
    metrics = {}
    for metric in ['accuracy', 'precision',
                   'recall', 'f1']:
        scores = cross_val_score(
            pipeline, X, y, cv=cv,
            scoring=metric)
        metrics[metric] = {
            'mean': float(np.mean(scores)),
            'std': float(np.std(scores)),
            'scores': scores.tolist()
        }

    return metrics
```

25.4 敏感度分析

敏感度分析評估每個參數對檢測結果的影響程度。了解哪些參數最敏感，有助於集中精力調校關鍵參數，並設定合理的容差範圍。

```
[python]
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

def sensitivity_analysis(evaluate_fn,
                         params_range,
                         base_params):
"""單因子敏感度分析
evaluate_fn: 評估函數，輸入參數，回傳分數
params_range: dict {參數名: [值列表]}
base_params: 基準參數 dict"""
    results = {}

    for param_name, values in \
            params_range.items():
        scores = []
        for val in values:
            test_params = base_params.copy()
            test_params[param_name] = val
            score = evaluate_fn(test_params)
            scores.append(score)
        results[param_name] = {
            'values': values,
            'scores': scores,
            'sensitivity': (
                max(scores) - min(scores))
        }

# 繪製敏感度圖
    n = len(results)
    fig, axes = plt.subplots(
        1, n, figsize=(4*n, 3))
    if n == 1:
        axes = [axes]

    for ax, (name, data) in \
            zip(axes, results.items()):
        ax.plot(data['values'],
                data['scores'], 'bo-')
        ax.set_xlabel(name)
        ax.set_ylabel('F1 Score')
        ax.set_title(
            f'Sensitivity: '
            f'{data["sensitivity"]:.3f}')
    plt.tight_layout()
    plt.savefig('sensitivity.png', dpi=150)
```

```
    plt.close()

    return results
```

25.5 自動參數搜索

手動調參效率低且難以找到全局最優。自動搜索方法可系統化地探索參數空間。

三種主要方法：
- 網格搜索（Grid Search）：遍歷所有組合。簡單但組合爆炸。
- 隨機搜索（Random Search）：隨機採樣。在高維空間中效率高於網格搜索。
- 貝葉斯優化（Bayesian Optimization）：根據已有結果智能選擇下一個試驗點。最高效。

```
[python]
import optuna
import numpy as np

def optimize_parameters(images, labels):
 """使用 Optuna 貝葉斯優化搜索最佳參數"""

    def objective(trial):
  # 定義搜索空間
        blur_k = trial.suggest_int(
            'blur_kernel', 3, 11, step=2)
        thresh_c = trial.suggest_int(
            'threshold_c', 5, 30)
        morph_k = trial.suggest_int(
            'morph_kernel', 3, 9, step=2)
        min_area = trial.suggest_int(
            'min_area', 20, 500, step=20)
        block_size = trial.suggest_int(
            'block_size', 11, 101, step=10)

  # 用這組參數檢測所有影像
        tp = fp = fn = tn = 0
        for img, label in \
                zip(images, labels):
            pred = detect_with_params(
                img, blur_k, thresh_c,
                morph_k, min_area,
                block_size)
            if pred and label:     tp += 1
            elif pred and not label: fp += 1
            elif not pred and label: fn += 1
            else:                  tn += 1

  # 計算 F1
        prec = tp/(tp+fp) if tp+fp else 0
        rec = tp/(tp+fn) if tp+fn else 0
        f1 = (2*prec*rec/(prec+rec)
              if prec+rec else 0)

        return f1
```

```
# 執行優化
    study = optuna.create_study(
        direction='maximize',
        sampler=optuna.samplers.TPESampler(
            seed=42))
    study.optimize(objective, n_trials=200,
                   show_progress_bar=True)

    print(f'Best F1: '
          f'{study.best_value:.4f}')
    print(f'Best params: '
          f'{study.best_params}')

    return study.best_params
```

25.6 參數驗證與版本管理

確定最佳參數後，必須凍結並納入版本管理。參數應以配置檔形式存儲，而非硬編碼在程式中。每次參數變更都應記錄原因和效果。

```
[python]
import yaml
from pathlib import Path
from datetime import datetime

class ParamManager:
 """參數配置管理系統"""

    def __init__(self, config_dir):
        self.config_dir = Path(config_dir)
        self.config_dir.mkdir(
            parents=True, exist_ok=True)

    def save_config(self, params, version,
                    description=''):
  """保存參數配置"""
        config = {
            'version': version,
            'created': datetime.now()
                .isoformat(),
            'description': description,
            'parameters': params
        }
        path = (self.config_dir /
                f'config_v{version}.yaml')
        with open(path, 'w',
                  encoding='utf-8') as f:
            yaml.dump(
                config, f,
                allow_unicode=True,
                default_flow_style=False)
        return path
```

```
    def load_config(self, version=None):
    """載入參數配置
    version=None 時載入最新版本"""
        if version:
            path = (self.config_dir /
                    f'config_v{version}.yaml')
        else:
            files = sorted(
                self.config_dir.glob(
                    'config_v*.yaml'))
            if not files:
                raise FileNotFoundError(
                    'No config found')
            path = files[-1]

        with open(path, 'r',
                  encoding='utf-8') as f:
            config = yaml.safe_load(f)
        return config

# 使用範例
pm = ParamManager('./configs')

# 保存最佳參數
best_params = {
    'preprocessing': {
        'blur_kernel': 5,
        'clahe_clip': 2.0,
        'clahe_grid': 8
    },
    'segmentation': {
        'method': 'adaptive',
        'block_size': 51,
        'threshold_c': 10
    },
    'morphology': {
        'kernel_size': 5,
        'open_iter': 1,
        'close_iter': 1
    },
    'classification': {
        'min_area': 100,
        'max_area': 50000,
        'min_circularity': 0.3,
        'score_threshold': 40
    }
}

pm.save_config(
    best_params, version='1.2.0',
 description='Optuna 優化後參數')
```

參數	推薦搜索範圍	步進	敏感度
高斯核大小	3 ~ 11	2	中
自適應閾值 C	5 ~ 30	1	高
自適應 blockSize	11 ~ 101	10	高
形態學核大小	3 ~ 9	2	中
最小面積	20 ~ 500	20	高
圓度閾值	0.2 ~ 0.9	0.05	中
差分閾值	10 ~ 80	5	極高
z-score 倍數 k	2.0 ~ 5.0	0.5	高
SSIM 閾值	0.8 ~ 0.99	0.01	中

案例研究：Otsu 閾值 vs 自適應閾值的系統化比較

場景：金屬零件表面刮痕檢測，照明存在輕微不均勻。

實驗設計：
- 驗證資料集：200 張（100 良品 + 100 瑕疵品，標註人員交叉驗證）
- 方法 A：Otsu 全域閾值
- 方法 B：自適應高斯閾值（掃描 blockSize 和 C）
- 評估指標：F1、Recall、Precision

結果：
- Otsu 最佳 F1 = 0.82 (Recall=0.78, Precision=0.87)
- 自適應閾值最佳 F1 = 0.93 (blockSize=41, C=12)
  (Recall=0.95, Precision=0.91)

分析：
- Otsu 在照明均勻區域表現好，但不均勻區域漏檢嚴重
- 自適應閾值在所有區域都穩定
- blockSize 對結果影響最大（敏感度 0.15）
- C 值的影響次之（敏感度 0.08）

決策：選用自適應閾值，參數 blockSize=41, C=12，經 5-fold 交叉驗證確認穩定。

提示

參數調校的經驗法則：
1. 先固定大部分參數，一次只調一個——理解每個參數的獨立影響
2. 從粗搜索開始（大步進），縮小範圍後再細搜索
3. 始終在獨立測試集上評估——不要在調參集上看結果
4. 記錄每次實驗的參數和結果——方便回溯
5. 最終參數需通過交叉驗證確認穩定性
6. 留安全餘量：如果最佳閾值為 42，設 40 或 45 測試結果是否仍可接受
7. 考慮業務需求：是寧可誤報（高 Recall）還是寧可漏報（高 Precision）

注意

過度擬合參數的風險：

1. 參數完美匹配當前資料集，但換一批材料就失效
2. 同時優化太多參數導致交互作用複雜，難以理解和維護
3. 使用太少的驗證資料（< 30 張），結果不具統計意義
4. 忽略極端案例：調校時遇到的最難樣本決定了實際表現
5. 不記錄參數版本：無法追溯哪個版本在產線上運行
6. 調校一次後永不更新：產品和環境在變化，參數也需要定期驗證